



Automata Formal Languages & Logic

UE24CS243A

Divyaprabha K N

Department of Computer Science & Engineering

Acknowledgement: Preet Kanwal

Undecidability

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

The Church Turing Thesis:

A **computable function** is a function that a Turing Machine can compute.

An **algorithm** is defined simply as a TM that does not enter into an infinite loop for any input.

- It must terminate after a finite number of steps in either in final or non-final state.
- It is a systematic procedure

Alonzo Church and Alan Turing showed anything that can be computed can be computed by Turing Machine.

- They independently showed the result (Alonzo church came up with Lambda Calculus).
- This statement has no proof and it's not a theorem. However no one so far has been able to find a computing problem that TM cannot compute.
- We can also say, only functions that are computable by TM are computable.

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

The Church Turing Thesis:

- Consider the set of all possible Turing Machines $= \{M_1, M_2, M_3, \dots\}$ and set of all languages accepted by each Turing Machine $= \{L(M_1), L(M_2), \dots\}$ this set of languages corresponds to the set of all computable functions or we can say, set of all valid computer programs that one could write. This is the largest class of formal languages that are computable and are called Recursively Enumerable Languages.
- Multiple attempts have been made to enhance / modify the design of TM to make it more powerful.
- However all the resulting variations turned out to be strictly equivalent to the standard Turing machine in terms of the set of the functions that it can complete thereby supporting Church Turing Thesis.

The Universal Turing Machine

- Until now , we have build TM for each computable function.
- The TM which performs addition operation, cannot multiply and vice versa.
- Turing overcame this apparent limitation and showed how a single computing machine can capture / compute any computable function.
- **It is called Universal Turing Machine.**
- A UTM M_u simulate the computation of any TM M on a given input w .
- M_u is similar to the compiler software which can compile other programs.

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

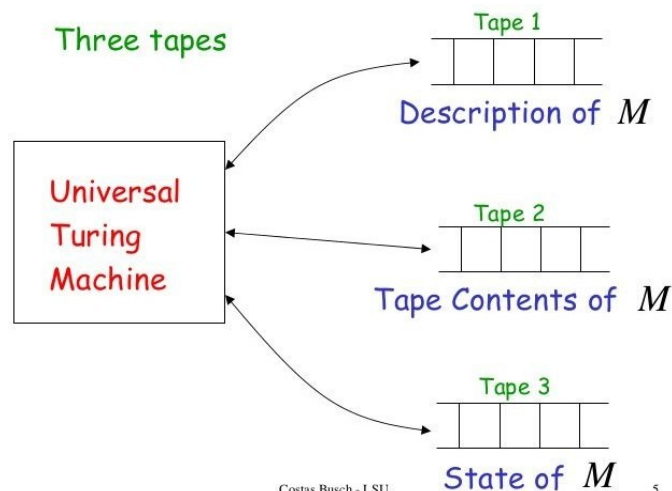
The Universal Turing Machine

U_M is a three tape machine (3 read/write heads)

Tape 1: Compiled program tape contains encoding of the TM M (Description of M)

Tape 2: Input-output tape

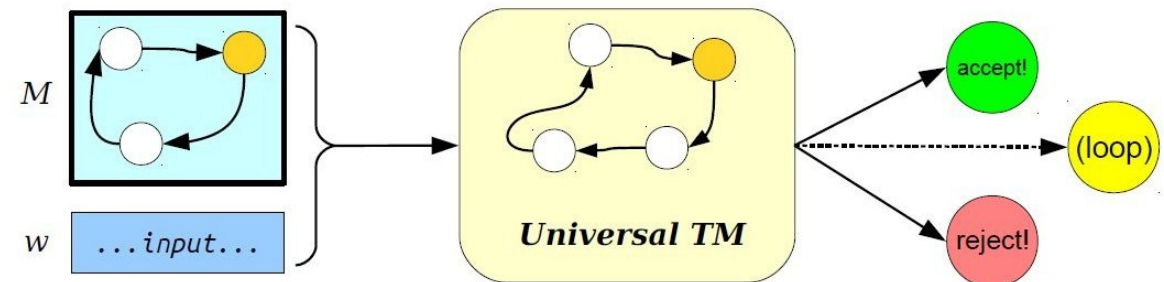
Tape 3: Working memory tape, contains current state of TM M being simulated



Costas Busch - LSU

5

- The observable behavior of U_{TM} is the following:
 - If M accepts w , then U_{TM} accepts $\langle M, w \rangle$.
 - If M rejects w , then U_{TM} rejects $\langle M, w \rangle$.
 - If M loops on w , then U_{TM} loops on $\langle M, w \rangle$.
- U_{TM} accepts $\langle M, w \rangle$ if and only if M accepts w .**



Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

The Universal Turing Machine

Working of M_u

- Look at current state in working memory and current symbol on input-output tape.
- With information of current state and current symbol, scan the compiled program tape to find a transition for the given configuration.
- Execute the transition i.e. write new symbol on input-output tape and move its read/write head.
- Repeat above steps until there are no more transitions in compiled program.
- It halts and accepts or halts and rejects.

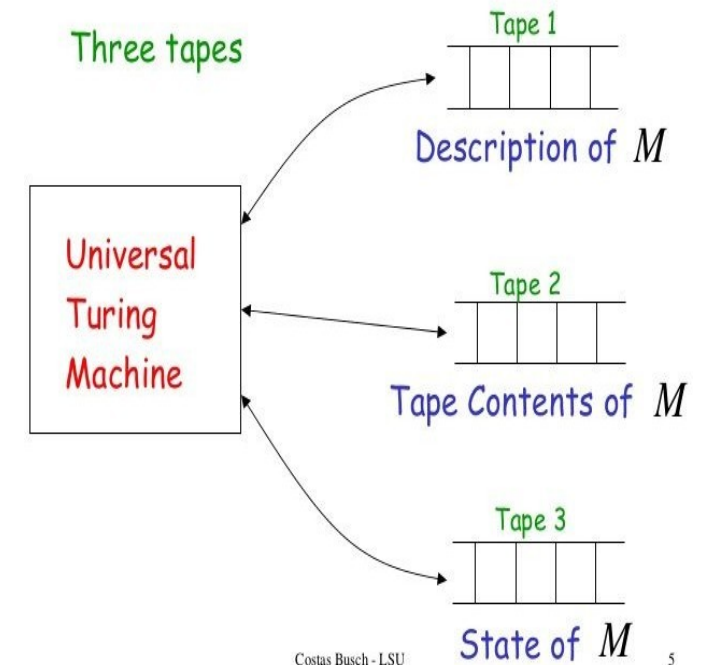
We require a way to encode transition of M as a binary string.

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

The Universal Turing Machine

- This idea of Universal TM has been implemented in modern digital computers.
- Programs written in High Level Language are compiled by a compiler.
- Turing's idea of storing an encoded table of instructions (i.e. program) of a TM on one of the tapes of UTM, made it possible for us to build modern computers that can compute practically anything that is computable.
- Also Turing machines for simple computable functions can be combined to compute more complex functions. This is possible as we can easily concatenate encoded programs of multiple Turing machines to create a program for a complex Turing machine.
- This is similar to what we do today in the name of modularity in programming.



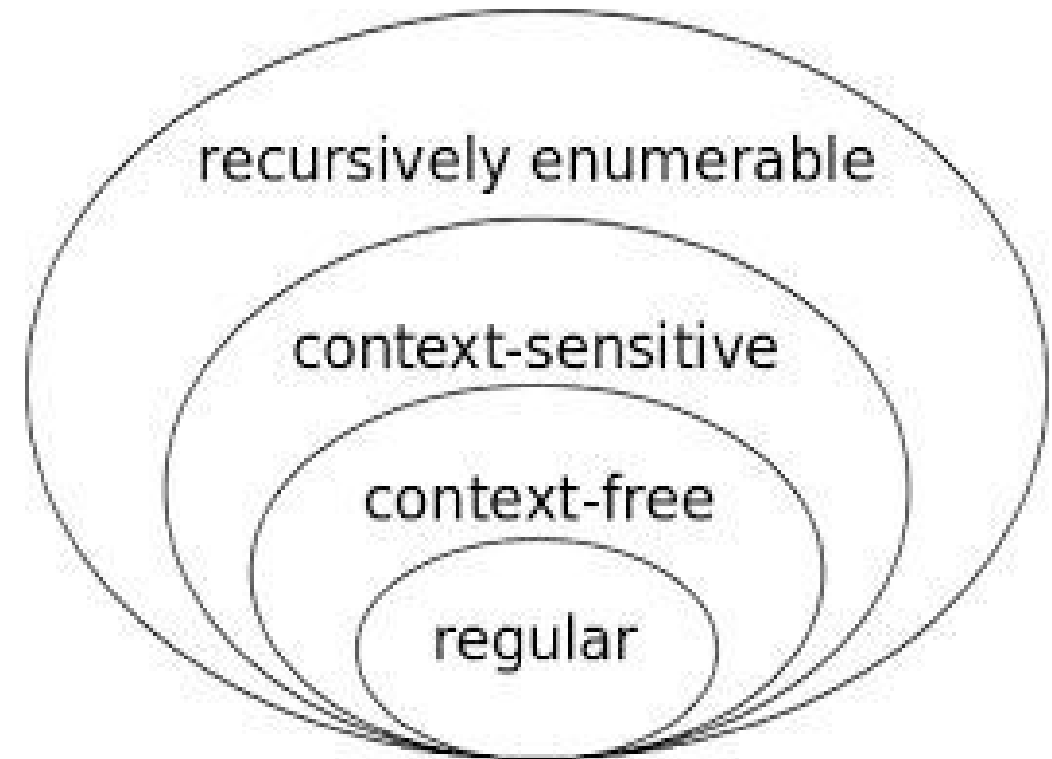
Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

The Chomsky Hierarchy

The big picture of formal languages, grammar and computing machines known as the Chomsky hierarchy.

We have already looked at two main classes of formal languages namely, Regular Languages and Context Free Languages. Turing machines are way more powerful than PDA's in terms of solving more problems.



Language of Turing Machine is called Recursively Enumerable Language or Semi decidable or partially decidable or Turing Recognizable.

Recursively enumerable Language:

A language is said to be REL if

- It is accepted by Turing Machine
- If there exists a procedure / algorithm that can enumerate all the members of the set.
- If we can speak about the i^{th} element of the set.
- If we can generate any n^{th} element in finite amount of time.
- If we can map elements of the set with the set of natural numbers with one to one correspondence.

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

Recursively enumerable Language

Examples:

1) Set of all Natural Numbers is enumerable.

We can write a program to enumerate the set of all natural numbers N .

i.e. we can build a TM for enumerating natural numbers for example by writing each successive number on to its tape

Such a program will obviously never terminate because the set N is infinite, the machine will not halt. However, give a particular finite number 'n' irrespective of how large 'n' is it will be written in finite time. Hence this set is enumerable.

Note: Any set which can be placed in one correspondence with the set of natural numbers 'N' (i.e. we can index each element, it makes sense to talk to ith element of the set) is called countably infinite.

2) Set of all even numbers is enumerable.

$$N=\{1,2,3,4,\dots\}$$

$$E=\{2,4,6,8,\dots\}$$

$$f(x)=2x$$

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

3) Σ^* is enumerable

Σ^* is the set of all strings over the alphabet. It is a countably infinite set, that means we have a way to enumerate strings $\in \Sigma^*$

Σ^* enumerable procedure:

- We would print strings in the language $L = \Sigma^*$ in alphabetical order. (lexicographic order)
 - For example if $\Sigma = \{a, b, c\}$
 - As per our enumeration procedure, we print $\{\lambda, a, aa, aaa, aaaa, \dots\}$
 - we will never be able to print strings that begin with b or c
 -
- Hence we must print the strings in proper order i.e. by order of increasing length.
 - $\{\lambda, a, b, c, aa, ab, ac, ba, bb, bc, ca, cb, cc, aaa, aab, \dots\}$
- If we can enumerate any such set, we can immediately talk about ith element of the set and thereby map the elements to the set of natural numbers.
- In this case, given any string of finite length, $w \in \Sigma^*$, we will be able to print w in finite time following our enumeration procedure.

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

4) Set of all Turing Machines are enumerable

- We know from our study of UTM that a TM can be encoded as a binary string
- Binary alphabet $\Sigma = \{0,1\}$
 - Set of all binary strings is Σ^* i.e. $(0+1)^*$
 - Σ^* is an infinite set
- we must note that , not all binary strings are valid encodings of TMs
 - With this we can conclude that set of Turing Machines is proper subset of Σ^*
- Also, TM always has a finite description as there are a finite number of transitions and tape symbols.
- Since we have a way to enumerate all strings in Σ^* . We can enumerate all TM's. Since every Turing Machine can be encoded as a finite string over a finite alphabet, the set of all TM's is enumerable.

5) $Q = \{m/n \mid m, n \in \mathbb{N}\}$ is a set of positive rational numbers . Q is countable

Proof:

We provide a correspondence with $\mathbb{N}$. we can list all members and elements of Q . we can make an infinite matrix containing all positive rational numbers.

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

Row headings -> indicates numerator

Column headings -> indicates denominator

With Row(i) column(j) we get, rational numbers i/j .

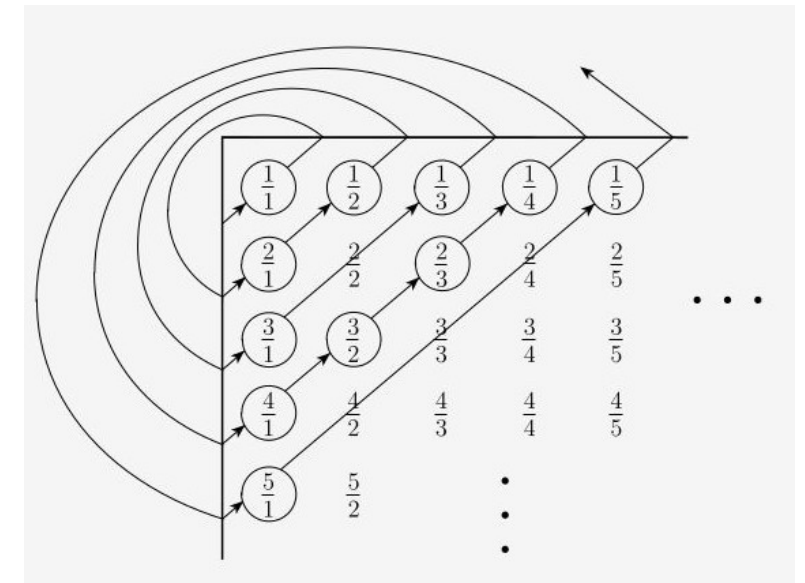
This matrix can be turned into a list. List of all elements on the diagonals starting from the corner.

First diagonal contains a single element $1/1$.

Second diagonal contains $2/1, 1/2$.

Third diagonal, a complication arises, it contains $3/1, 2/2, 1/3$, we see $1/1 = 2/2$, we must skip duplicate elements.

Hence our list will have: $1/1, 2/1, 1/2, 3/1, 1/3, \dots$



Conclusion:

- All the above sets are infinite as well as RE sets and have the same size (cardinality) as that of a set of natural numbers and also, we are able to keep the elements from both the sets in one to one correspondence.
- **Cantor proposed this idea to compare or measure the size of infinite sets.**
- Cantor's idea of counting the elements, pairs with the elements of the other sets.
- **Note: A set is countable if either it is finite or it has same size as $\mathbb{N}$**

Non-Recursively Enumerable Sets

Do all infinite sets have the same size / cardinality?

Are there sets that are not enumerable (cannot be kept in one to one correspondence with sets of Natural numbers?)

Is it same as saying: Are there sets for which no TM enumerator can be constructed?

The answer is yes. Such sets are called Non-Recursively Enumerable.

1) Set of real numbers $\mathbb{R}$ is not enumerable.

- Consider the set of real numbers. Try enumerating all elements between 1 and 2, 1.0, 1.00, 1.000, ... , 1.1, 1.10, 1.100...
- We face trouble listing the elements unless we place a limit on the precision of the numbers. If the precision is limited, we can enumerate all real numbers. We can write the numbers ordered by length first and then by value.
- Example: 0.1, 1.0, 0.2, 0.3, ... 0.9, 1.0, ... 9.9, 0.01, 0.02, ...
- Print all the numbers that have one decimal place only, then the numbers within two decimal places and so on... But if precision is unlimited we cannot ever enumerate a real number in finite time in any attempt.
- Hence such a set is not enumerable,

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

- There doesn't exist any algorithm to enumerate all elements of the set
- It doesn't make sense to talk about i^{th} real number
- There are infinitely many elements between two elements of the set
- We cannot enumerate any element of the set in finite time.
 - Such sets are also called as uncountably infinite, such sets have "too many elements"
- Formal proof to show to show that set R is not enumerable:
 - Proof uses the technique called **Cantor's Diagonalization**
 - It is a proof by contradiction
 - **Our Aim:** our aim is to set up a contradiction to prove that our assumption is wrong.
 - **Assumption:** Set R is enumerable
 - Hence we enumerate all elements of R, let's say

n	F(n)
1	3.14567
2	7.82123.
3	3.41222
4	2.15679
5	5.78255
.	
.	
.	

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

- This table has an infinite number of rows and columns
- However we can talk about 1st real number , 2nd real number and so on
- Consider the left to right diagonal which contains the 1st decimal place of first real number (1 in our case) 2nd decimal place of 2nd real number (2 in our case) and so on . If we read this diagonal as a real number, we get $R_d = .12275$
- Let us insert these element (you could use any procedure may add a 2, subtract 1, add 3, add 4 etc..) basically choose some digit between 1 and 8) in our case we add 1 (to each digit) we get,
- $R_d' = .23386$. Result is also a real number.

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

- Question is, Does this inverted diagonal number R_d' exist in the enumerated list? i.e. is it the same as one of the rows in the table we have constructed?
- We see that it is not same as first row since it differs in first decimal place, it's not same as second number as it differs in 2nd decimal place, it is not same as i^{th} row, as it differs in i^{th} decimal place. It differs from every enumerated real number in the corresponding diagonal element (i.e. i^{th} decimal place). But, we stated with the assumption that the real number in set R has been enumerated in the table. Yet we got a new number. This is a contradiction, hence our assumption is wrong.
- We have proved our assumption is wrong using the diagonalization method. Hence, Set R is not enumerable

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

2) Set of all languages are not enumerable

- We can use the same method of diagonalization to prove that sets of all languages are not enumerable. Any formal language L is a subset of Σ^* $L \subseteq \Sigma^*$, we must remember that Σ^* is enumerable(countably infinite)
- However, a set of formal languages is not enumerable (uncountably infinite). Since any language is a subset of Σ^*
- Total number of subsets that that can be formed using the set Σ^* is **nothing but the power set of $\Sigma^* = 2^{\Sigma^*}$** .
- Formal Proof: set of all languages over Σ are enumerable. This means the languages can be numbered as $L_1, L_2, L_3, \dots$
- Let's construct a 2-D matrix with an infinite number of rows
- Rows-> are enumerated as languages from the power set 2^{Σ^*} (each row represents a language).we can also see the rows as enumeration of Turing Machines with row L_i corresponding to the language of the i th Turing Machine.
- Columns-> Columns in the matrix are all the strings from $\Sigma^* = \{s_1, s_2, s_3, \dots\}$ (we know that Σ^* is enumerable)

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

1 indicates string $s_i \in L_j$, and 0 indicates string $s_i \notin L_j$,

$L_1 = \{s_1, s_3, s_4, \dots\}$

$L_2 = \{s_1, s_2, s_3, \dots\}$

$L_3 = \{s_2, \dots\}$

$L_4 = \{s_1, \dots\}$

$L_{15} = \{s_1, s_2, \dots\}$

Consider left to right diagonal, this will represent another language let's say L_{diag}

Thus, $L_{diag} = \{s_1, s_2, \dots\}$

This language L_{diag} must be one of the enumerated languages (must be same as one of the rows) In this case

$L_{diag} = L_{15}$

Thus the language is computable and recursively enumerable since we can construct

Turing machine for such a language (as per our assumption)

Complement this language, what we get is a new language $L_{non_RE} = L_{diag}'$

$L_{non_RE} = \{s_3, s_4, \dots\}$

This new language is not the same as any of the enumerated elements (rows) because it differs from i^{th} diagonal place. Hence it is a new language and not the same as any of the enumerated languages.

This is contradiction; hence our assumption that all languages are enumerable is wrong.

	S1	S2	S3	S4.....
L1	1	0	1	1
L2	1	1	0	1
L3	0	1	0	0
L4	1	0	0	0
.				
.				
L15	1	1	0	0

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

Note: From our previous study of Universal Turing Machine, we know that we can encode all the transitions of a Turing Machine as a binary string. Since we can enumerate all possible binary strings (i.e. Σ^* where $\Sigma=\{0,1\}$). Hence, we can enumerate all Turing Machines as each Turing Machine has a finite description and can be encoded as a unique binary string. The total number of possible Turing Machines is less than size of Σ^* for the binary alphabet. So we can just enumerate all possible strings and discard invalid encodings.

There are countably infinite numbers of Turing Machines in the world. But there are uncountably infinite number of formal languages(we just proved)

Hence, There exists a formal language for which we cannot construct a TM i.e. there exists a language which is not Recursively Enumerable.

Set of Recursively Enumerable Languages is a proper subset of a set of all formal languages.

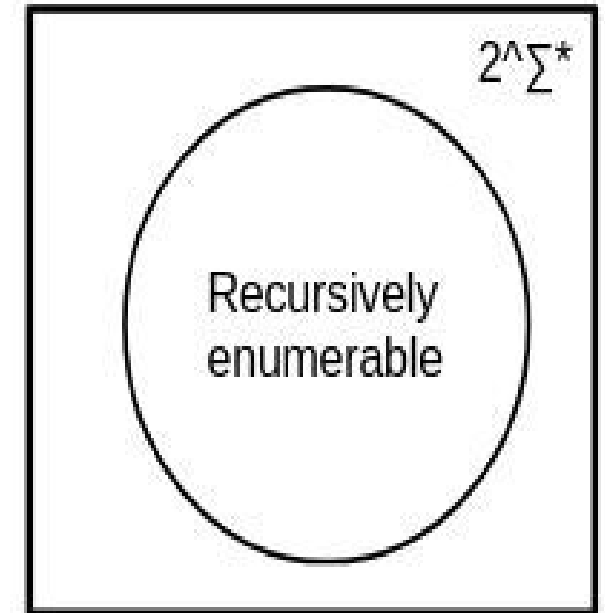
Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

Not Recursively Enumerable means not computable.

Most languages are not Recursively Enumerable but that's okay, as these innumerable, uncountable languages are not interesting.

Not only computable, most of the times these languages have no direct description. Because if we could describe directly, we could convert such a description to an algorithm and there by a Turing machine.



Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

The Idea of Acceptance and Membership

We know that languages accepted by TM are called RE Languages.

If a Turing Machine halts on every input it is called TM decider. Language of TM Decider is Recursive language.

Such TM's complete membership of strings in the language i.e.

For any given string $w \in \Sigma^*$.

"A Turing machine decider halts and accepts if $w \in L(M)$, or halts and rejects if $w \notin L(M)$."

If the language has a membership function its complement also has a membership function. Such languages are called recursive languages or Turing decidable languages. Recursive languages are a proper subset of recursively enumerable languages.

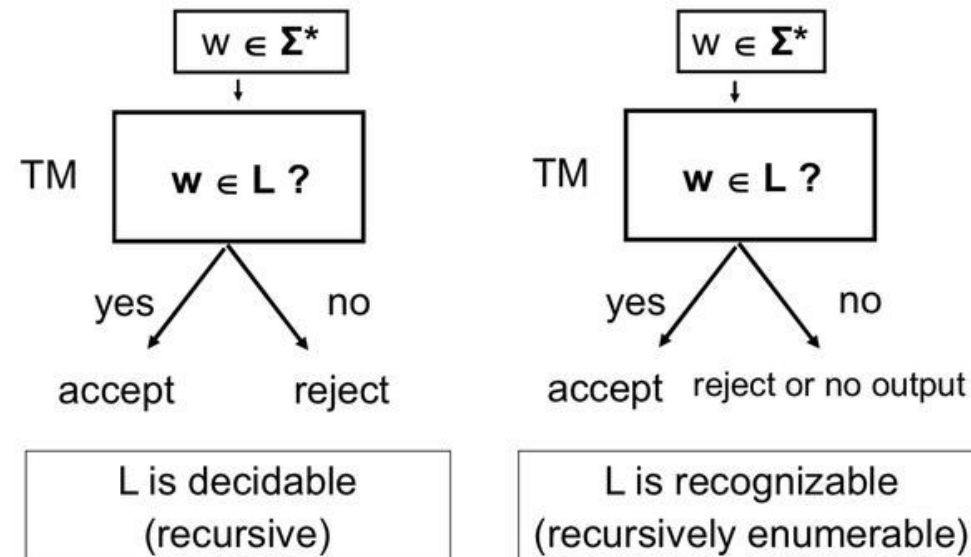
Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

Recursively enumerable languages

Recursive languages

There exist RE languages (for example L_{diag} , PCP) which are not recursive i.e. we can create only the acceptor for such languages. i.e.

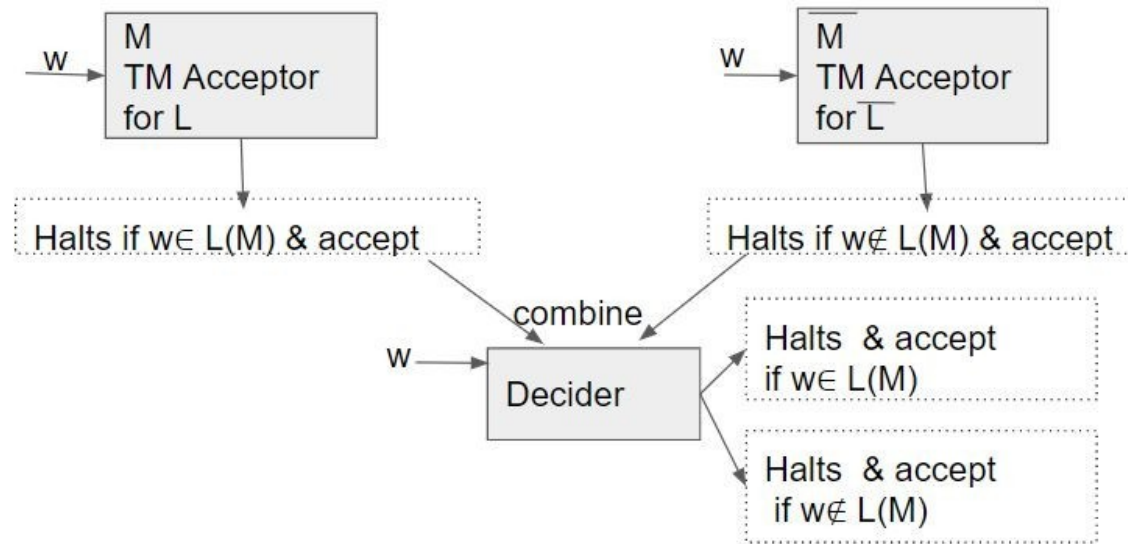


Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

TM created is called TM acceptor. Such machines do not implement membership function. Hence languages like L_{diag} or PCP have an Acceptor but not Decider and are called undecidable.

RE Languages are not closed under complement because if they were, we could have an acceptor for L and its complement L' , the language will become recursive.



Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

Note: Set of languages where both L and L' are RE are called Recursive languages.

Summary:

RE languages

- Have enumeration procedure
- No membership function
- TM acceptor can be implemented

Recursive Languages:

- Have Enumeration Procedure
- Membership function
- Proper subset of RE languages
- TM decider is implemented



THANK YOU

Divyaprabha K N

Department of Computer Science &
Engineering divyaprabha@pes.edu



Automata Formal Languages & Logic

UE24CS243A

Divyaprabha K N

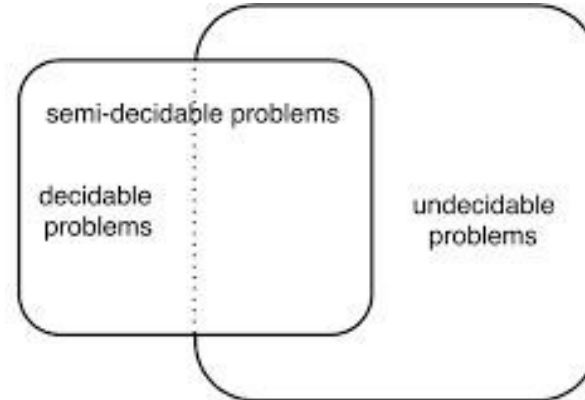
Department of Computer Science & Engineering

Acknowledgement : Preet Kanwal

Undecidability

Undecidable Problems (No algorithm)

Definition :



A language L is **undecidable** if there is no TM M that halts on every input.

Either L is Non-RE

i.e. No Turing machine can be constructed for such a language , **no enumeration procedure to enumerate all elements of such a language set.**

L is RE but not decidable.

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

Examples:

Recursively Enumerable (sets) languages

- Set of Natural Numbers
- Set of even numbers
- Set of Input
- Set of Turing Machines
- Set of positive rational numbers

Non-Recursively Enumerable (sets) languages

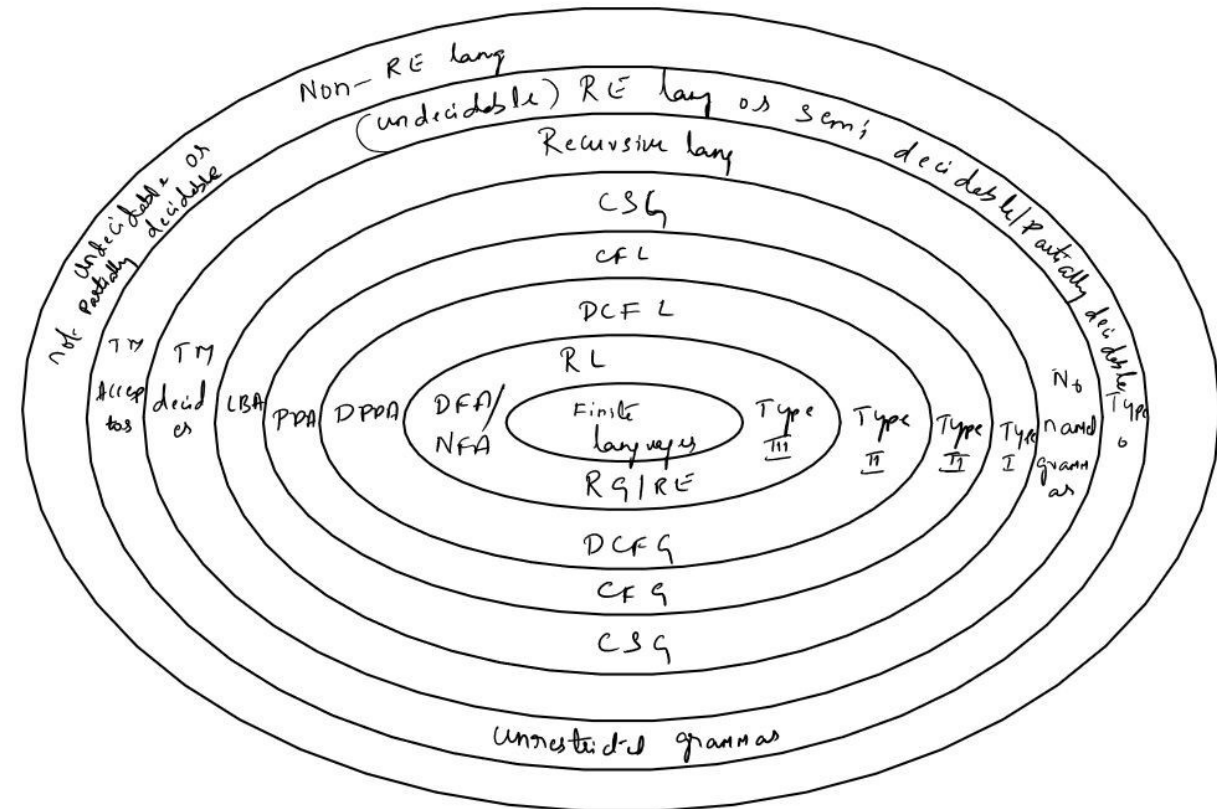
- Set of real numbers
- Set of all languages $\in 2^{\Sigma^*}$

Recursively Enumerable but not decidable

- PCP
- $L1 \cap L2 = \text{Null?}$
- Halting Problem
- Acceptance problem of TM

The Complete Chomsky Hierarchy:

Summary of relationship between class of formal languages, machines and grammars, such a hierarchy of languages is known as Chomsky Hierarchy in honor of Noam Chomsky who developed much of theory of formal languages and grammars.



Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

Undecidability or unsolvability:

We begin to investigate the power of algorithms to solve problems. We demonstrate certain problems that can be solved algorithmically and others that cannot. Our objective is to explore the limits of algorithmic solvability.

Why should one study unsolvability?

First, knowing when a problem is algorithmically unsolvable is useful because then you realize that the problem must be simplified or altered before you can find an algorithmic solution. Like any tool, computers have capabilities and limitations that must be appreciated if they are to be used well.

The second reason is cultural. Even if you deal with problems that clearly are solvable, a glimpse of the unsolvable can stimulate your imagination and help you gain an important perspective on computation.

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

In one type of unsolvable problem, you are given a computer program and a precise specification of what that program is supposed to do (e.g., sort a list of numbers). You need to verify that the program performs as specified (i.e., it is correct). Because both the program and the specification are mathematically precise objects, you hope to automate the process of verification by feeding these objects into a suitably programmed computer. However, you will be disappointed. The general problem of software verification is not solvable by computer.

For any undecidable language, either it or its complement is not Turing-recognizable.

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

Post Correspondence Problem(PCP)

- Undecidable decision problem (RE and undecidable)
- Introduced by Emil post in 1946

Introduction :

Somebody gives you two sets of strings

Example 1:

	List A	List B
1	10	10110
2	111	01
3	110110	1010

[In general, any number of pairs of strings can be given -here we have 3 pairs]

The question is, can I make a sequence of these numbers for example, 1 1 3 2 2 1

So that when I go ahead and concatenate all the strings from each list A,B in that order (in that sequence) the two strings will be the same.

Sequence	1	1	3	2	2	1
A	10	10	110110	111	111	10
B	10110	10110	1010	01	01	10110

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

The question is after concatenating the string from A and B in the given sequence. Are the resulting strings the same?

Is there a sequence that lets them be the same. So the post correspondence problem is:

Given two sets of strings A and B(over same input Σ) each having n strings.

A	B
a1	b1
a2	b2
a3	b3
a4	b4
.	.
.	.
.	.
.	.
an	bn

- The problem is to come up with a sequence in which when one concatenates all strings in A(in that order) and concatenates all strings in B (in same order) .The resulting string is the same.
- Is there some order of concatenation that will give id identical results for both the sets.
- Can we find a permutation(arrangement) that results in same string.
- If we do not allow repetition of strings since our n is finite(n=3) in our example. We can enumerate all permutations and try each resulting concatenation.
- Hence ,the problem can be solved easily. If repetition of strings is allowed :
- There will be no limit on the length of the concatenation although n-finite.
- This is the general case of PCP-more complex.

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

- It turns out that only some instances of PCP have solutions.
- Other pairs of sets A and B, for which there is no solution, we cannot solve them and they may enter into an infinite loop.
- There is no algorithm that can solve all instances of PCP.

There will never be an algorithm that can solve all instances of this problem . So it is an example of a computing problem which is unsolvable.

PCP-There are various variations of this problem which are unsolvable, undecidable.

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

Example 2:

	List A	List B
1	10	101
2	011	11
3	101	011

Only way to match is start with 1.

Sequence A	1		3			3			
Sequence A	1	0	1	0	1	1	0	1	
Sequence B	1			3			3		
Sequence B	1	0	1	0	1	1	0	1	1 (extra 1)

You can only continue with 3. The only way to match then is to continue with 3. Sequence: 1 3 3 3 3
And none of these ever even out because always the bottom is going to be one bit longer than the top.

Example 3:

	List A	List B
1	bb	bbb
2	baa	aab
3	bbb	bb

Sequenc e A	1		2			3		
Sequenc e A	b	b	b	a	a	b	b	b
Sequenc e B	1			2			3	
Sequenc e B	b	b	b	A	a	b	b	b

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

Example 4:

	List A	List B
1	a	baa
2	ab	aa
3	bba	bb

Sequenc e A	3			2		3			1
Sequenc e A	b	b	a	a	B	b	b	a	a
Sequenc e B	3		2		3		1		
Sequenc e B	b	b	a	a	b	b	b	a	a

Example 5:

	List A	List B
1	a	aa
2	aa	ab
3	b	bb

Sequenc e A	1	2		3	3			
Sequenc e A	a	a	a	b	b			
Sequenc e B	1		2		3		3	
Sequenc e B	a	a	a	b	b	b	b	b

Always 2 b's(bb) extra. No solution to this instance of PCP.

Example 5:

Sequenc e A	3	3	3			
Sequenc e A	b	b	b			
Sequenc e B	3		3		3	
Sequenc e B	b	b	b	b	b	b

No solution.

Usage of PCP

Once we know PCP is undecidable. We can prove that other things out there are also undecidable.

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

Idea of Reduction – ($P1 \leq P2$)

Given, problem $P1$ is unsolvable(undecidable).

We reduce $P1$ to another problem $P2$ and show that,

Solution of $P2$ implies solution to $P1$ i.e if $P2$ becomes solvable then $P1$ also becomes solvable. But, as we know $P1$ is unsolvable hence $P2$ is also unsolvable.

Example:

Find out whether two CFL's have something in common or not.

i.e $L1 \cap L2 = \emptyset$?

- Basically, we reduce PCP to empty intersection.
- We show that if there's a method to solve the empty intersection problem, we can solve PCP too.

How are we going to connect the PCP solution to the empty intersection problem?

- Take the PCP problem.
- Come up with two grammars (corresponding to two lists)
- Give these grammars to the person who claims to have an hypothetical algorithm to solve the empty intersection problem.
- The person then comes back with the string that both the languages have in common.
- But if there is no string in common then there is no way to solve this problem.

We took PCP and hid it inside two grammars. Now a person who knows how to check empty intersections between CFL's can solve PCP.

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

Reduction:

Reduction is the method of changing the input of hard problem (PCP) to the input of an unknown problem (empty n) and showing that the answer to unknown problem is true only if the answer to PCP is true.

Hence, we can't do an empty intersection as it implies to PCP and a solution to PCP is impossible.

Example 1:

PCP Problem

	List A	List B	
1	1	111	a
2	10111	10	b
3	10	0	c

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

Grammar 1

$SA \rightarrow 1SAaa \mid 10111SAb \mid 10SAc \mid \lambda$

Grammar 2

$SB \rightarrow 111SBa \mid 10SBb \mid 0SBc \mid \lambda$

a's,b's,c's -keep track of the sequence (way if remembering sequences).

We know this PCP problem has a solution : 2113

Lets generate the string 101111110

$SA \Rightarrow 10111SAb$

$\Rightarrow 101111SAab$

$\Rightarrow 1011111SAaab$

$\Rightarrow 101111110SAcaab$

$\Rightarrow 101111110-caab \text{ caab}=3112 \text{ (reverse as its nested)}$

We can generate the same string using S_B .

Since we have a's,b's ,c's we can be sure that strings are formed using the same sequence(makes sure we have a solution for PCP).

Hence if S_A and S_B generate the same string, there is a solution to PCP.

We actually try to solve PCP using $L1 \cap L2 = \emptyset$ as a subroutine. As there is no way to solve PCP in the subroutine $L1 \cap L2 = \emptyset$ cannot exist.

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

2. $A_{TM} = \{\langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts } w\}$ is undecidable

Note : Recursively Enumerable but not Decidable

The problem of determining whether a Turing machine accepts a given input.

Proof using Diagonalization :

Assume that a TM H decides A_{TM} .

We construct a new Turing machine D with H as a subroutine.

Finally, run D on itself.

Thus, the machines take the following actions, with the last line being the contradiction.

- H accepts $\langle M, w \rangle$ exactly when M accepts w .
- D rejects $\langle M \rangle$ exactly when M accepts $\langle M \rangle$.
- D rejects $\langle D \rangle$ exactly when D accepts $\langle D \rangle$???

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

We list all TM s down the rows, $M_1, M_2, \dots$, and all their descriptions across the columns, $\langle M_1 \rangle, \langle M_2 \rangle, \dots$. The entries tell whether the machine in a given row accepts the input in a given column. The entry is *accept* if the machine accepts the input but is blank if it rejects or loops on that input.

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	$\dots$
M_1	<i>accept</i>		<i>accept</i>		
M_2	<i>accept</i>	<i>accept</i>	<i>accept</i>	<i>accept</i>	
M_3					$\dots$
M_4	<i>accept</i>	<i>accept</i>			
$\vdots$			$\vdots$		

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

If we run H on inputs corresponding to above table, H rejects input $\langle M_3, \langle M_2 \rangle \rangle$. H accepts input $\langle M_4, \langle M_1 \rangle \rangle$. Hence, table for H becomes :

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...
M_1	<i>accept</i>	<i>reject</i>	<i>accept</i>	<i>reject</i>	
M_2	<i>accept</i>	<i>accept</i>	<i>accept</i>	<i>accept</i>	
M_3	<i>reject</i>	<i>reject</i>	<i>reject</i>	<i>reject</i>	...
M_4	<i>accept</i>	<i>accept</i>	<i>reject</i>	<i>reject</i>	
$\vdots$			$\vdots$		

By our assumption, H is a TM and so is D. Therefore, it must occur on the list $M_1, M_2, \dots$ of all TM s.

Note that D computes the opposite of the diagonal entries. The contradiction occurs at the point of the question mark where the entry must be the opposite of itself.

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	$\dots$	$\langle D \rangle$	$\dots$
M_1	<u>accept</u>	reject	accept	reject		accept	
M_2	accept	<u>accept</u>	accept	accept	$\dots$	accept	$\dots$
M_3	reject	reject	<u>reject</u>	reject		reject	
M_4	accept	accept	reject	<u>reject</u>		accept	
$\vdots$			$\vdots$		$\ddots$		
D	reject	reject	accept	accept		<u>?</u>	
$\vdots$			$\vdots$				$\ddots$

No matter what D does, it is forced to do the opposite, which is obviously a contradiction. Thus, neither TM D nor TM H can exist.

(Trying to construct Machine for **LD** by using Machine for **A TM** as subroutine)

Halting Problem: The Unsolvable Problem

The halting problem can never be solved algorithmically.

Why the Halting Problem Matters in Theory of Computation

The Halting Problem has an extensive scope and plays a crucial role in understanding the limitations of **computation**. It sets the boundary for what computers can and cannot solve and has substantial implications for multiple areas of study, from Artificial Intelligence to Cybersecurity.

Alan Turing and the Halting Problem

Alan Turing, often referred to as the father of theoretical computer science and artificial intelligence, made significant contributions towards solving the Halting Problem.

Turing's efforts were instrumental in proving that **a general algorithm that solves the Halting problem for all possible program-input pairs cannot exist**. As such, he demonstrated the restrictions of computers, thus establishing a limitation on the power of mechanical computation.

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

Definitions of Halting Problem

There are many equivalent definitions of Halting Problem:

- Given a TM M and string w , does M halt on w ?. Note that M doesn't have to accept w ; it just has to halt on w .
As a formal language: $\text{HALTTM} = \{ \langle M, w \rangle \mid M \text{ is a TM and } M \text{ halts on } w. \}$
- The Halting Problem, in the simplest terms, is a statement about computational processes in computer science.
Is it possible to write a program that determines whether another program will halt or run indefinitely?
- Suppose we have a Turing machine that never halts. Can we make a Turing machine that can detect this? In other words, can we make an infinite loop detector?

3. Let's consider a related problem, HALT TM , the problem of determining whether a Turing machine halts (by accepting or rejecting) on a given input. This problem is widely known as the halting problem. We use the undecidability of ATM to prove the undecidability of the halting problem by reducing ATM to HALT TM . Let $\text{HALT TM} = \{ \langle M, w \rangle \mid M \text{ is a TM and } M \text{ halts on input } w \}$.

Let's assume for the purpose of obtaining a contradiction that TM R decides HALT TM . We construct TM S to decide ATM , with S operating as follows

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

$S =$ “On input $\langle M, w \rangle$, an encoding of a TM M and a string w :

1. Run TM R on input $\langle M, w \rangle$.
2. If R rejects, reject . If R indicates that M doesn't halt on w , reject because $\langle M, w \rangle$ isn't in A TM
3. If R accepts, simulate M on w until it halts.
4. If M has accepted, accept ; if M has rejected, reject .”

Clearly, if R decides $HALT$ TM , then S decides ATM . Because A TM is undecidable, $HALT$ TM also must be undecidable.

Closure properties

Recursive languages are closed under the following operations. That is, if L and P are two recursive languages, then the following languages are recursive as well:

- the Kleene star L^* of L
- the concatenation $L.P$ of L and P
- the union $L \cup P$
- the intersection $L \cap P$.
- The complement of L
- The set difference $L - P$

The last property follows from the fact that the set difference can be expressed in terms of intersection and complement.

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

Note:

- If a type of problem or, equivalently, a formal language has a decider Turing Machine, then it is said to be computable or decidable.

In other words, all recursive languages are computable or decidable.

- If there was no halting problem, we could convert every acceptor machine to a decider, thereby making every recursively enumerable language recursive.

Examples :

As noted above, every **context-sensitive language** is recursive. Thus, a simple example of a recursive language is the set $L = \{abc, aabbcc, aaabbbccc, \dots\}$; more formally, the set

$$L = \{a^n b^n c^n \mid n \geq 1\}$$

is context-sensitive and therefore recursive.

$L = \{a^n b^n c^n \mid n \geq 1\}$ is recursive because we can construct a Turing machine which will move to final state if the string is of the form $anbn cn$ else move to non-final state. So, the TM will always halt in this case.

A **context-sensitive grammar** for $L = \{a^n b^n c^n \mid n \geq 1\}$ is

$$S \rightarrow aBSc \mid abc$$

$$Ba \rightarrow aB$$

$$Bb \rightarrow bb$$

Try to derive $aaabbbccc$?

$$[S \Rightarrow aBSc \Rightarrow aBaBSc \Rightarrow aBaBabccc \Rightarrow aaBBabccc \Rightarrow aaBaBbccc \Rightarrow aaBabbccc \Rightarrow aaaBbbccc \Rightarrow aaabbbccc]$$

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

Note:

Context-Sensitive Grammar:

o CSG Production rules constraints: $\alpha A \beta \rightarrow \alpha \gamma \beta$ (where α, β, γ = string of terminals and/or non-terminals, A = Non-terminal)

A linear bounded automata (LBA) is a restricted form of Turing machine.

An LBA satisfies the following three conditions:

- o Its input alphabet includes two special symbols, serving as left and right endmarkers.
- o Its transitions may not print other symbols over the endmarkers.
- o Its transitions may neither move to the left of the left endmarker nor to the right of the right endmarker.

Closure properties –

Recursively enumerable languages (REL) are closed under the following operations. That is, if L and P are two recursively enumerable languages, then the following languages are recursively enumerable as well:

- the Kleene star L^* of L
- the concatenation $L.P$ of L and P
- the union $L \cup P$
- the intersection $L \cap P$.

Recursively enumerable languages are not closed under set difference or complementation. The set difference $L - P$ is recursively enumerable if P is recursive. If L is recursively enumerable, then the complement of L is recursively enumerable if and only if L is also recursive.

Automata Formal Languages and Logic

Unit 4 - Undecidability & Declarative Logic

Note:

- An undecidable language maybe a partially decidable language or something else but not decidable.
- Undecidable languages are not recursive languages, they may be subsets of Turing recognizable languages: i.e., such undecidable languages may be recursively enumerable.
- If a language is not even partially decidable, then there exists no Turing machine for that language.
- All **regular, context-free, context-sensitive** and **recursive** languages are recursively enumerable.



THANK YOU

Dr. Divyaprabha K N

Department of Computer Science &
Engineering divyaprabha@pes.edu



Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

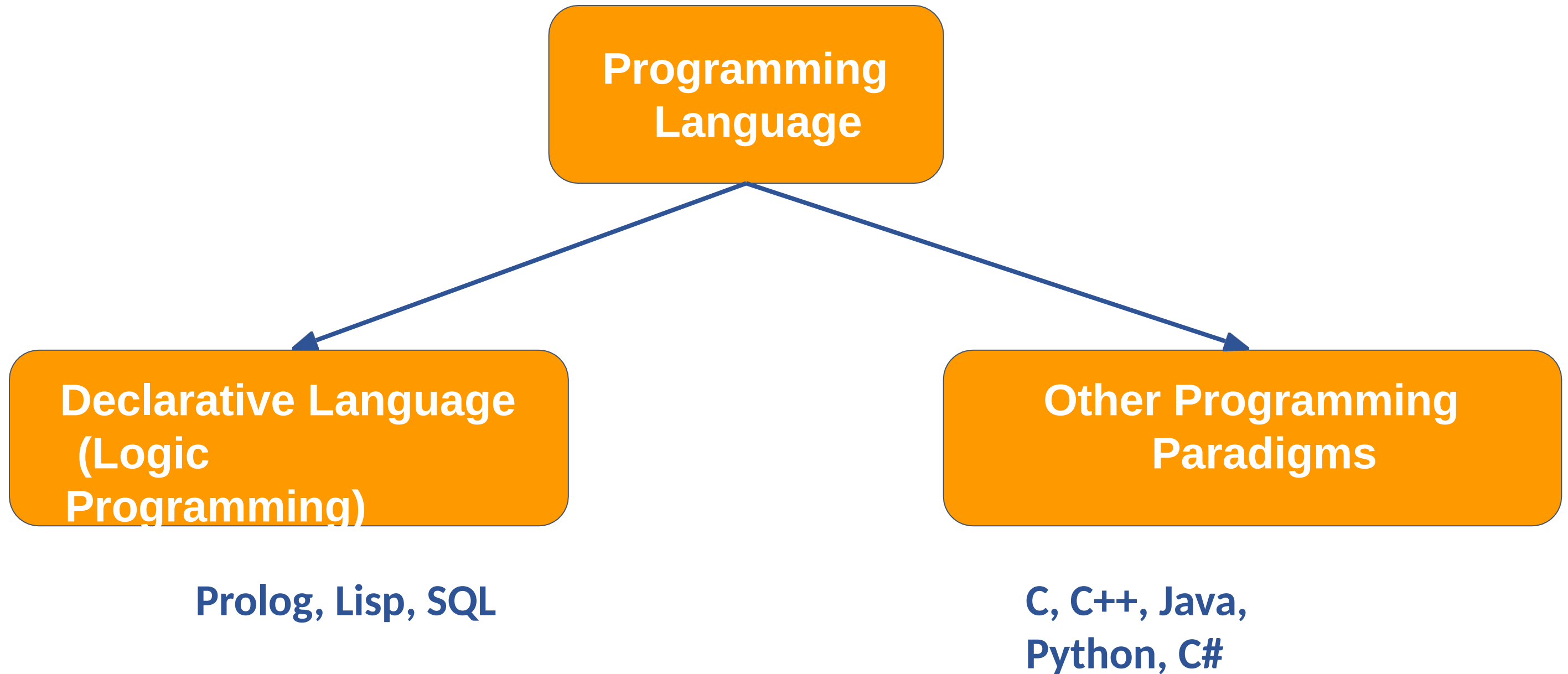
Unit 4

Preet Kanwal

Department of Computer Science & Engineering

In this unit, we look at the application of what we have studied so far.

We look at the design and working of a programming language.



What is the difference??

- **Declarative programming–**
 - Describes what you want to do, and not how you want to do it.
 - It is left up to the compiler/interpreter to figure out the how.
 - expresses the logic of a computation without describing its control flow.
- **Other Programming Paradigms–**
 - Focuses on how to execute,
 - Defines control flow as statements that change a program state.
 - Programmer understands and has access to each step of the code.
 - Lacks mechanism of deriving facts from other facts. Each update to the data structure is done by a domain-specific procedure provided by the programmer.

Unit 4 - Formal Languages and Logic

- In **imperative languages**, the description of the problem is incorporated implicitly in this specification, and usually it is not possible to clearly distinguish between the description of the problem, and the method used for its solution.
- In **logic programming**, the description of the problem and the method for solving it are explicitly separated from each other.

$$\text{algorithm} = \text{logic} + \text{control}$$

- The term 'logic' in this equation indicates the descriptive component of the algorithm, that is, the description of the problem; the term 'control' indicates the component that tries to find a solution, taking the description of the problem as a point of departure.
- So, the logic component defines what the algorithm is supposed to do; the control component indicates how it should be done.

In order to learn the design and working of a logic programming language like Prolog one must study the following :

- Basic concepts of logic (via Propositional logic)
- First order Predicate Logic (more powerful)(FOPL)

Prolog

PROgramming in LOgic

- High level interactive language
- Logic Programming Language
- Declarative language
- Emphasis on what rather than how.
- It is widely used in the field of AI.

It is based on the principles of **First Order Predicate Logic**.

Specifying a logical program (Prolog program) amounts to:

- **Specifying the facts concerning the objects and relations between objects relevant to the problem at hand;**
- **Specifying the rules concerning the objects and their interrelationships;**
- **Posing queries concerning the objects and relations.**

A Prolog program consists of :

Solving a problem in PROLOG starts with describing the objects that are relevant to the particular problem, and the relationships that exist between them.

When we have identified all relevant objects and relations, it must be specified which facts and rules hold for the objects and their interrelationships.

When all facts and rules have been identified, then a specific problem may be looked upon as a query concerning the objects and their interrelationships.

Prolog Basic Constructs

<i>fact</i>	$A.$	
<i>rule</i>	$A :- B_1, B_2, B_3, \dots, B_n.$	if $B_1, B_2, B_3, \dots, B_n$ then A or $B_1 \wedge B_2 \wedge B_3 \wedge \dots \wedge B_n$ $\rightarrow A$
	$A :- B_1; B_2; B_3, \dots, B_n.$	$B_1 \vee B_2 \vee B_3 \vee \dots \vee B_n \rightarrow A$
<i>query</i>	$?- B_1, \dots, B_n$	

Facts and rules form the *knowledge base*

Note :

Following terms are used interchangeably:

fact, sentence, axiom - data which is given without being derived from other facts/sentences.

Deriving new sentences from existing facts is called **inferencing.**

// A simple logical program

/* facts */

bigger(elephant, horse).

bigger(horse, monkey).

bigger(monkey, cat).

bigger(cat, ant).

bigger(lion, monkey).

/*rules*/

find(X, Z):- (bigger(X,Y), bigger(Y,Z)) ; bigger(X,Z).

Query the following:

?- find(elephant, cat)

?- find(X, monkey)

?- find(monkey, X)

?- find(X, Y)

Execute here : <https://swish.swi-prolog.org/example/examples.swinb>

Procedure View of Prolog:

Like normal *imperative* programming languages, a Prolog program consists of procedures, which are collections of statements. A program is executed by running a sequence of statements. Each of these statements executes by "calling" a procedure, which calls other procedures, etc.

fact(0,1).

fact(N,Result) :-

 N > 0,

 M is N - 1,

fact(M,Y),

 Result is Y

 * N.

Execute here : <https://swish.swi-prolog.org/example/examples.swinb>

Query the following:

?- **fact**(0, 1)

?- **fact**(5, Value)

?- **fact**(3, 6)

A binary tree Example:

`/*1*/ branch(a,b).`

`/*2*/ branch(a,c).`

`/*3*/ branch(c,d).`

`/*4*/ branch(c,e).`

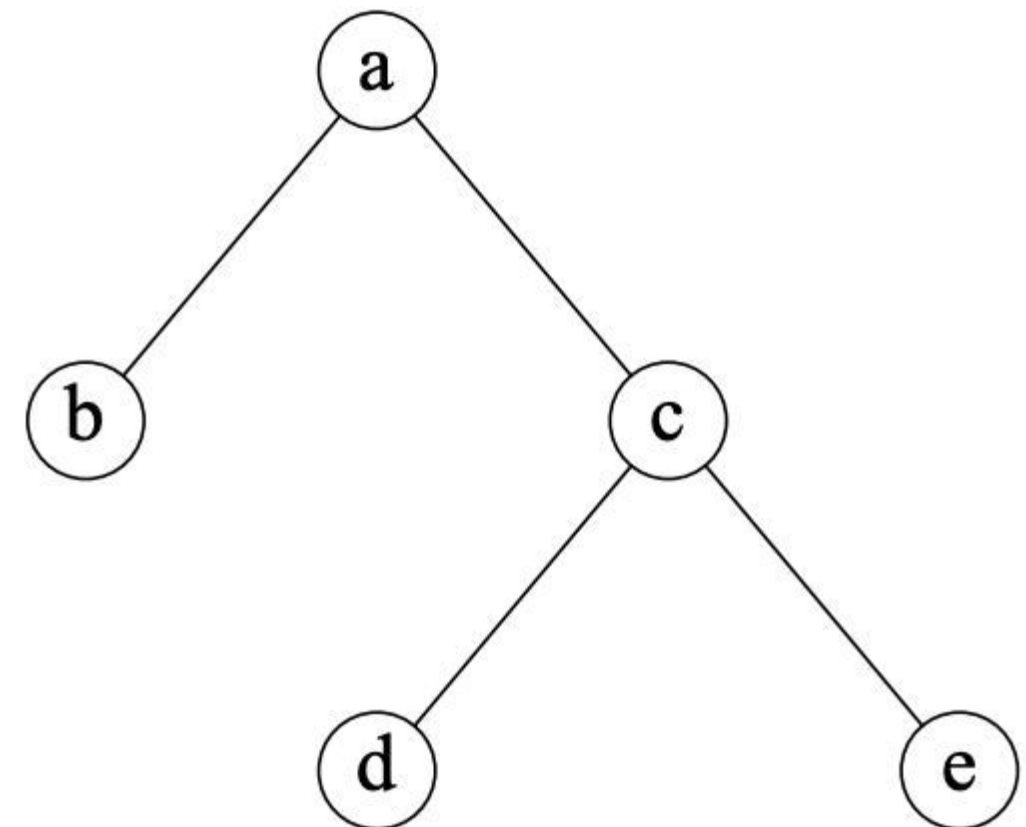
`/*5*/ path(X,X).`

`/*6*/ path(X,Y) :- branch(X,Z), path(Z,Y).`

Query the following:

`?- path(a,a)`

`?- path(a,d)`



PROLOG interpreter examines rules in the order in which they have been specified in the database.

The database created is called Knowledge Base. It is a list of facts (predicates) and rules about the domain(possible world-situation of interest).

What is Logic :

A formal language

- Syntax – what expressions are legal
- Semantics – what legal expressions mean - It's truth value!
- Proof system – a way of manipulating syntactic expressions to get other syntactic expressions (which will tell us something new)



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724



Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 4

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Propositional Logic (Basics)

- We would like to acknowledge efforts of Prof. Channabankapur, Associate Professor, Dept. of CSE in preparation of these slides.

Preet Kanwal

Department of Computer Science & Engineering

Unit 4 - Propositional Logic

Outline:

- What is a Proposition?
- What is Propositional Logic?
- Atomic Sentence
- Complex Sentences using Logical Connectives
($\neg$, $\vee$, $\wedge$, $\rightarrow$, $\leftrightarrow$)
- Syntax & Semantics of Propositional Logic
- Tautology
- Contradiction
- Logical Equivalence (Laws of Logic)
- Logical Entailment
- Methods to prove Logical Entailment
- Rules of Inference
- Proof by Resolution

Unit 4 - Propositional Logic

Proposition: A proposition is a declarative sentence that is either true or false, but not both.

Propositional Logic (Propositional Calculus):

- The area of logic that deals with propositions.
- It is also a Declarative language.
- Its expressive power is low (limited to simple environments).

Proposition or not?

- Bangalore is the capital city of India.
- New Delhi is the capital city of India.
- Do you like coffee?
- Get me a cup of coffee.
- $1 + 2 = 3$
- $2 + 3 = 4$
- $x + 3 = 10$
- $x + y = z$
- Mahatma Gandhi was born in the year 1869.
- It's raining today.
- Earth is flat.

Unit 4 - Propositional Logic

Following are Propositions(Atomic Sentences):

- **New Delhi is the capital city of India. // true**
- **Bangalore is the capital city of India. // false**
- **$1 + 2 = 3$ // true**
- **$2 + 3 = 4$ // false**
- **Mahatma Gandhi was born in 1869. // true**
- **It's raining today. // false**
- **Earth is flat. // false**

Unit 4 - Propositional Logic

These are NOT propositions:

- Get me a cup of coffee. // not a declarative sentence
- Do you like coffee? // not a declarative sentence
 - Though the response to the question could be yes or no; a binary response.
- $x + 1 = 2$ // has a variable
 - It's a declarative sentence, but depending on the value of 'x', it can be true or false.
 - It can be turned into a proposition by giving a value for 'x'.
- $x + y = z$. // has variables

Unit 4 - Propositional Logic

Compound Propositions/ Complex Sentence: are propositions formed from existing propositions using **logical connectives/operators**.

Logical Operators:

- Negation ($\neg$)
- Conjunction ($\wedge$)
- Disjunction ($\vee$)
- Conditional Statement ($\rightarrow$)
- Biconditional Statement ($\leftrightarrow$)

Negation:

- Let p be a proposition. The negation of p , denoted by $\neg p$, is the statement “It is not the case that p ”.
- The proposition $\neg p$ is read “not p ”.
- The truth value of the negation of p is the opposite of the truth value of p .

Eg: Let p be “Today is Friday”

$\neg p$: “It’s not the case that today is Friday” or

$\neg p$: “Today is not Friday”

p	$\neg p$
F	T
T	F

Unit 4 - Propositional Logic

Conjunction: Let p and q be propositions.

- The conjunction of p and q , denoted by $p \wedge q$, is the proposition “ p and q ”.
- The conjunction $p \wedge q$ is true when both p and q are true and is false otherwise.

p	q	$p \wedge q$
F	F	F
F	T	F
T	F	F
T	T	T

Unit 4 - Propositional Logic

Examples for Conjunction:

Example 1:

p: “He brought pen to the class”

q: “He brought notebook to the class”

$p \wedge q$: “He brought pen and notebook to the class”

Example 2:

p: “Tea is served in the conference”

q: “Coffee is served in the conference”

$p \wedge q$: “Tea and Coffee are served in the conference”

Disjunction: Let p and q be propositions.

- The disjunction of p and q , denoted by $p \vee q$, is the proposition “ p or q ”.
- The disjunction $p \vee q$ is false when both p and q are false and is true otherwise.

p	q	$p \vee q$
F	F	F
F	T	T
T	F	T
T	T	T

Example for Disjunction:

p : “You are tall”

q : “You are an adult”

$p \vee q$: “you are tall or you are an adult”

Unit 4 - Propositional Logic

Conditional Statement (implies) :Let p and q be propositions. The “conditional statement” $p \rightarrow q$ is false when p is true and q is false, and true otherwise. Conditional statements are also called implications.

Truth table of Conditional Statement:

In $p \rightarrow q$,
 p is called hypothesis, premise or claim
or antecedent
 q is called conclusion or consequence

p	q	$p \rightarrow q$
F	F	T
F	T	T
T	F	F
T	T	T

Unit 4 - Propositional Logic

Note :

- **Implies($p \rightarrow q$) doesn't mean Suggestion/Causation. For example :**
 - a) **Green is a color implies Austin is in Texas**
 - b) **9 is even implies x is 5.**

Both statements are perfectly okay from the logic point of view.

- **All Statements must have a truth value (even though it doesn't make sense).**

Unit 4 - Propositional Logic

Biconditional Statement: Let p and q be propositions. The biconditional statement $p \leftrightarrow q$ is the proposition “ q if and only if p ”. The biconditional statement $p \leftrightarrow q$ is true when p and q have the same truth values, and is false otherwise.

Biconditional statements are also called as bi-implications.

Example:

$p \rightarrow q$: “if the switch is on, then the current flows”.

$p \leftrightarrow q$: “current flows if and only if the switch is on”.

p	q	$p \leftrightarrow q$
F	F	T
F	T	F
T	F	F
T	T	T

Automata Formal Languages and Logic Unit 4 - Propositional Logic



Syntax & Semantics of Propositional Logic :

Syntax - Specified using Grammar.

$$\begin{aligned} \text{Sentence} &\rightarrow \text{AtomicSentence} \mid \text{ComplexSentence} \\ \text{AtomicSentence} &\rightarrow \text{True} \mid \text{False} \mid P \mid Q \mid R \mid \dots \\ \text{ComplexSentence} &\rightarrow (\text{Sentence}) \mid [\text{Sentence}] \\ &\mid \neg \text{Sentence} \\ &\mid \text{Sentence} \wedge \text{Sentence} \\ &\mid \text{Sentence} \vee \text{Sentence} \\ &\mid \text{Sentence} \Rightarrow \text{Sentence} \\ &\mid \text{Sentence} \Leftrightarrow \text{Sentence} \end{aligned}$$

OPERATOR PRECEDENCE : $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$

Semantics - Based on Truth value of the sentences. Meaning of the sentence is a function of meaning of its parts.

Unit 4 - Propositional Logic

Let p , q , and r be the propositions p : You have the flu.

q : You miss the final examination. r : You pass the course.

Express the propositions as English sentences:

- $p \rightarrow q$
- $\neg q \leftrightarrow r$
- $q \rightarrow \neg r$
- $(p \rightarrow \neg r) \vee (q \rightarrow \neg r)$
- $(p \wedge q) \vee (\neg q \wedge r)$

Unit 4 - Propositional Logic

Solution :

Let p , q , and r be the propositions p : You have the flu.

q : You miss the final examination. r : You pass the course.

Express the propositions as English sentences:

- $p \rightarrow q$: If you have the flu then you miss the final examination.
- $\neg q \leftrightarrow r$: You pass the course iff you do not miss the final examination.
- $q \rightarrow \neg r$: If you miss the final examination then you will not pass the course.
- $(p \rightarrow \neg r) \vee (q \rightarrow \neg r)$: If you have the flu then you will not pass the course or if you miss the final examination then you will not pass the course.
- $(p \wedge q) \vee (\neg q \wedge r)$: You have the flu and you miss the final examination or you will not miss the final examination and you pass the course.

Unit 4 - Propositional Logic

Let p and q be the propositions p : You drive over 60 kmph.
 q : You get a speeding ticket.

Write the following propositions using p , q and logical connectives.

- a. You do not drive over 60 kmph
- b. You drive over 60 kmph, but you do not get a speeding ticket.
- c. You will get a speeding ticket if you drive over 60 kmph.
- d. If you do not drive over 60 kmph, then you will not get a speeding ticket.
- e. You get a speeding ticket, but you do not drive over 60 kmph.

Solution:

p: You drive over 60 kmph. **q:** You get a speeding ticket.

Write the following propositions using p, q and logical connectives.

- a.** You do not drive over 60 kmph: $\neg p$.
- b.** You drive over 60 kmph, but you do not get a speeding ticket : $p \wedge \neg q$.
- c.** You will get a speeding ticket if you drive over 60 kmph : $p \rightarrow q$
- d.** If you do not drive over 60 kmph, then you will not get a speeding ticket :
 $\neg p \rightarrow \neg q$
- e.** You get a speeding ticket, but you do not drive over 60 kmph : $q \wedge \neg p$.

Unit 4 - Propositional Logic

Determine the truth values of the following propositions.

- a. $2+2=4$ if and only if $1+1=2$.
- b. $1+1=2$ if and only if $2+3=4$.
- c. $0>1$ if and only if $2>1$.
- d. If $1+1=2$, then $2+2=5$.
- e. If $1+1=3$, then $2+2=4$.
- f. If $1+1=3$, then $2+2=5$.
- g. If $2+2=4$, then $1+2=3$.
- h. $1+1=3$ if and only if monkeys can fly.
 - i. If monkeys can fly, then $1+1=3$
 - j. If $1+1=3$, then unicorns exist.
 - k. If $1+1=3$, then dogs can fly.
 - l. If $1+1=2$, then dogs can fly.

Unit 4 - Propositional Logic

Solution :

Determine the truth values of the following propositions.

- a. $2+2=4$ if and only if $1+1=2$. **True**
- b. $1+1=2$ if and only if $2+3=4$. **False**
- c. $0>1$ if and only if $2>1$. **False**
- d. If $1+1=2$, then $2+2=5$. **False**
- e. If $1+1=3$, then $2+2=4$. **True**
- f. If $1+1=3$, then $2+2=5$. **True**
- g. If $2+2=4$, then $1+2=3$. **True**
- h. $1+1=3$ if and only if monkeys can fly. **True**
- i. If monkeys can fly, then $1+1=3$ **True**
- j. If $1+1=3$, then unicorns exist. **True**
- k. If $1+1=3$, then dogs can fly. **True**
- l. If $1+1=2$, then dogs can fly. **False**

~~Tautology~~: is a compound proposition that is **always true** no matter what the truth values of the propositions that occur in it.

Contradiction: is a compound proposition that is **always false** no matter what the truth values of the propositions that occur in it. .

p	q	$(p \rightarrow q) \wedge (q \rightarrow p) \leftrightarrow (p \leftrightarrow q)$	$\neg (p \rightarrow q) \wedge \neg (p \wedge \neg q)$
F	F	T	F
F	T	T	F
T	F	T	F
T	T	T	F

Automata Formal Languages and Logic

Unit 4 - Propositional Logic

p	q	p → q	q → p	(p → q) ∧ (q → p)	p ↔ q
F	F	T	T	T	T
F	T	T	F	F	F
T	F	F	T	F	F
T	T	T	T	T	T

∴ **p ↔ q** is equivalent to **(p → q) ∧ (q → p)**

Unit 4 - Propositional Logic

Logical Equivalence: The compound propositions p and q are called logically equivalent if $p \leftrightarrow q$ is a tautology. The notation $p \equiv q$ denotes that p and q are logically equivalent. (Their truth value is the same)

In other words, compound propositions that have the same truth values in all possible cases are called logically equivalent. $(p \rightarrow q) \wedge (q \rightarrow p) \equiv (p \leftrightarrow q)$

Note: The symbol $\equiv$ is **not** a logical connective. Hence “ $p \equiv q$ ” is **not** a compound proposition. “ $p \equiv q$ ” means $p \leftrightarrow q$ is a tautology.

Using truth table, show that

1. $(p \leftrightarrow q) \equiv (p \rightarrow q) \wedge (q \rightarrow p)$.
2. $p \rightarrow q \equiv \neg p \vee q$

Automata Formal Languages and Logic

Unit 4 - Propositional Logic



Example 1: Using truth table, show that
 $(p \leftrightarrow q) \equiv (p \rightarrow q) \wedge (q \rightarrow p)$.

p	q	$p \leftrightarrow q$	$p \rightarrow q$	$q \rightarrow p$	$(p \rightarrow q) \wedge (q \rightarrow p)$	$(p \leftrightarrow q) \leftrightarrow (p \rightarrow q) \wedge (q \rightarrow p)$
F	F					
F	T					
T	F					
T	T					

Example 1: Using truth table, show that

$$(p \leftrightarrow q) \equiv (p \rightarrow q) \wedge (q \rightarrow p).$$

p	q	$p \leftrightarrow q$	$(p \rightarrow q) \wedge (q \rightarrow p)$	$(p \leftrightarrow q) \leftrightarrow (p \rightarrow q) \wedge (q \rightarrow p)$
F	F	T	T	T
F	T	F	F	T
T	F	F	F	T
T	T	T	T	T

Truth table demonstrates,

$(p \leftrightarrow q) \leftrightarrow (p \rightarrow q) \wedge (q \rightarrow p)$ is a tautology.

Therefore, $(p \leftrightarrow q) \equiv (p \rightarrow q) \wedge (q \rightarrow p)$

Example 2: Using truth tables, show that $p \rightarrow q$ and $\neg p \vee q$ are logically equivalent.

p	q	$\neg p$	$p \rightarrow q$	$\neg p \vee q$	$(p \rightarrow q) \leftrightarrow (\neg p \vee q)$
F	F				
F	T				
T	F				
T	T				

Example 2: Using truth tables, show that $p \rightarrow q$ and $\neg p \vee q$ are logically equivalent.

$(p \rightarrow q) \leftrightarrow (\neg p \vee q)$ is a tautology. Therefore, $(p \rightarrow q) \equiv (\neg p \vee q)$

p	q	$\neg p$	$p \rightarrow q$	$\neg p \vee q$	$(p \rightarrow q) \leftrightarrow (\neg p \vee q)$
F	F	T	T	T	T
F	T	T	T	T	T
T	F	F	F	F	T
T	T	F	T	T	T



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724



Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 4

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages and Logic

Unit 4 - Propositional Logic



Equivalence	Name of Identity
$p \wedge T \equiv p$ $p \vee F \equiv p$	Identity Laws
$p \wedge F \equiv F$ $p \vee T \equiv T$	Domination Laws
$p \wedge p \equiv p$ $p \vee p \equiv p$	Idempotent Laws
$\neg(\neg p) \equiv p$	Double Negation Law
$p \wedge q \equiv q \wedge p$ $p \vee q \equiv q \vee p$	Commutative Laws
$(p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$ $(p \vee q) \vee r \equiv p \vee (q \vee r)$	Associative Laws
$p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$ $p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$	Distributive Laws
$\neg(p \wedge q) \equiv \neg p \vee \neg q$ $\neg(p \vee q) \equiv \neg p \wedge \neg q$	De Morgan's Laws
$p \wedge (p \vee q) \equiv p$ $p \vee (p \wedge q) \equiv p$	Absorption Laws
$p \wedge \neg p \equiv F$ $p \vee \neg p \equiv T$	Negation Laws

Laws of Logic

~~Q: Prove the following logical equivalences using the laws of logic (without using the truth table).~~

1. $\neg (p \rightarrow q) \equiv p \wedge \neg q$
2. $p \wedge (p \rightarrow q) \equiv p \wedge q$

Homework

3. $\neg (p \vee (\neg p \wedge q)) \equiv \neg (p \vee q)$

Q: Prove the following logical equivalences using the laws of logic (without using the truth table).

1. $\neg (p \rightarrow q) \equiv p \wedge \neg q$ Soln: LHS $\equiv \neg (p \rightarrow q)$
 $\equiv$

Q: Prove the following logical equivalences using the laws of logic (without using the truth table).

1. $\neg (p \rightarrow q) \equiv p \wedge \neg q$ Soln: LHS $\equiv \neg (p \rightarrow q)$

$$\equiv \neg (\neg p \vee q) \quad \because (p \rightarrow q) \equiv \neg p \vee q \text{ De Morgan's law}$$

$$\equiv \neg (\neg p) \wedge \neg q$$

$$\therefore \neg (\neg p) \wedge \neg q \equiv p \wedge \neg q$$

2. $p \wedge (p \rightarrow q) \equiv p \wedge q$

Soln: $\text{LHS} \equiv p \wedge (p \rightarrow$
 $q))$

$\equiv$

2. $p \wedge (p \rightarrow q) \equiv p \wedge q$

Soln: LHS $\equiv p \wedge (p \rightarrow$

$q) \equiv p \wedge (\neg p \vee q) \quad \because (p \rightarrow q) \equiv (\neg p \vee q)$

$\equiv (p \wedge \neg p) \vee (p \wedge q)$

$q) \equiv F \vee (p \wedge q)$ Distribution law

$\equiv p \wedge q$

$\therefore p \wedge (p \rightarrow q) \equiv p \wedge q$

Homework

3. $\neg (p \vee (\neg p \wedge q)) \equiv \neg (p \vee q)$ Soln: LHS $\equiv \neg (p \vee (\neg p \wedge q))$
 $\equiv$

$$\begin{aligned} 3. \quad & \neg (p \vee (\neg p \wedge q)) \equiv \neg (p \\ & \vee q) \text{ Soln: } \text{LHS} \equiv \neg (p \vee \\ & (\neg p \wedge q)) \quad \text{De Morgan's law} \\ & \equiv \neg p \wedge \neg (\neg p \vee q) \quad \text{De Morgan's} \\ & \quad \text{law} \\ & \equiv \neg p \wedge \neg (p \vee \neg q) \quad \text{Distribution law} \\ & \equiv (\neg p \wedge \neg p) \vee (\neg p \wedge \neg \\ & q) \\ & \equiv F \vee (\neg p \wedge \neg q) \quad \text{De Morgan's law} \\ & \equiv \neg p \wedge \neg q \\ & \equiv \neg (p \vee q) \end{aligned}$$

Q: Prove the following expressions are tautologies using the laws of logic (without using the truth table)

1. $(p \wedge q) \rightarrow (p \vee q)$
2. $(p \wedge q) \rightarrow (p \rightarrow q)$

Q: Prove the following expressions are tautologies using the laws of logic (without using the truth table)

1. $(p \wedge q) \rightarrow (p \vee q)$ Soln: $(p \wedge q) \rightarrow (p \vee q)$
 $\equiv$

Unit 4 - Propositional Logic

Q: Prove the following expressions are tautologies using laws of logic (without using truth table)

1. $(p \wedge q) \rightarrow (p \vee q)$

Soln: $(p \wedge q) \rightarrow (p \vee q)$

$$\equiv \neg (p \wedge q) \vee (p \vee q) \quad \because (p \rightarrow q) \equiv (\neg p \vee q)$$

$$\equiv (\neg p \vee \neg q) \vee (p \vee q) \quad \text{De Morgan's law}$$

$$\equiv \neg p \vee q \vee p \vee \neg q \quad \text{Associative law}$$

$$\equiv \neg p \vee p \vee \neg q \vee q \quad \text{Commutative law}$$

$$\equiv T \vee T$$

$$\equiv T$$

$$\therefore (p \wedge q) \rightarrow (p \vee q) \text{ is a tautology}$$

$$\equiv T$$

2. $(p \wedge q) \rightarrow (p \rightarrow q)$

Soln: $(p \wedge q) \rightarrow (p \rightarrow q)$

$\equiv$

Unit 4 - Propositional Logic

$$2. \quad (p \wedge q) \rightarrow (p \rightarrow q)$$

$$\text{Soln: } (p \wedge q) \rightarrow (p \rightarrow q)$$

$$\equiv \neg (p \wedge q) \vee (\neg p \vee q)$$

$$\equiv (\neg p \vee \neg q) \vee (\neg p \vee q)$$

$$\equiv \neg p \vee \neg q \vee \neg p \vee q$$

$$\equiv \neg p \vee \neg p \vee \neg q \vee q$$

$$\equiv \neg p \vee T$$

$$\equiv \neg p \vee T$$

$\therefore (p \rightarrow q) \equiv (\neg p \vee q)$ De Morgan's law

Associative law

Commutative law

Domination law

$\therefore (p \wedge q) \rightarrow (p \rightarrow q)$ is a tautology

Unit 4 - Propositional Logic

Logical entailment ($\alpha \models \beta$):

Logical entailment describes the relationship between statements that hold true when one statement logically follows from one or more statements.

A rule of the form $(p_1 \wedge p_2 \wedge \dots \wedge p_n) \rightarrow q$ is called a **logical entailment**(or a **valid argument**). It must be a Tautology.

The sentence q logically follows from $(p_1 \wedge p_2 \wedge \dots \wedge p_n)$ which is another sentence.

$(p_1 \wedge p_2 \wedge \dots \wedge p_n)$ are called premises and q is called the conclusion .We can say conclusion is entailed by the premises

$$\begin{array}{l} p_1 \quad p_2 \\ \dots \quad p_n \\ \hline \therefore q \end{array}$$

Mathematically, we can specify logical entailment as: $\alpha \models$

β We can say, $\alpha \equiv \beta$ iif $\alpha \models \beta$ and $\beta \models \alpha$

Automata Formal Languages and Logic

Unit 4 - Propositional Logic

Logical Entailment can be done via the following methods:

1. Model Checking (by constructing the truth table)
2. Rules of Reference (Theorem Proving)
 - Using Resolution rule alone we can construct the required proof. (one rule is all you need to prove things)

Rules of Inference:

- We can use a truth table to show that an argument form is valid.
- But when we have more propositional variables involved in the argument, solving by truth table is too laborious (having just **30 variables** will require over a **billion rows** in the truth table).
- That's when we need well-known argument forms to simplify a complex argument form at hand.
- These well-known valid argument forms are called **Rules of Inference**.

Rule of Inference	Tautology	Name
$\frac{p \quad p \rightarrow q}{\therefore q}$	$(p \wedge (p \rightarrow q)) \rightarrow q$	Modus Ponens
$\frac{\neg q \quad p \rightarrow q}{\therefore \neg p}$	$(\neg q \wedge (p \rightarrow q)) \rightarrow \neg p$	Modus Tollens
$\frac{p \rightarrow q \quad q \rightarrow r}{\therefore p \rightarrow r}$	$((p \rightarrow q) \wedge (q \rightarrow r)) \rightarrow (p \rightarrow r)$	Hypothetical syllogism
$\frac{\neg p \quad p \vee q}{\therefore q}$	$(\neg p \wedge (p \vee q)) \rightarrow q$	Disjunctive Syllogism
$\frac{p}{\therefore (p \vee q)}$	$p \rightarrow (p \vee q)$	Addition
$\frac{(p \wedge q) \rightarrow r}{\therefore p \rightarrow (q \rightarrow r)}$	$((p \wedge q) \rightarrow r) \rightarrow (p \rightarrow (q \rightarrow r))$	Exportation
$\frac{p \vee q \quad \neg p \vee r}{\therefore q \vee r}$	$((p \vee q) \wedge (\neg p \vee r)) \rightarrow q \vee r$	Resolution

Magical Rule

Automata Formal Languages and Logic

Unit 4 - Propositional Logic

Proof by Resolution :

- Pretty much every modern automated theorem-prover implements Resolution technique.
- It's apparently the best way for computers to think about proving things.

Resolution rule (it is an $\wedge$ of $\vee$'s) :

Each premise has to in the form a disjunction!

$$\begin{array}{cc} p \vee q & p \\ \neg p \vee r & \neg p \vee r \end{array}$$

----- or -----
--

Automata Formal Languages and Logic

Unit 4 - Propositional Logic

Proof Strategy is called **Resolution Refutation**(act of proving), It contains 3 steps:

1. Convert all of your sentences to **Conjunctive Normal Form**(CNF)
2. **Negate the desired conclusion** (then convert to CNF) - We are essentially doing a proof by Contradiction.
(You assert that the thing that you're trying to prove is false, and then you try to derive a contradiction.)
3. Add each of these clauses(conclusion) as a premise of your proof.
4. **Apply resolution rule until** either –
 - If we derive false (contradiction), then the conclusion follows from the axioms
 - If we can't apply any more, then the conclusion cannot be proved from the axioms.

Automata Formal Languages and Logic

Unit 4 - Propositional Logic

Resolution refutation is sound and complete :

It's guaranteed that you'll always either prove false, or run out of possible steps.

- It's **complete**, because it always generates an answer.
- The process is **sound**: the answer is always correct.

Automata Formal Languages and Logic

Unit 4 - Propositional Logic

Conjunctive Normal Form :

To convert a formula into a CNF.

- Open up the implications($p \rightarrow q$) to get ORs.
- Get rid of double negations.
- Convert :

$A \vee (B \wedge C)$ to $(A \vee B) \wedge (A \vee C)$
//distributivity

Example:

Convert $A \rightarrow (B \wedge C)$ to CNF. $A \rightarrow (B \wedge C) \equiv \neg A \vee (B \wedge C)$
 $\equiv (\neg A \vee B) \wedge (\neg A \vee C)$

Automata Formal Languages and Logic

Unit 4 - Propositional Logic

Note:

- $(y \vee \neg z) \wedge (\neg y) \wedge (y \vee z)$ is a CNF
- $(x \vee \neg y \vee z)$ is a CNF. So is $(x \wedge \neg y \wedge z)$.
- $(x \vee \neg y \wedge z)$ is not a CNF

Automata Formal Languages and Logic

Unit 4 - Propositional Logic

Answer which of the following is/are in Conjunctive Normal Form
:

1. $(x \vee \neg z \vee y) \wedge (\neg x \vee \neg y) \wedge (\neg y)$
2. $(x \wedge z) \vee (\neg y \wedge z \wedge x) \vee (\neg x \wedge z)$
3. $(x \wedge \neg(y \vee z) \vee u)$

Modus Ponens and Modus Tollens (as a special case of the Resolution rule)

Prove the following :

1) Modus Ponens

$p \rightarrow q$

p

$\therefore q$

2) Modus Tollens

$p \rightarrow q$

$\neg q$

$\therefore \neg p$

Modus Ponens and Modus Tollens (as a special case of the Resolution rule)

Prove the following :

1) Modus Ponens

$p \rightarrow q$

p

$\therefore q$

Solution 1):

$\neg p \vee q$ p

$\therefore q$

2) Modus Tollens

$p \rightarrow q$

$\neg q$

$\therefore \neg p$ **Solution**

2):

$\neg p \vee q$

$\neg q$

$\therefore \neg p$

Automata Formal Languages and Logic

Unit 4 - Propositional Logic

For each of the given question use Resolution Refutation to prove that the conclusion logically entails from the premises :

Q	Premises	Conclusion	In terms of implication
1	$P \vee Q$ $P \rightarrow R$ $Q \rightarrow R$	R	$((P \vee Q) \wedge (P \rightarrow R) \wedge (Q \rightarrow R)) \rightarrow R$
2	$P \rightarrow Q$ $R \rightarrow S$	$(P \vee R) \rightarrow (Q \vee S)$	$((P \rightarrow Q) \wedge (R \rightarrow S)) \rightarrow ((P \vee R) \rightarrow (Q \vee S))$

Automata Formal Languages and Logic

Unit 4 - Propositional Logic

For each of the given question use Resolution Refutation to prove that the conclusion logically entails from the premises :

Q	Premises	Conclusion	Solution
1	$P \vee Q$ $P \rightarrow R$ $Q \rightarrow R$	R	Step 1 : Translate to CNF: i) $P \vee Q$ ii) $\neg P \vee R$ iii) $\neg Q \vee R$ Step 2 : Resolve away the variable P using i) and ii) we get iv) $Q \vee R$ Step 3: Resolve away variable Q using iii) and iv) we get R, hence proved

Automata Formal Languages and Logic

Unit 4 - Propositional Logic

For each of the given question use Resolution Refutation to prove that the conclusion logically entails from the premises :

Q	Premises	Conclusion
2	$P \rightarrow Q$ $R \rightarrow S$	$(P \vee R) \rightarrow (Q \vee S)$

Step 1 : Translate to CNF:

$$\text{i) } \neg P \vee Q$$

$$\text{ii) } \neg R \vee S$$

Step 2 : Negate the conclusion

$$= \neg (\neg (P \vee R) \vee (Q \vee S))$$

$$= (P \vee R) \wedge$$

$(\neg Q \wedge \neg S)$ Therefore, we have the following premises :

$$\text{iii) } \neg P \vee Q$$

$$\text{iv) } \neg R \vee S$$

$$\text{v) } P \vee R$$

$$\text{vi) } \neg Q$$

Automata Formal Languages and Logic

Unit 4 - Propositional Logic

For each of the given question use Resolution Refutation to prove that the conclusion logically entails from the premises :

Step 3:

**Resolve away Q
using i) and iv)**

i) $\neg P \vee Q$

ii) $\neg R \vee S$

iii) $P \vee R$

iv) $\neg Q$

v) $\neg S$ we get :

ii) $\neg R \vee S$

iii) $P \vee R$

iv) $\neg S$

vi) $\neg P$

Step 4:

**Resolve away S
using ii) and v)**

ii) $\neg R \vee S$

iii) $P \vee R$

v) $\neg S$

vi) $\neg P$ we get :

iii) $P \vee R$

vi) $\neg P$

vii) $\neg R$

Step 5:

**Resolve away P
using iii) and vi)**

iii) $P \vee R$

vi) $\neg P$

vii) $\neg R$ we get :

viii) $\neg R$

ix) R

**Step 6: Resolving
R we get False.**

Automata Formal Languages and Logic

Unit 4 - Propositional Logic

For each of the given question use Resolution Refutation to prove that the conclusion logically entails from the premises :

Q	Premises	Conclusio n	In terms of implication
3	P ¬ P	Z	$(P \wedge \neg P) \rightarrow Z$
Example that shows that logical systems are brittle(fragile/breakable)			
4	A V ¬ B ¬ C V B ¬ A C V B V D	D	$((A \vee \neg B) \wedge (\neg C \vee B) \wedge (\neg A) \wedge (C \vee B \vee D)) \rightarrow D$

Automata Formal Languages and Logic

Unit 4 - Propositional Logic

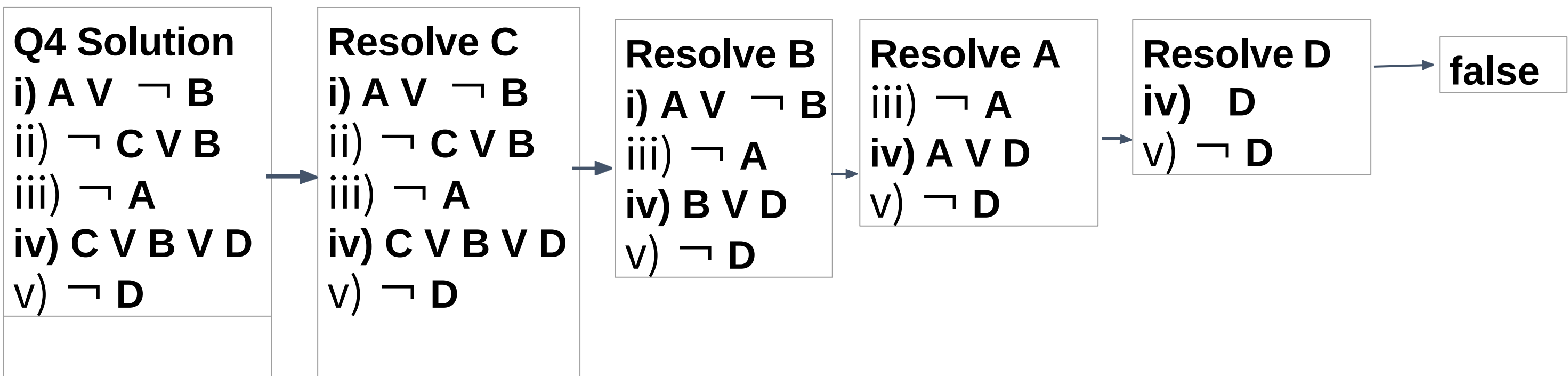
For each of the given question use Resolution Refutation to prove that the conclusion logically entails from the premises :

Q	Premises	Conclusion	Solution
3	P $\neg P$	Z	$(P \wedge \neg P) = \text{False} \rightarrow Z \text{ is True.}$

Automata Formal Languages and Logic

Unit 4 - Propositional Logic

For each of the given question use Resolution Refutation to prove that the conclusion logically entails from the premises :



Automata Formal Languages and Logic

Unit 4 - Propositional Logic

Homework :

For the given question use Resolution Refutation to prove that the conclusion logically entails from the premises :

Premises	Conclusion
$(P \rightarrow Q) \rightarrow Q$ $(P \rightarrow P) \rightarrow R$ $(R \rightarrow S) \rightarrow \neg (S \rightarrow Q)$	R

Automata Formal Languages and Logic

Unit 4 - Propositional Logic

Homework :

For the given question use Resolution Refutation to prove that the conclusion logically entails from the premises :

Given Premises Prove R	Solving i) $(P \rightarrow Q) \rightarrow Q$
i) $(P \rightarrow Q) \rightarrow Q$ ii) $(P \rightarrow P) \rightarrow R$ iii) $(R \rightarrow S) \rightarrow \neg (S \rightarrow Q)$	$(\neg P \vee Q) \rightarrow Q$ $\neg (\neg P \vee Q) \vee Q \quad (P \wedge \neg Q) \vee Q$ $(P \vee Q) \wedge (\neg Q \vee Q)$ $(P \vee Q) \wedge T$ $P \vee Q$

Automata Formal Languages and Logic

Unit 4 - Propositional Logic

Homework :

For the given question use Resolution Refutation to prove that the conclusion logically entails from the premises :

Given Premises Prove R	Solving ii) $(P \rightarrow P) \rightarrow R$
i) $(P \rightarrow Q) \rightarrow Q$ ii) $(P \rightarrow P) \rightarrow R$ iii) $(R \rightarrow S) \rightarrow \neg (S \rightarrow Q)$	$(\neg P \vee P) \rightarrow R$ $\neg (\neg P \vee P) \vee R$ $\neg T \vee R$ $F \vee R$ R

Automata Formal Languages and Logic

Unit 4 - Propositional Logic

Homework :

For the given question use Resolution Refutation to prove that the conclusion logically entails from the premises :

Given Premises Prove R	Solving iii) $(R \rightarrow S) \rightarrow \neg (S \rightarrow Q)$
i) $(P \rightarrow Q) \rightarrow Q$ ii) $(P \rightarrow P) \rightarrow R$ iii) $(R \rightarrow S) \rightarrow \neg (S \rightarrow Q)$	$(\neg R \vee S) \rightarrow \neg (\neg S \vee Q) \quad (\neg R \vee S) \rightarrow (S \wedge \neg Q)$ $\neg (\neg R \vee S) \vee (S \wedge \neg Q)$ $(R \wedge \neg S) \vee (S \wedge \neg Q)$ $(R \vee S) \wedge (R \vee \neg Q) \wedge (\neg S \vee S) \wedge (\neg S \vee \neg Q)$ $(R \vee S) \wedge (R \vee \neg Q) \wedge T \wedge (\neg S \vee \neg Q) \quad (R \vee S) \wedge (R \vee \neg Q) \wedge (\neg S \vee \neg Q)$

Homework :

For the given question use Resolution Refutation to prove that the conclusion logically entails from the premises :

Given Premises Prove R	We have the following premises
i) $(P \rightarrow Q) \rightarrow Q$ ii) $(P \rightarrow P) \rightarrow R$ iii) $(R \rightarrow S) \rightarrow \neg (S \rightarrow Q)$	i) $P \vee Q$ ii) R iii) $(R \vee S)$ iv) $(R \vee \neg Q)$ v) $(\neg S \vee \neg Q)$ Append the negated conclusion we get vi) $\neg R$ We get false by resolving R using ii) and vi) Hence we can say the conclusion follows logically from the premises.

Automata Formal Languages and Logic

Unit 4 - Propositional Logic

Homework :

What are the results of applying Propositional Resolution to the following pairs of clauses.

- a) $\{p \vee q \vee \neg r\}$ and $\{r \vee s\}$
- b) $\{p \vee q \vee r\}$ and $\{r \vee \neg s \vee \neg t\}$
- c) $\{\neg p \vee q \vee r\}$ and $\{p \vee \neg q \vee \neg r\}$

Automata Formal Languages and Logic

Unit 4 - Propositional Logic

Homework :

What are the results of applying Propositional Resolution to the following pairs of clauses.

Q	Given clauses	Result of applying Propositional Resolution
1	$p \vee q \vee \neg r \quad r \vee s$	$p \vee q \vee s$ (resolve away r)
2	$p \vee q \vee r \quad r \vee \neg s \vee \neg t$	$p \vee q \vee r \vee \neg s \vee \neg t$ (nothing can be resolved)
3	$\neg p \vee q \vee r \quad p \vee \neg q \vee \neg r$	False (resolve away p, q and r)



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724



Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Wumpus World

- An application of Propositional logic in the area of Artificial Intelligence

Preet Kanwal

Department of Computer Science & Engineering

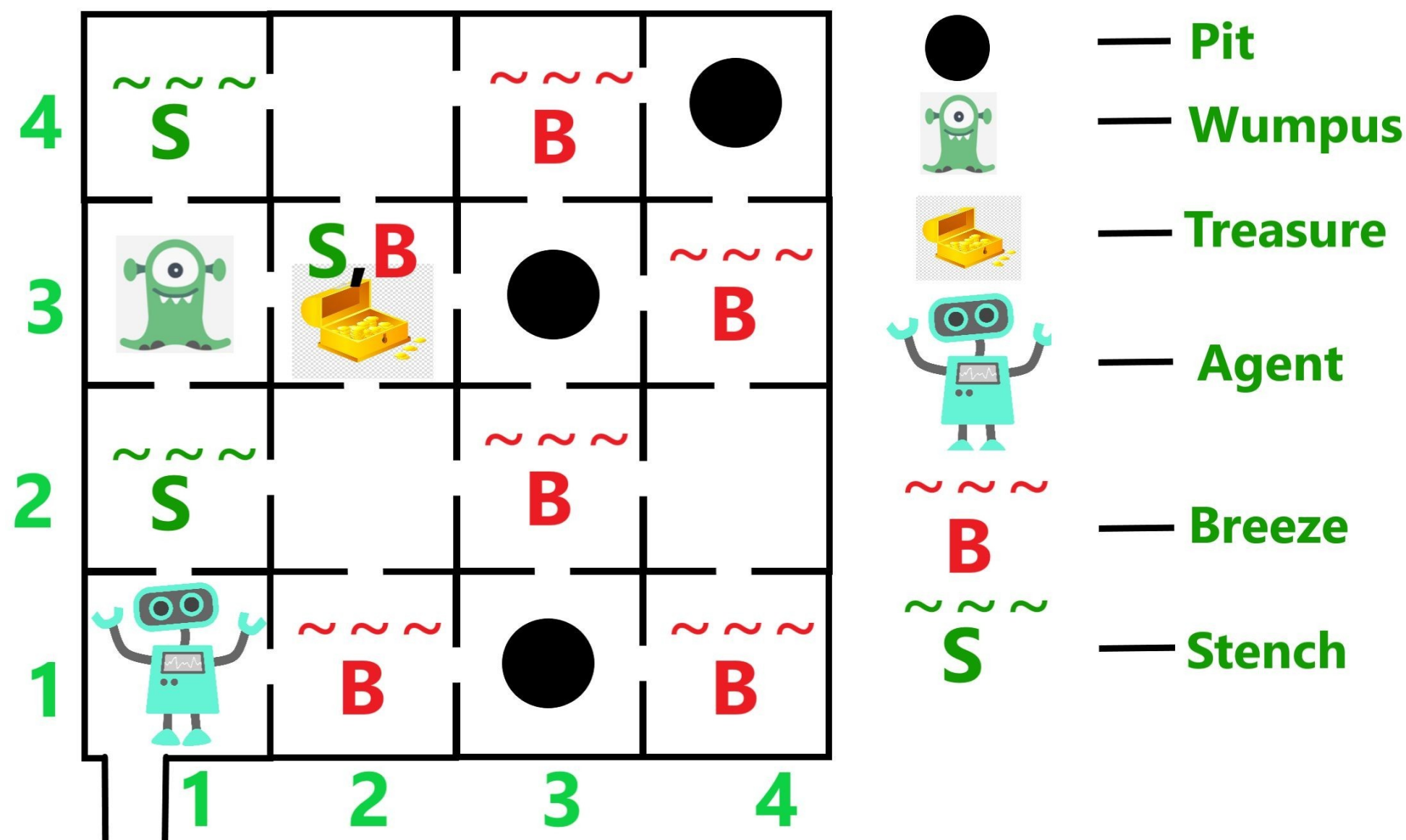
Automata Formal Languages and Logic

Unit 4 - The Wumpus World

Let us see What the game is about :

[wumpus world simulator](#)

AI is about
"Making computational models of human behavior that behave intelligently"



Automata Formal Languages and Logic

Unit 4 - The Wumpus World

Knowledge-based Agent :

It is a Software that gathers information about an environment and takes actions based on that(Basically a **Player** in a game), It could be :

- a robot
- a web shopping program
- a traffic control system...

Knowledge-based agents are composed of two main parts:

- Knowledge-base and
- Inference system.

Automata Formal Languages and Logic

Unit 4 - The Wumpus World

Knowledge-base:

Knowledge-base is a central component of a knowledge-based agent, it is also known as KB. It is a collection of sentences (Stores facts about the world)

These sentences are expressed in a knowledge representation language/ Declaration language for example : Propositional Logic or First-Order Predicate Logic.

Knowledge-base is required for updating knowledge for an agent to learn with experiences and take action as per the knowledge.

Automata Formal Languages and Logic

Unit 4 - The Wumpus World

Inference System:

- Inference means deriving new sentences from old.
- Inference system allows us to add a new sentence to the knowledge base.
- A sentence is a proposition about the world.
- Inference system applies logical rules to the KB to deduce new information (logical entailment).
- Inference system generates new facts so that an agent can update the KB.

Automata Formal Languages and Logic

Unit 4 - The Wumpus World

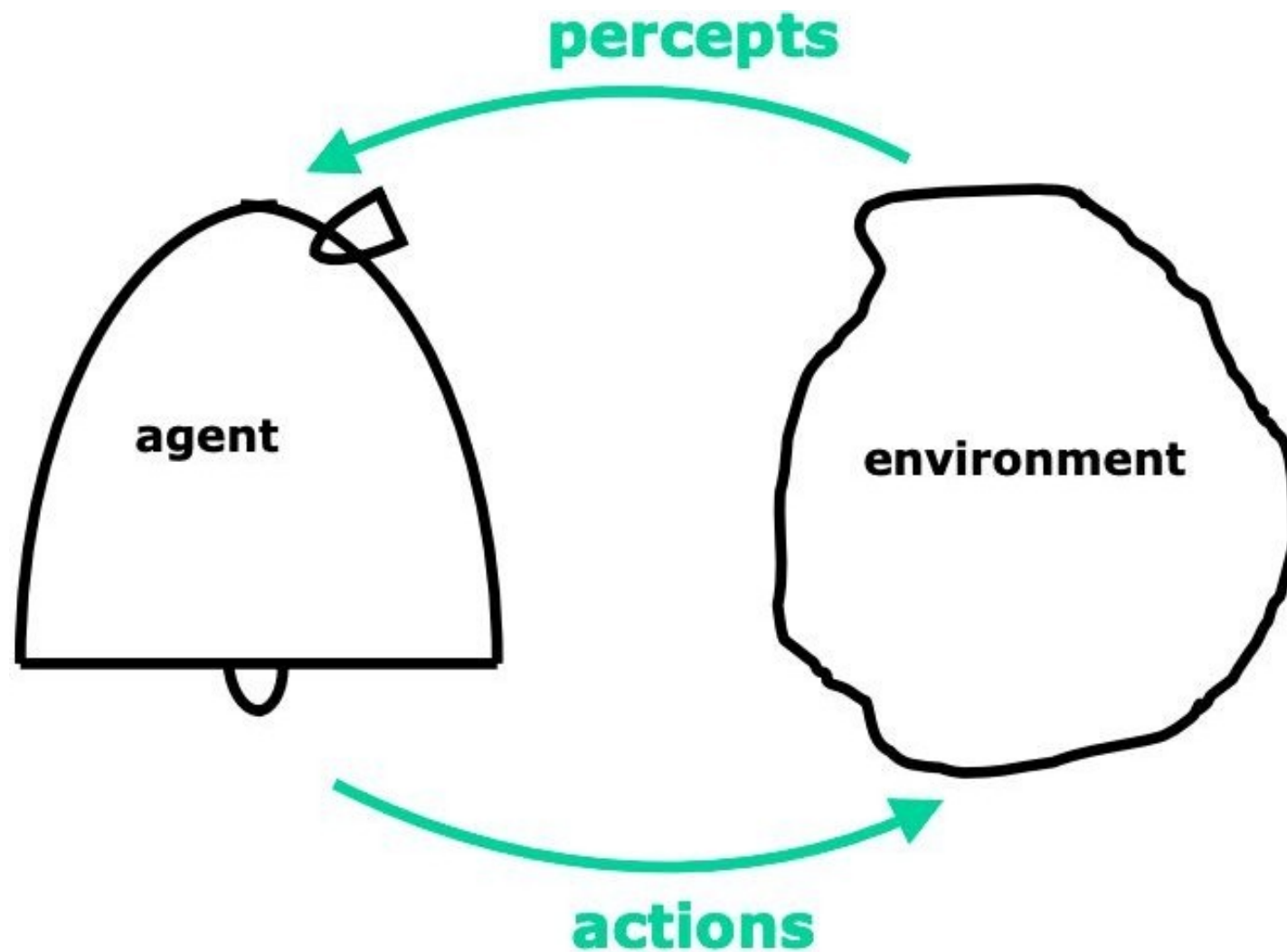
Our Objective :

- We will see how a knowledge-based agent can represent the world in which it operates (KB) and deduce what actions it should take.
- We will use Propositional Logic as representational language.

Automata Formal Languages and Logic

Unit 4 - The Wumpus World

The Agent and the Environment



In Wumpus World :

Agent - Player

Environment - Wumpus World Model

Actions - Moving left, right, up or down

Percepts - Knowledge about the environment received via the sensors for example:
[Stench, Breeze, Glitter, Bump, Scream]

Automata Formal Languages and Logic

Unit 4 - The Wumpus World

- The image specifies an agent(player) and the world(environment) it lives in.
- The agent is going to take actions that affect the state of the environment
- The agent receive percepts that provide information about what's happening in the environment. Agent updates its knowledge after observations
- The agent changes the state of the environment when it perform an action, it then perceives some new information about the state of the environment. This happens in a loop until the game is over!

Automata Formal Languages and Logic

Unit 4 - The Wumpus World

Three Operations performed by the Knowledge-based Agent:

- TELLS the knowledge base what it perceives from the environment. (Input)
- ASKS the knowledge base what action it should perform.
- PERFORMs the selected action. (Output)

The agent maintains the knowledge base (KB), and it initially has some background knowledge of the real world.

Automata Formal Languages and Logic

Unit 4 - The Wumpus World

How to define the Environment in which the agent lives: (World Model)

- **Action Space (Actuators)**
- **Percept Space (Sensors)**
- **Environment**

Automata Formal Languages and Logic

Unit 4 - The Wumpus World

How to define the Environment in which the agent lives: (World Model)

- **Action Space (Actuators)** : Actions that the agent can take for example:
 - Move Forward
 - Turn Left or Right by 90 degrees
 - Grab (to pick the gold)
 - Shoot (Shoot in Straight line)

Automata Formal Languages and Logic

Unit 4 - The Wumpus World

How to define the Environment in which the agent lives: (World Model)

- **Percept Space** : provides information about the environment with the help of the sensors:
 - If a cell contains the Wumpus, Agent perceives **Stench** in adjacent cells (not diagonally)
 - If a cell contains a Pit, Agent perceives **Breeze** in adjacent cells (not diagonally)
 - If a cell contains Gold, Agent perceives **Glitter** in the same cell.
 - When Agent walks into the wall, it perceives a **Bump**.
 - When the Agent is killed by Wumpus, a woeful **Scream** can be perceived anywhere in the cave.

Automata Formal Languages and Logic

Unit 4 - The Wumpus World

How to define the Environment in which the agent lives: (World Model)

- **Environment :**
 - 4 X 4 Grid of rooms
 - Agent always starts in the [1,1] cell facing to the right
 - Locations of Gold and Wumpus are chosen randomly with a uniform distribution (except [1,1]).
 - Any cell can be Pit (except [1,1]) with a Probability 0.2

Automata Formal Languages and Logic

Unit 4 - The Wumpus World

The game ends either when the agent dies or when the agent exits the game successfully.

Points are awarded each time the gold is obtained. You could also introduce points for other actions.

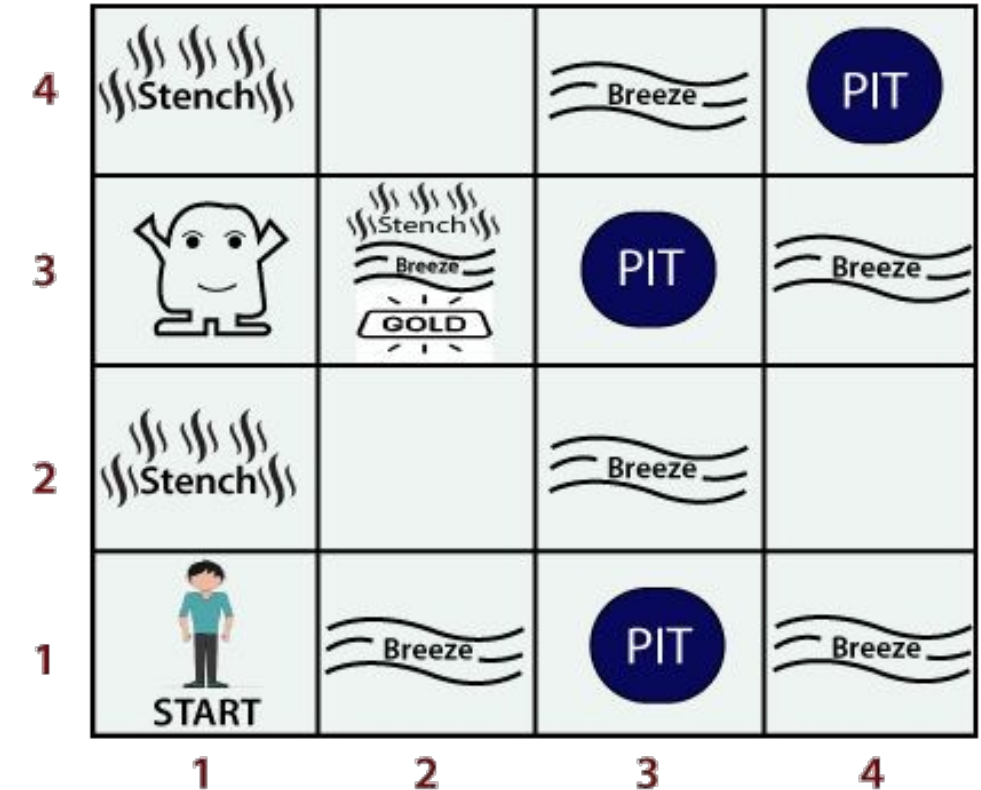
Automata Formal Languages and Logic

Unit 4 - The Wumpus World

The Wumpus World Knowledge Base :

Given the model, we define a set of propositions for each cell $[x,y]$

1. P_{xy} is true if there is a pit in $[x,y]$. There are 16 propositions in total since it is a 4 X 4 grid.
2. W_{xy} is true if there is a Wumpus in $[x,y]$. There are 16 propositions in total since it is a 4 X 4 grid.
3. B_{xy} is true if agent perceives a Breeze in $[x,y]$. There are 16 propositions in total since it is a 4 X 4 grid.
4. S_{xy} is true if agent perceives a Stench in $[x,y]$. There are 16 propositions in total since it is a 4 X 4 grid.



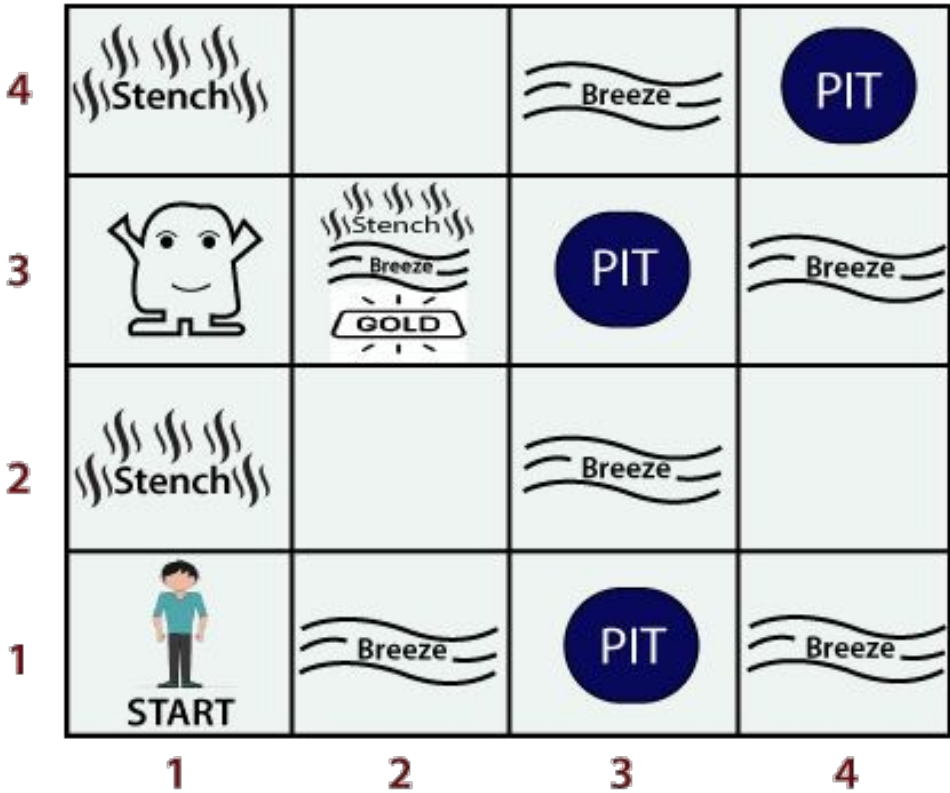
Automata Formal Languages and Logic

Unit 4 - The Wumpus World

The Wumpus World Knowledge Base :
 Given the model, we define a set of propositions for each cell [x,y]

- P_{xy} is true if there is a pit in [x,y]. There are 16 propositions in total since it is a 4 X 4 grid. That is,

Proposition	Truth Value
P_{11} - There is a pit in [1,1]	False
P_{12} - There is a pit in [1,2]	False
...	...
...	...
P_{44} - There is a pit in [4,4]	True



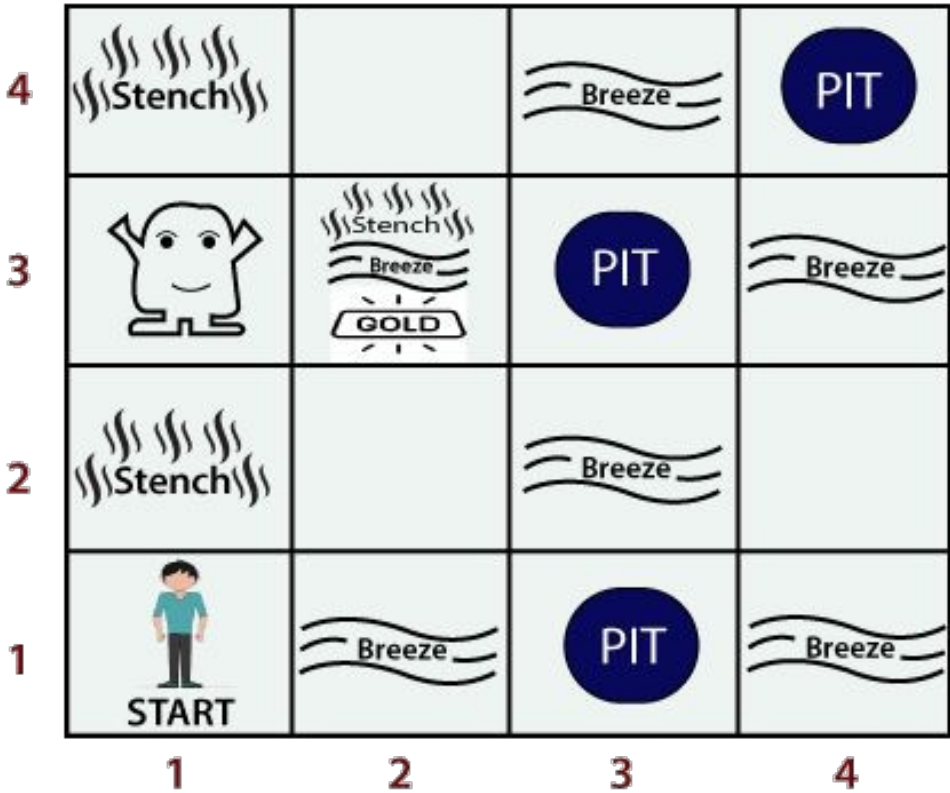
Automata Formal Languages and Logic

Unit 4 - The Wumpus World

The Wumpus World Knowledge Base :
Given the model, we define a set of propositions for each cell [x,y]

2. W_{xy} is true if there is a Wumpus in [x,y]. There are 16 propositions in total since it is a 4 X 4 grid. That is,

Proposition	Truth Value
W_{11} - There is a Wumpus in [1,1]	False
W_{12} - There is a Wumpus in [1,2]	False
W_{13} - There is a Wumpus in [1,3]	True
...	...
W_{44} - There is a Wumpus in [4,4]	False



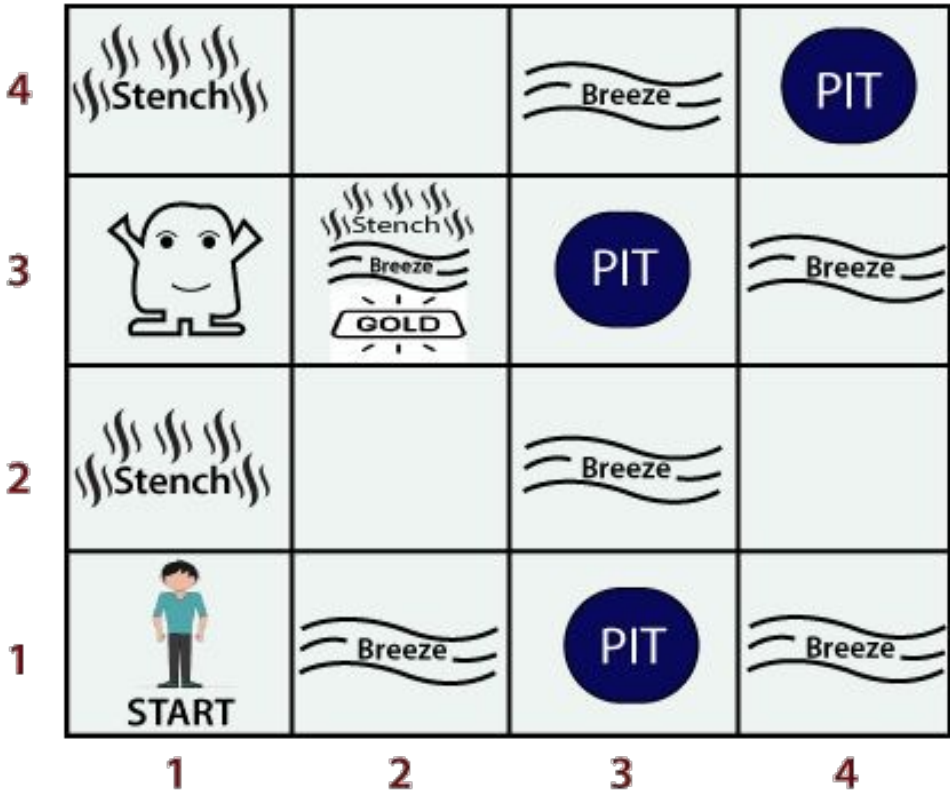
Automata Formal Languages and Logic

Unit 4 - The Wumpus World

The Wumpus World Knowledge Base :
Given the model, we define a set of propositions for each cell [x,y]

3. B_{xy} is true if agent perceives a Breeze in [x,y]. There are 16 propositions in total since it is a 4 X 4 grid. That is,

Proposition	Truth Value
B_{11} - There is a Breeze in [1,1]	False
B_{12} - There is a Breeze in [1,2]	False
B_{21} - There is a Breeze in [2,1]	True
...	...
B_{44} - There is a Breeze in [4,4]	False



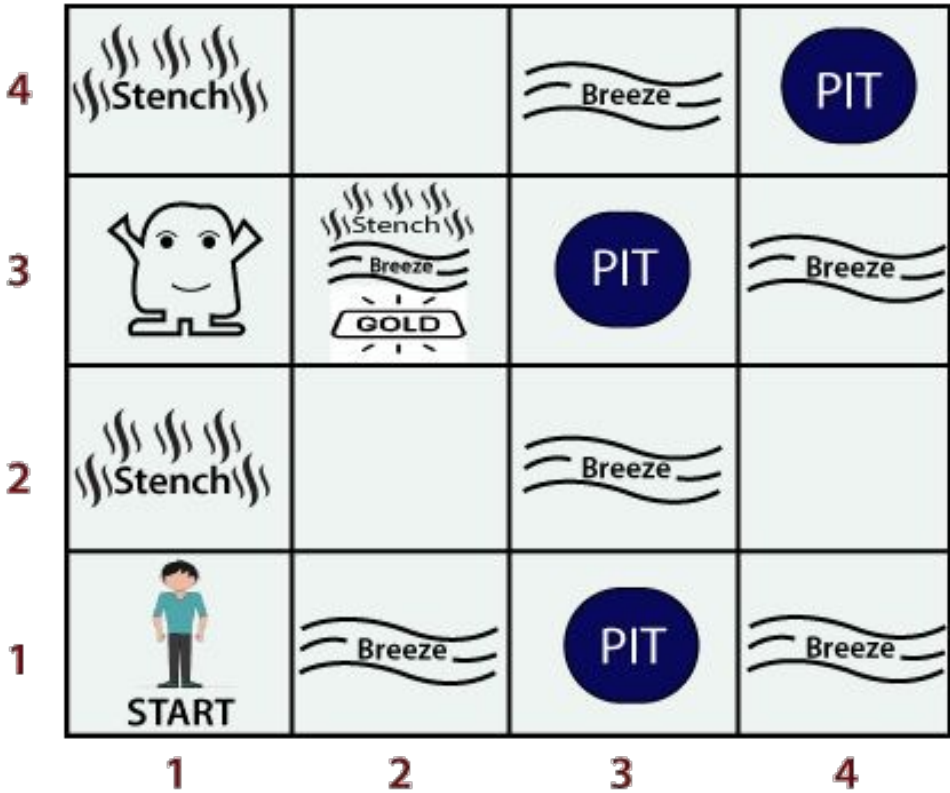
Automata Formal Languages and Logic

Unit 4 - The Wumpus World

The Wumpus World Knowledge Base :
Given the model, we define a set of propositions for each cell [x,y]

4. S_{xy} is true if agent perceives a Stench in [x,y]. There are 16 propositions in total since it is a 4 X 4 grid. That is,

Proposition	Truth Value
S_{11} - There is a Stench in [1,1]	False
S_{12} - There is a Stench in [1,2]	True
S_{21} - There is a Stench in [2,1]	False
...	...
S_{44} - There is a Stench in [4,4]	False

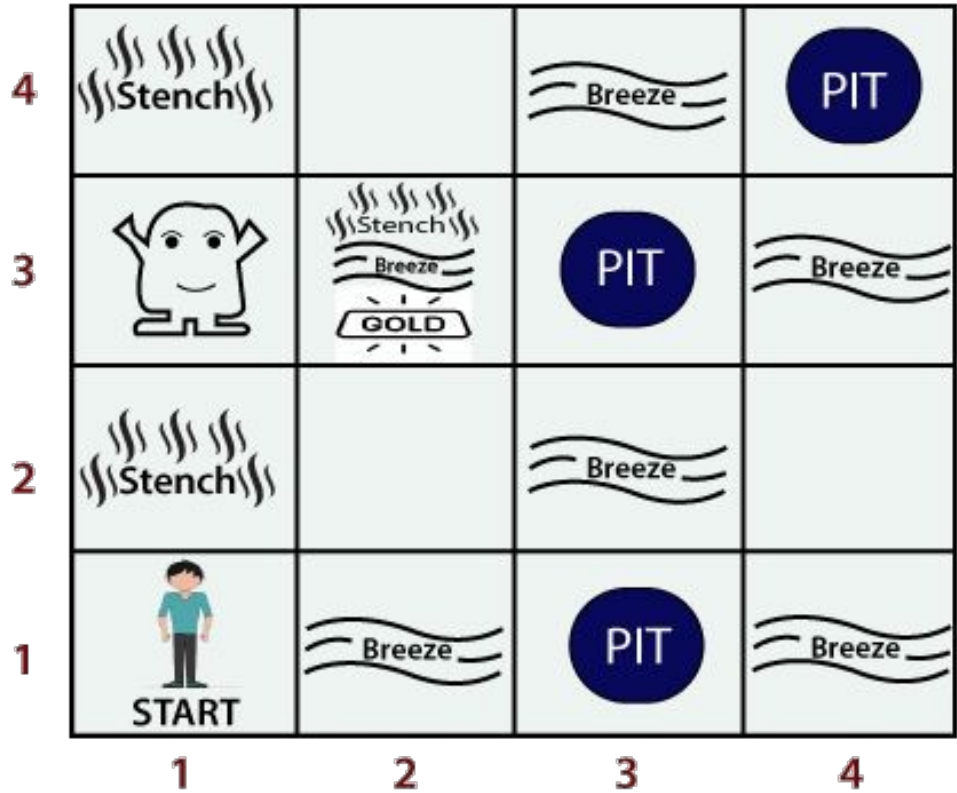


Automata Formal Languages and Logic

Unit 4 - The Wumpus World

Some Aspects of The Wumpus World

Proposition	Meaning
General Aspects - Immutable (Applies to all models) - facts known	
$R_1 : \neg P_{11}$	There is no pit in [1,1]
$R_2 : B_{11} \leftrightarrow (P_{12} \vee P_{21})$	There is a breeze in [1,1] iff there is a pit in adjoining cell i.e. [1,2] or [2,1]
$R_3 : B_{21} \leftrightarrow (P_{11} \vee P_{31} \vee P_{22})$	There is a breeze in [1,1] iff there is a pit in adjoining cells i.e. [1,1] or [3,1] or [2,2]
Aspects related to a Specific Model like the one shown in the figure - facts gained	
$R_4 : \neg B_{11}$	There is no breeze in [1,1]
$R_5 : B_{21}$	There is breeze in [2,1]



Automata Formal Languages and Logic

Unit 4 - The Wumpus World

The Wumpus World Knowledge Base Example:

Assume the Knowledge base (KB) is made up of the rules R_1 to R_5

The propositions involves in these rules are : B_{11} ,

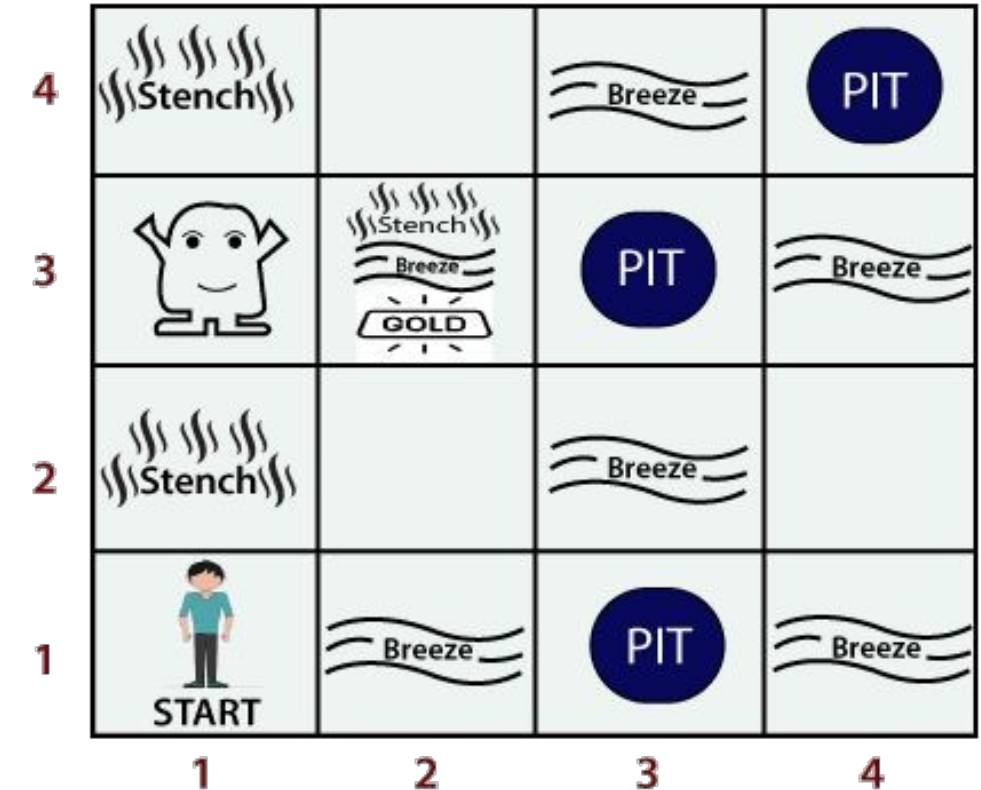
B_{21} , P_{11} , P_{12} , P_{21} , P_{22} , P_{31}

Can we entail any logical result from the KB?? i.e.

What is α ??

$KB \models \alpha$

α must be true in every model in which KB is true

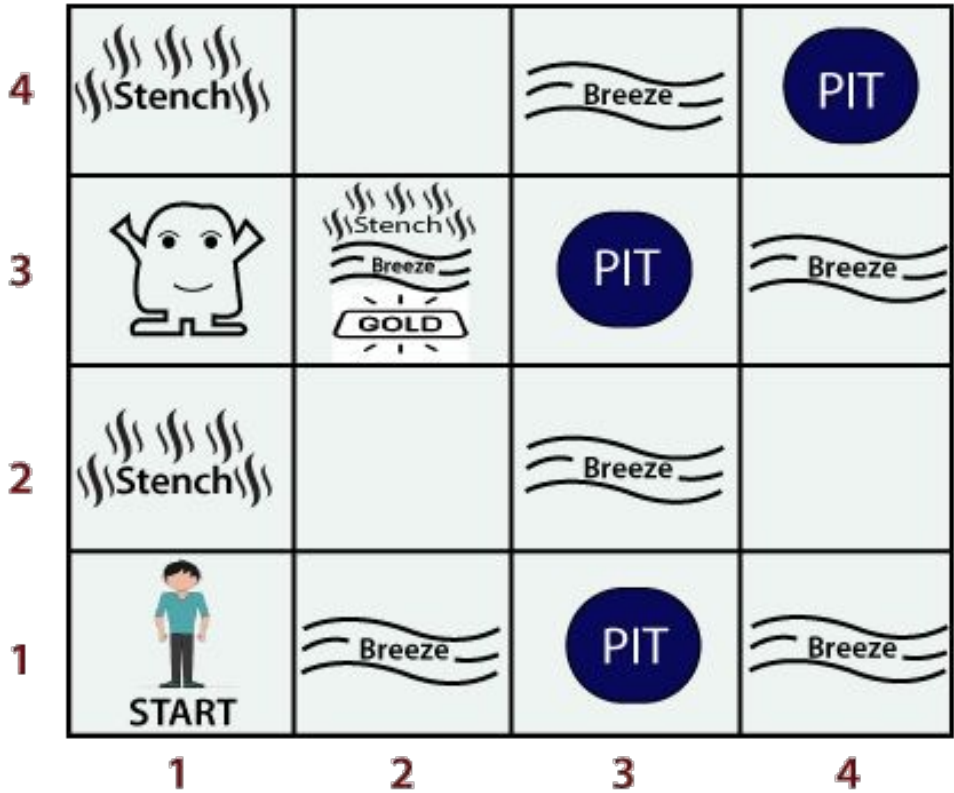


Automata Formal Languages and Logic

Unit 4 - The Wumpus World

We can create a Truth table for the KB involving these 7 propositions. We will have 2^7 entries in the truth table.(128 rows)

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	KB
false	false	false	false	false	false	false	false
false	false	false	false	false	false	true	false
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
false	true	false	false	false	false	false	false
false	true	false	false	false	false	true	<u>true</u>
false	true	false	false	false	true	false	<u>true</u>
false	true	false	false	false	true	true	<u>true</u>
false	true	false	false	true	false	false	false
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
true	true	true	true	true	true	true	false

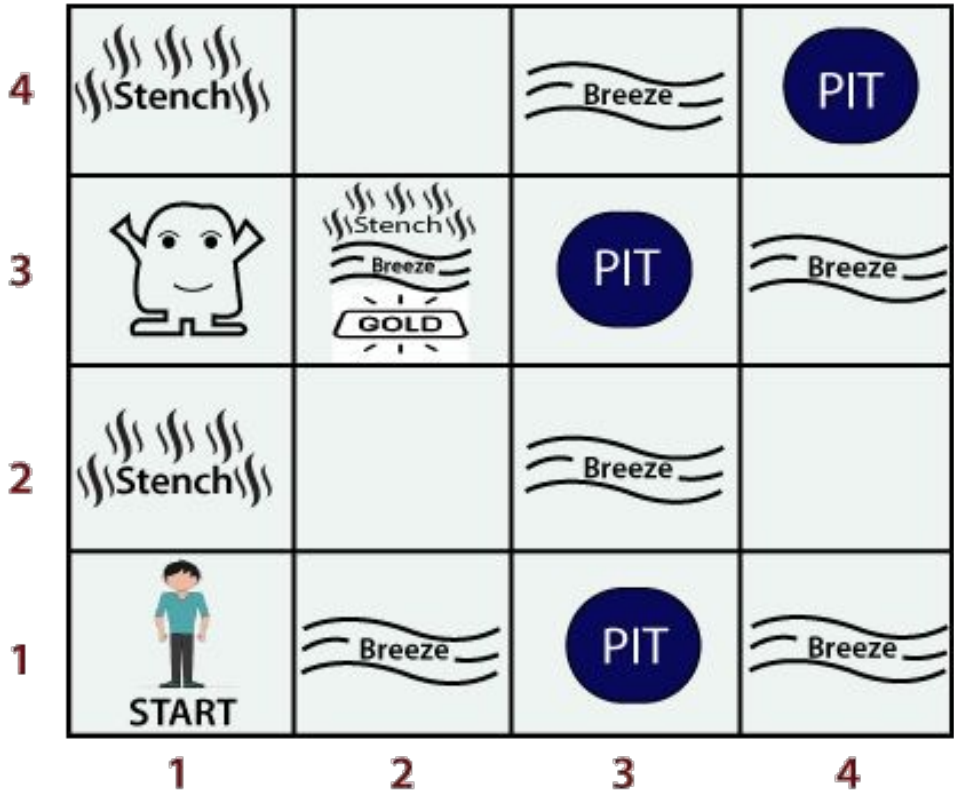


Automata Formal Languages and Logic

Unit 4 - The Wumpus World

Each row represents a Model.(It specifies each instance of game)
 Only in the highlighted models(rows), Our KB is true.
 Whenever our KB is true, we can infer other logical conclusions.

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	KB
false	false	false	false	false	false	false	false
false	false	false	false	false	false	true	false
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
false	true	false	false	false	false	false	false
false	true	false	false	false	false	true	<u>true</u>
false	true	false	false	false	true	false	<u>true</u>
false	true	false	false	false	true	true	<u>true</u>
false	true	false	false	true	false	false	false
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
true	true	true	true	true	true	true	false



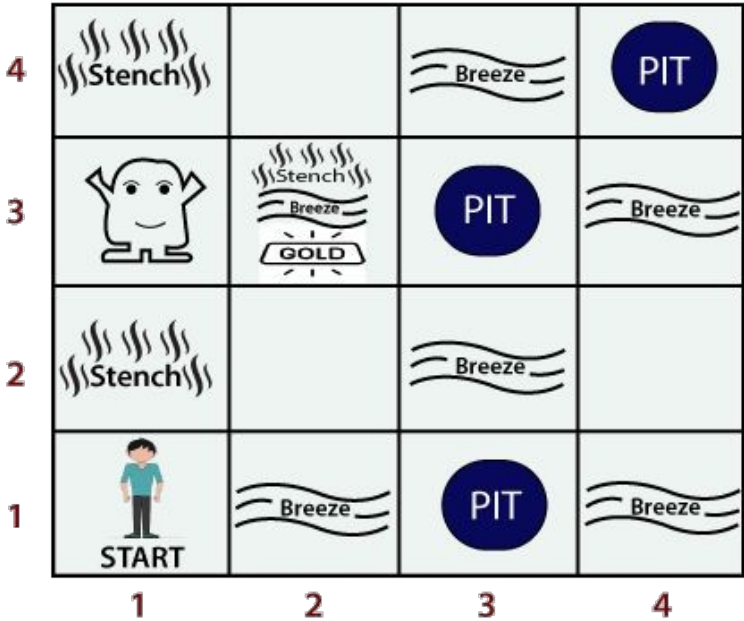
Automata Formal Languages and Logic

Unit 4 - The Wumpus World

Here, we can entail the following from the KB that :

- There is no Breeze in [1,1]
- There is a Breeze in [2, 1]
- There is no Pit in [1,1], [1,2], [2,1]

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	KB
false	false	false	false	false	false	false	false
false	false	false	false	false	false	true	false
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
false	true	false	false	false	false	false	false
false	true	false	false	false	false	true	<u>true</u>
false	true	false	false	false	true	false	<u>true</u>
false	true	false	false	false	true	true	<u>true</u>
false	true	false	false	true	false	false	false
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
true	true	true	true	true	true	true	false



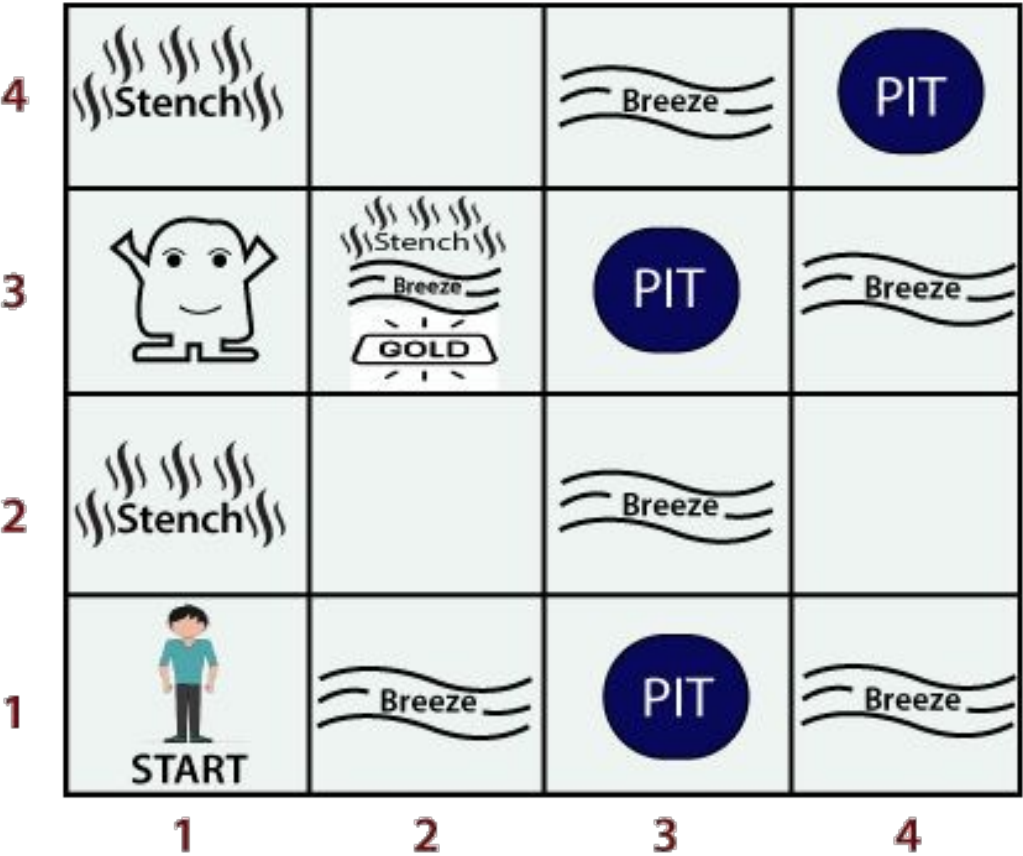
Automata Formal Languages and Logic

Unit 4 - The Wumpus World

Here, we cannot infer the following from the KB that :

- Whether or not there is Pit in [2,2] , [3,1]

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	KB
false	false	false	false	false	false	false	false
false	false	false	false	false	false	true	false
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
false	true	false	false	false	false	false	false
false	true	false	false	false	false	true	<u>true</u>
false	true	false	false	false	true	false	<u>true</u>
false	true	false	false	false	true	true	<u>true</u>
false	true	false	false	true	false	false	false
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
true	true	true	true	true	true	true	false



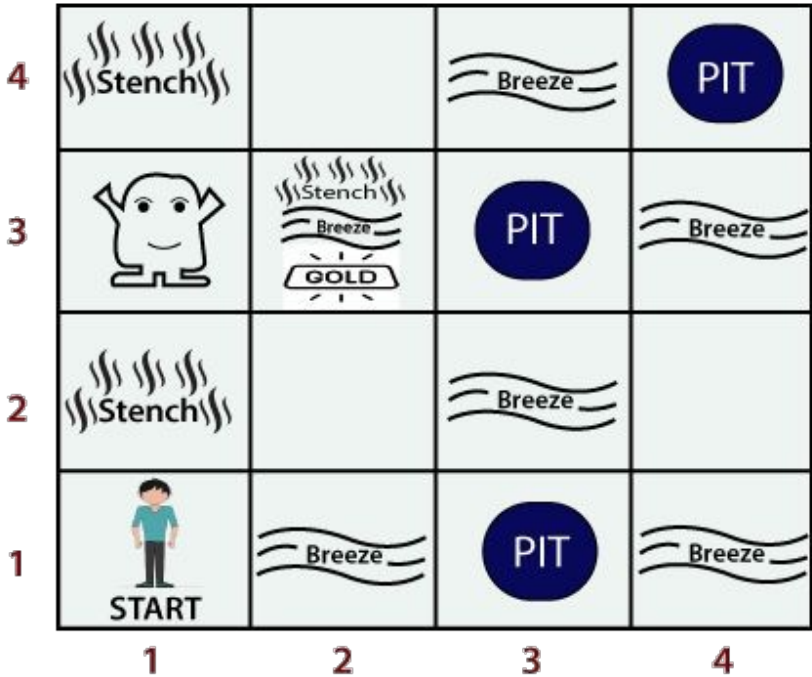
Automata Formal Languages and Logic

Unit 4 - The Wumpus World

Hence we can entail the following from the knowledge base (KB)

$$\begin{aligned}
 &KB \models \neg B_{1,1} \quad KB \models B_{2,1} \quad KB \models \neg P_{1,1} \quad KB \models \neg P_{1,2} \\
 &KB \models \neg P_{2,1} \quad KB \models \neg P_{2,2} \quad KB \models \neg P_{3,1} \quad KB \models \neg P_{12}
 \end{aligned}$$

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	KB
false	false	false	false	false	false	false	false
false	false	false	false	false	false	true	false
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
false	true	false	false	false	false	false	false
false	true	false	false	false	false	true	true
false	true	false	false	false	true	false	true
false	true	false	false	false	true	true	true
false	true	false	false	true	false	false	false
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
true	true	true	true	true	true	true	false



Automata Formal Languages and Logic

Unit 4 - The Wumpus World

Proof by Resolution that the KB $\models \neg$

Premises	Conclusion
$ \begin{array}{l} 1. \neg P_{11} \\ 2. B_{11} \leftrightarrow (P_{12} \vee P_{21}) \\ 3. B_{11} \leftrightarrow (P_{11} \vee P_{31}) \\ 4. \neg B_{11} \\ 5. B_{21} \end{array} $	$\neg P_{12}$

Automata Formal Languages and Logic

Unit 4 - The Wumpus World

Proof by Resolution that the KB $\models \neg$

Premises	Conclusion
1. $\neg P_{11}$ -ignore (not related) 2. $B_{11} \leftrightarrow (P_{12} \vee P_{21})$ 3. $B_{21} \leftrightarrow (P_{11} \vee P_{31} \vee P_{22})$ - ignore (no P term) 4. $\neg B_{11}$ 5. B_{21} - ignore (not related)	$\neg P_{12}$

Automata Formal Languages and Logic

Unit 4 - The Wumpus World

Proof by Resolution that the KB $\models \neg$

Premises	Conclusion	Solution
$1. B_{11} \leftrightarrow (P_{12} \vee P_{21})$ $2. \neg B_{11}$ Therefore we have: i) $\neg B_{11} \vee P_{12} \vee P_{21}$ ii) $\neg P_{12} \vee B_{11}$ iii) $\neg P_{21} \vee B_{11}$	$\neg P_{12}$	Converting $(B_{11} \rightarrow (P_{12} \vee P_{21})) \wedge ((P_{12} \vee P_{21}) \rightarrow B_{11})$ to CNF $= (\neg B_{11} \vee (P_{12} \vee P_{21})) \wedge (\neg (P_{12} \vee P_{21}) \vee B_{11})$ $= (\neg B_{11} \vee P_{12} \vee P_{21}) \wedge ((\neg P_{12} \wedge \neg P_{21}) \vee B_{11})$ $= (\neg B_{11} \vee P_{12} \vee P_{21}) \wedge ((\neg P_{12} \vee B_{11}) \wedge (\neg P_{21} \vee B_{11}))$ $= (\neg B_{11} \vee P_{12} \vee P_{21}) \wedge (\neg P_{12} \vee B_{11}) \wedge (\neg P_{21} \vee B_{11})$ Solution: Resolve away B_{11} using ii) and iv) result is $\neg P_{12}$. Using the above result $(\neg P_{12})$ and v) we get FALSE. Hence Conclusion logically follows from the premises.

Automata Formal Languages and Logic

Unit 4 - The Wumpus World

Satisfiability and validity:

- In mathematical logic, satisfiability and validity are elementary concepts of semantics.
- A sentence is satisfiable if it is possible to find an interpretation (model) that makes the formula true.
- In our example, KB is satisfiable because 3 models(rows) make the KB true. Hence, KB is satisfiable.
- A sentence is valid if all interpretations make the formula true. That is $KB \models \alpha$
is valid iff $KB \rightarrow \alpha$ is a tautology.

Automata Formal Languages and Logic

Unit 4 - The Wumpus World

Given the following Knowledge Base, how many models can you frame??

$$R_1: \neg P_{1,1}$$

$$R_2: B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

$$R_3: B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$$

$$R_4: \neg B_{1,1}$$

$$R_5: B_{2,1}$$

$$R_6: (B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$$

$$R_7: ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$$

$$R_8: (\neg B_{1,1} \Rightarrow \neg(P_{1,2} \vee P_{2,1}))$$

$$R_9: \neg(P_{1,2} \vee P_{2,1})$$

$$R_{10}: \neg P_{1,2} \wedge \neg P_{2,1}$$

$$R_{11}: \neg B_{1,2}$$

$$R_{12}: B_{1,2} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{1,3})$$

$$R_{13}: \neg P_{2,2}$$

$$R_{14}: \neg P_{1,3}$$

$$R_{15}: P_{1,1} \vee P_{2,2} \vee P_{3,1}$$

$$R_{16}: P_{1,1} \vee P_{3,1}$$

$$R_{17}: P_{3,1}$$

Automata Formal Languages and Logic

Unit 4 - The Wumpus World

Limitations of Propositional Logic:

- Can be used to model simple environments/models
- Does not capture Pattern

For example : We will have to write down a separate Rule for every cell about Breezes and Pits, although all the rules would convey the same information which is :

The Breeze exists in cell $[x,y]$ iff there is a Pit in adjoining cells i.e. it could in either of the four locations:

$[x, y+1]$ (up) or
 $[x, y-1]$ (down) or
 $[x-1, y]$ (left) or
 $[x+1,y]$ (right)

	Pit here	
Pit here	Breeze [x,y]	Pit here
	Pit here	

We overcome this limitation with the help of **First-Order Logic (FOPL)**.

Automata Formal Languages and Logic

Unit 4 - The Wumpus World

Limitations of Propositional Logic:

How can we say that adding 1 to an even number produces an odd number?

- We need infinite symbols and infinite rules.
- – A symbol O1 for “1 is odd”, a symbol E2 for “2 is even”, ...

What do these limitations buy us?

- Simple syntax: just symbols and connectives.
- Inference algorithms (like TT-Entails) that are horribly slow (exponential time), but at least terminate in finite time.



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724



Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 4 - First-Order Predicate Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Propositional Logic (PL)	First Order Predicate Logic (FOPL)
Declarative Language	Declarative Language; An extension to PL
Used to represent knowledge about simple environments	Used to represent knowledge about complex environments
Lacks Expressive power to concisely describe an environment Example : We are forced to write a separate rule about breezes and pits for each square	Can represent rules generally. Example : “Squares neighboring the wumpus are smelly”
Assumes the world contains facts	Assumes the world contains Objects, Relations and Functions.
facts either hold or do not hold	relations between objects hold or do not hold

~~Basic Elements which describe a Model in FOPL consists of:~~

- **Set of Objects** (called Domain of the Model).
- **Relations** (specify how objects are related, whether that relationship holds or not)
 - **Input** : Object(s) or Function(s)
 - **Output** : True or False
- **Functions** (A special case of a relation, where given object related to exactly one object). Must be a **Total Function** (i.e. It must be defined/provide output for all inputs of the right type)
 - **Input** : Object(s)
 - **Output** : Object

Basic Elements which describe a Model in FOPL consists of:

- **Set of Objects** (called Domain of the Model).
Example : A, B, people, numbers, colors, wars, theories, squares, pits, wumpus, John, Mary, house, backpack, Arlington, Texas...

Basic Elements which describe a Model in FOPL consists of:

- **Relations** (specify how objects are related, whether that relationship holds or not)
 - **Input** : Object(s) or Function(s)
 - **Output** : True or False

Example : Siblings(John, Mary), >(100, 5), Red(laptop551), Team(John, Mary, Sue, Jim)

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Basic Elements which describe a Model in FOPL consists of:

- **Functions** (A special case of a relation, where given object related to exactly one object). Must be a **Total Function** (i.e. It must be defined/provide output for all inputs of the right type)
 - **Input : Object(s)**
 - **Output : Object** **Example : Father(Raj), Capital(Texas), Mother(John), Sum(25, 12), sqrt(20),...**

Read the functions as : Father of Raj, Capital of Texas, Mother of John, Sum of 25 and 12, Sqrt of of

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Note : Both Relations and Functions can have any arity(i.e. any no. of arguments)

Should "sibling" be a relation or a function?

- Relation: siblings(John, Mary)
- Function: sibling(John) returns Mary.

• "Sibling" should be a **relation but not function**, because someone can have many (or no) siblings. A function must and can only return one value.

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Note : Both Relations and Functions can have any arity(i.e. any no. of arguments)

Should "mother" be a relation or a function?

- Relation: mother(Liza, John)
- Function: mother(John) returns Liza.

"Mother" can be either a relation or a function.

- A person (or animal) has only one mother.

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

First-order Predicate Logic as a Declarative language has:

- **A Syntax**
- **Semantics**

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Basic Elements of First-Order logic Syntax:

Constant Symbols (Objects)	1, 2, A John, Mumbai, Person, Chair
Predicate Symbols (Relations)	Brother, Father, >,
Function Symbols	sqrt, multiply
Variables	x,y,z,a,b...
Term	Constant Symbol or Function Symbol or Variable
Equality	=
Connectives	$\neg$, $\vee$, $\wedge$, $\rightarrow$, $\leftrightarrow$
Quantifier	$\forall$, $\exists$

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Syntax of First-Order logic:

- Model in First-Order Logic must be specified as per the Syntax.
- The syntax of FOL determines which collection of symbols is a logical expression in first-order logic.

Basic Syntactic elements of first-order logic are symbols, which are:

- Constant Symbols (Objects)
- Predicate Symbols (Relations)
- Function Symbols (Functions)

Model must provide the **Interpretation**, that is :

- Which objects are referred by the Constant Symbols (as a constant symbol can map to any object).
- Which relations and functions are referred by Predicate and Function Symbols respectively.

The model must provide all the information required to determine the truth value of any given sentence.

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Syntax of First-Order logic (Continued ...) :

Quantifiers in First-order logic:

What makes first-order logic powerful is that **it allows us to make general assertions using quantifiers**. These are the symbols that permit to determine or identify the range and scope of the variable in the logical expression. There are two types of quantifier:

- **Universal Quantifier**, (for all, everyone, everything) : If x is a variable, then $\forall x$ is read as:
 - For all x
 - For each x
 - For every x .
- **Existential quantifier**, (for some, at least one). If x is a variable, then $\exists x$ is read as:
 - There exists a ' x .'
 - For some ' x .'
 - For at least one ' x .'

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Syntax of First-Order logic (Continued ...)

Points to remember:

- The main connective for universal quantifier $\forall$ is implication $\rightarrow$.
- The main connective for existential quantifier $\exists$ is and $\wedge$.

Properties of Quantifiers:

- In universal quantifier, $\forall x \forall y$ is similar to $\forall y \forall x$.
- In Existential quantifier, $\exists x \exists y$ is similar to $\exists y \exists x$.
- $\exists x \forall y$ is not similar to $\forall y \exists x$.

Let the **domain** be all the animals.

$D(x)$: x is a dog $M(x)$: x is a mammal

$C(x)$: x is cute

“All dogs are mammals” is equivalent to

$\forall x (D(x) \rightarrow M(x))$ or $\forall x (D(x) \wedge M(x))$?

“Some dogs are cute” is equivalent to

$\exists x (D(x) \rightarrow C(x))$ or $\exists x (D(x) \wedge C(x))$?

Explanation for: “All dogs are mammals”

What does $\forall x (D(x) \rightarrow M(x))$ mean?

$\forall x (D(x) \rightarrow M(x)) \equiv \forall x (\neg D(x) \vee M(x))$ All animal is a non-dog or a mammal.

It means all dogs are mammals.

What does $\forall x (D(x) \wedge M(x))$ mean?

$\forall x (D(x) \wedge M(x)) \equiv (\forall x D(x)) \wedge (\forall x M(x))$

All animals are dogs and all animals are mammals.

It is not same as “All dogs are mammals”.

Explanation for: “Some dogs are Cute”

What does $\exists x (D(x) \rightarrow C(x))$ mean?

There exists an animal, if that animal is a Dog, then it is cute. It means all dogs are cute.

It is not same as “Some dogs are Cute”

What does $\exists x (D(x) \wedge C(x))$ mean?

There exists some animal, which is a dog and is cute.

Please note the question says only few dogs are cute and not all.

Hence, there could exist a dog who is not cute.

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Syntax of First-Order logic (Continued ...) :

Free and Bound Variables:

The quantifiers interact with variables which appear in a suitable way. There are two types of variables in First-order logic which are given below:

Free Variable: A variable is said to be a free variable in a formula if it occurs outside the scope of the quantifier.

Example: $\forall x \exists (y)[P(x, y, z)]$, where z is a free variable.

Bound Variable: A variable is said to be a bound variable in a formula if it occurs within the scope of the quantifier.

Example: $\forall x \forall y [A(x) B(y)]$, here x and y are the bound variables.

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Syntax of First-Order logic (Continued ...) :

Variable :

- Can take values in a domain. (Domain is a set of objects)
- Can only be used together with quantifiers.

Example (Here x and y are variables) :

$$\forall x, y \quad \text{brothers}(x, y) \Rightarrow \text{siblings}(x, y)$$

Meaning: For every x and y, if x is a brother of y then x is a sibling of y

$$\exists x \quad 2 * 5 + x = 18$$

Meaning: There exists x such that result of $2 * 5 + x$ is equal to 18

Syntax of First-Order logic (Continued ...) :

Term can be:

- Constant Symbols (Objects)
- Function Symbols (Functions which can take Term as argument)
- Variable

Predicate symbols can take Term as argument :

- Which objects are referred by the Constant Symbols (as a constant symbol can map to any object).
- Which relations and functions are referred by Predicate and Function Symbols respectively.

Term = Term:

- Equality symbol (=) is used to signify that two terms refer to the same object.

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Syntax of First-Order logic (Continued ...) :

Atomic sentences:

- Atomic sentences are the most basic sentences of first-order logic.
- These sentences are formed from a predicate symbol followed by a parenthesis with a sequence of terms.

We can represent atomic sentences as **Predicate (term1, term2,, term n).**

In a Prolog program, this refers to a fact

Example: Ravi and Ajay are brothers: $\Rightarrow$ Brothers(Ravi, Ajay). elephant is bigger than horse $\Rightarrow$ bigger(elephant, horse). Kitty is a cat: $\Rightarrow$ cat (Kitty).

Note: Functions do not state facts and form no sentence.

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Syntax of First-Order logic (Continued ...) :

Complex Sentences:

Complex sentences are made by combining atomic sentences using connectives($\neg$, $\vee$, $\wedge$, $\rightarrow$, $\leftrightarrow$).

Example:

find(X, Z):- (bigger(X,Y), bigger(Y,Z)) ; bigger(X,Z).

[In prolog user must write a rule to specify complex sentence]

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

$$\begin{aligned}
 \text{Sentence} &\rightarrow \text{AtomicSentence} \mid \text{ComplexSentence} \\
 \text{AtomicSentence} &\rightarrow \text{Predicate} \mid \text{Predicate}(\text{Term}, \dots) \mid \text{Term} = \text{Term} \\
 \text{ComplexSentence} &\rightarrow (\text{Sentence}) \mid [\text{Sentence}] \\
 &\mid \neg \text{Sentence} \\
 &\mid \text{Sentence} \wedge \text{Sentence} \\
 &\mid \text{Sentence} \vee \text{Sentence} \\
 &\mid \text{Sentence} \Rightarrow \text{Sentence} \\
 &\mid \text{Sentence} \Leftrightarrow \text{Sentence} \\
 &\mid \text{Quantifier Variable}, \dots \text{Sentence} \\
 \\
 \text{Term} &\rightarrow \text{Function}(\text{Term}, \dots) \\
 &\mid \text{Constant} \\
 &\mid \text{Variable} \\
 \\
 \text{Quantifier} &\rightarrow \forall \mid \exists \\
 \text{Constant} &\rightarrow A \mid X_1 \mid \text{John} \mid \dots \\
 \text{Variable} &\rightarrow a \mid x \mid s \mid \dots \\
 \text{Predicate} &\rightarrow \text{True} \mid \text{False} \mid \text{After} \mid \text{Loves} \mid \text{Raining} \mid \dots \\
 \text{Function} &\rightarrow \text{Mother} \mid \text{LeftLeg} \mid \dots
 \end{aligned}$$

OPERATOR PRECEDENCE : $\neg, =, \wedge, \vee, \Rightarrow, \Leftrightarrow$

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Number of Possible Worlds/Models :

In propositional logic, suppose that you have 100 symbols. How many possible worlds do you have?

– 2^{100}

– One for each row of the truth table.

In first-order logic, how can we even count the number of possible worlds? Number of

possible worlds is counted in terms of:

- The number of objects
- Mapping of Constant Symbols to Objects (No. of predicates and its arity)

Number of possible models are unbounded. Checking entailment via Model Checking is not feasible as the number of combinations can become very large.

Number of Possible worlds in FOPL:

Suppose that we have:

- Five constants.
 - No functions.
 - One predicate, that takes one argument.
- How many possible worlds can we have?
 - For each possible argument of the predicate, we must specify if the predicate returns true or false.
 - We have five possible arguments.
 - In total, 2^5 possible worlds.

Number of Possible worlds in FOPL:

Suppose that we have:

- Let the Five constants be V, W, X, Y, Z
- No functions.
- Let the predicate be P, that takes one argument(either of V, W, X, Y, Z) .

There will be 2^5 possible worlds/models (each row of the truth table is a model).

Model	P(V)	P(W)	P(X)	P(Y)	P(Z)
1	<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>
2	<i>true</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>
...	...	...	...	...	...
...	...	...	...	...	...
$2^5=32$	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>

Number of Possible worlds in FOPL:

Suppose that we have:

- Five constants.
 - No functions.
 - One predicate, that takes two arguments.
- How many possible worlds can we have?
 - For each possible combination of arguments of the predicate, we must specify if the predicate returns true or false.
 - We have 25 possible combinations of arguments.
 - In total, 2^{25} possible worlds.

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Number of Possible worlds in FOPL:

Suppose that we have:

- Let the Five constants be V, W, X, Y, Z
- No functions.
- Let the predicate be P, that takes two arguments(either of V, W, X, Y, Z).

• We have 2^{25} possible worlds/models

Model	P(V,V)	P(V,W)	...	...	P(Z,Z)
1	false	false	false	false	false
2	true	false	false	false	false
...	...	...	...	...	...
...	...	...	...	...	...
2^{25}	true	true	true	true	true

Number of Possible worlds in FOPL:

Suppose that we have:

- Five constants.
 - No functions.
 - Three predicates, that take two arguments.
 - One predicate that takes one argument.
-
- How many possible worlds can we have?
 - For each possible combination of arguments of each predicate, we must specify if the predicate returns true or false.
 - For two arguments, we have 25 combinations of arguments. We have three such predicates, so we must specify 75 values.
 - For one argument, we have 5 combinations of arguments. We have one such predicate, so we must specify 5 values.
 - In total, $2^{5+75} = 2^{80}$ possible worlds.

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Example:

For the wumpus world, to say that "pits cause breezes in adjacent squares" using **propositional logic**, we need 16 rules like:

$$B_{12} \rightarrow (P_{11} \vee P_{22} \vee P_{13})$$

In **first-order logic**, how can we say the same thing?

Objects : **Pits, Squares** Relations : **Breeze, adjacent**

$$\forall x_1, y_1 \text{ Breeze}(x_1, y_1) \leftrightarrow (\exists x_2, y_2 \text{ Pit}(x_2, y_2) \text{ AND Adjacent}(x_1, y_1, x_2, y_2))$$



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724



Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 4 - First-Order Predicate Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Propositional Logic (PL)	First Order Predicate Logic (FOPL)
Declarative Language	Declarative Language; An extension to PL
Used to represent knowledge about simple environments	Used to represent knowledge about complex environments
Lacks Expressive power to concisely describe an environment Example : We are forced to write a separate rule about breezes and pits for each square	Can represent rules generally. Example : “Squares neighboring the wumpus are smelly”
Assumes the world contains facts	Assumes the world contains Objects, Relations and Functions.
facts either hold or do not hold	relations between objects hold or do not hold

~~Basic Elements which describe a Model in FOPL consists of:~~

- **Set of Objects** (called Domain of the Model).
- **Relations** (specify how objects are related, whether that relationship holds or not)
 - **Input** : Object(s) or Function(s)
 - **Output** : True or False
- **Functions** (A special case of a relation, where given object related to exactly one object). Must be a **Total Function** (i.e. It must be defined/provide output for all inputs of the right type)
 - **Input** : Object(s)
 - **Output** : Object

Basic Elements which describe a Model in FOPL consists of:

- **Set of Objects** (called Domain of the Model).
Example : A, B, people, numbers, colors, wars, theories, squares, pits, wumpus, John, Mary, house, backpack, Arlington, Texas...

Basic Elements which describe a Model in FOPL consists of:

- **Relations** (specify how objects are related, whether that relationship holds or not)
 - **Input** : Object(s) or Function(s)
 - **Output** : True or False

Example : Siblings(John, Mary), >(100, 5), Red(laptop551), Team(John, Mary, Sue, Jim)

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Basic Elements which describe a Model in FOPL consists of:

- **Functions** (A special case of a relation, where given object related to exactly one object). Must be a **Total Function** (i.e. It must be defined/provide output for all inputs of the right type)
 - **Input : Object(s)**
 - **Output : Object** **Example : Father(Raj), Capital(Texas), Mother(John), Sum(25, 12), sqrt(20),...**

Read the functions as : Father of Raj, Capital of Texas, Mother of John, Sum of 25 and 12, Sqrt of of

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Note : Both Relations and Functions can have any arity(i.e. any no. of arguments)

Should "sibling" be a relation or a function?

- Relation: siblings(John, Mary)
- Function: sibling(John) returns Mary.

• "Sibling" should be a **relation but not function**, because someone can have many (or no) siblings. A function must and can only return one value.

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Note : Both Relations and Functions can have any arity(i.e. any no. of arguments)

Should "mother" be a relation or a function?

- Relation: mother(Liza, John)
- Function: mother(John) returns Liza.

"Mother" can be either a relation or a function.

- A person (or animal) has only one mother.

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

First-order Predicate Logic as a Declarative language has:

- **A Syntax**
- **Semantics**

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Basic Elements of First-Order logic Syntax:

Constant Symbols (Objects)	1, 2, A John, Mumbai, Person, Chair
Predicate Symbols (Relations)	Brother, Father, >,
Function Symbols	sqrt, multiply
Variables	x,y,z,a,b...
Term	Constant Symbol or Function Symbol or Variable
Equality	=
Connectives	$\neg$, $\vee$, $\wedge$, $\rightarrow$, $\leftrightarrow$
Quantifier	$\forall$, $\exists$

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Syntax of First-Order logic:

- Model in First-Order Logic must be specified as per the Syntax.
- The syntax of FOL determines which collection of symbols is a logical expression in first-order logic.

Basic Syntactic elements of first-order logic are symbols, which are:

- Constant Symbols (Objects)
- Predicate Symbols (Relations)
- Function Symbols (Functions)

Model must provide the **Interpretation**, that is :

- Which objects are referred by the Constant Symbols (as a constant symbol can map to any object).
- Which relations and functions are referred by Predicate and Function Symbols respectively.

The model must provide all the information required to determine the truth value of any given sentence.

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Syntax of First-Order logic (Continued ...) :

Quantifiers in First-order logic:

What makes first-order logic powerful is that **it allows us to make general assertions using quantifiers**. These are the symbols that permit to determine or identify the range and scope of the variable in the logical expression. There are two types of quantifier:

- **Universal Quantifier**, (for all, everyone, everything) : If x is a variable, then $\forall x$ is read as:
 - For all x
 - For each x
 - For every x .
- **Existential quantifier**, (for some, at least one). If x is a variable, then $\exists x$ is read as:
 - There exists a ' x .'
 - For some ' x .'
 - For at least one ' x .'

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Syntax of First-Order logic (Continued ...)

Points to remember:

- The main connective for universal quantifier $\forall$ is implication $\rightarrow$.
- The main connective for existential quantifier $\exists$ is and $\wedge$.

Properties of Quantifiers:

- In universal quantifier, $\forall x \forall y$ is similar to $\forall y \forall x$.
- In Existential quantifier, $\exists x \exists y$ is similar to $\exists y \exists x$.
- $\exists x \forall y$ is not similar to $\forall y \exists x$.

Let the **domain** be all the animals.

$D(x)$: x is a dog $M(x)$: x is a mammal

$C(x)$: x is cute

“All dogs are mammals” is equivalent to

$\forall x (D(x) \rightarrow M(x))$ or $\forall x (D(x) \wedge M(x))$?

“Some dogs are cute” is equivalent to

$\exists x (D(x) \rightarrow C(x))$ or $\exists x (D(x) \wedge C(x))$?

Explanation for: “All dogs are mammals”

What does $\forall x (D(x) \rightarrow M(x))$ mean?

$\forall x (D(x) \rightarrow M(x)) \equiv \forall x (\neg D(x) \vee M(x))$ All animal is a non-dog or a mammal.

It means all dogs are mammals.

What does $\forall x (D(x) \wedge M(x))$ mean?

$\forall x (D(x) \wedge M(x)) \equiv (\forall x D(x)) \wedge (\forall x M(x))$

All animals are dogs and all animals are mammals.

It is not same as “All dogs are mammals”.

Explanation for: “Some dogs are Cute”

What does $\exists x (D(x) \rightarrow C(x))$ mean?

There exists an animal, if that animal is a Dog, then it is cute. It means all dogs are cute.

It is not same as “Some dogs are Cute”

What does $\exists x (D(x) \wedge C(x))$ mean?

There exists some animal, which is a dog and is cute.

Please note the question says only few dogs are cute and not all.

Hence, there could exist a dog who is not cute.

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Syntax of First-Order logic (Continued ...)

Free and Bound Variables:

The quantifiers interact with variables which appear in a suitable way. There are two types of variables in First-order logic which are given below:

Free Variable: A variable is said to be a free variable in a formula if it occurs outside the scope of the quantifier.

Example: $\forall x \exists (y)[P(x, y, z)]$, where z is a free variable.

Bound Variable: A variable is said to be a bound variable in a formula if it occurs within the scope of the quantifier.

Example: $\forall x \forall y [A(x) B(y)]$, here x and y are the bound variables.

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Syntax of First-Order logic (Continued ...) :

Variable :

- Can take values in a domain. (Domain is a set of objects)
- Can only be used together with quantifiers.

Example (Here x and y are variables) :

$$\forall x, y \quad \text{brothers}(x, y) \Rightarrow \text{siblings}(x, y)$$

Meaning: For every x and y, if x is a brother of y then x is a sibling of y

$$\exists x \quad 2 * 5 + x = 18$$

Meaning: There exists x such that result of $2 * 5 + x$ is equal to 18

Syntax of First-Order logic (Continued ...) :

Term can be:

- Constant Symbols (Objects)
- Function Symbols (Functions which can take Term as argument)
- Variable

Predicate symbols can take Term as argument :

- Which objects are referred by the Constant Symbols (as a constant symbol can map to any object).
- Which relations and functions are referred by Predicate and Function Symbols respectively.

Term = Term:

- Equality symbol (=) is used to signify that two terms refer to the same object.

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Syntax of First-Order logic (Continued ...) :

Atomic sentences:

- Atomic sentences are the most basic sentences of first-order logic.
- These sentences are formed from a predicate symbol followed by a parenthesis with a sequence of terms.

We can represent atomic sentences as **Predicate (term1, term2,, term n).**

In a Prolog program, this refers to a fact

Example: Ravi and Ajay are brothers: $\Rightarrow$ Brothers(Ravi, Ajay). elephant is bigger than horse $\Rightarrow$ bigger(elephant, horse). Kitty is a cat: $\Rightarrow$ cat (Kitty).

Note: Functions do not state facts and form no sentence.

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Syntax of First-Order logic (Continued ...) :

Complex Sentences:

Complex sentences are made by combining atomic sentences using connectives($\neg$, $\vee$, $\wedge$, $\rightarrow$, $\leftrightarrow$).

Example:

find(X, Z):- (bigger(X,Y), bigger(Y,Z)) ; bigger(X,Z).

[In prolog user must write a rule to specify complex sentence]

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

$$\begin{aligned}
 \text{Sentence} &\rightarrow \text{AtomicSentence} \mid \text{ComplexSentence} \\
 \text{AtomicSentence} &\rightarrow \text{Predicate} \mid \text{Predicate}(\text{Term}, \dots) \mid \text{Term} = \text{Term} \\
 \text{ComplexSentence} &\rightarrow (\text{Sentence}) \mid [\text{Sentence}] \\
 &\mid \neg \text{Sentence} \\
 &\mid \text{Sentence} \wedge \text{Sentence} \\
 &\mid \text{Sentence} \vee \text{Sentence} \\
 &\mid \text{Sentence} \Rightarrow \text{Sentence} \\
 &\mid \text{Sentence} \Leftrightarrow \text{Sentence} \\
 &\mid \text{Quantifier Variable}, \dots \text{Sentence} \\
 \\
 \text{Term} &\rightarrow \text{Function}(\text{Term}, \dots) \\
 &\mid \text{Constant} \\
 &\mid \text{Variable} \\
 \\
 \text{Quantifier} &\rightarrow \forall \mid \exists \\
 \text{Constant} &\rightarrow A \mid X_1 \mid \text{John} \mid \dots \\
 \text{Variable} &\rightarrow a \mid x \mid s \mid \dots \\
 \text{Predicate} &\rightarrow \text{True} \mid \text{False} \mid \text{After} \mid \text{Loves} \mid \text{Raining} \mid \dots \\
 \text{Function} &\rightarrow \text{Mother} \mid \text{LeftLeg} \mid \dots
 \end{aligned}$$

OPERATOR PRECEDENCE : $\neg, =, \wedge, \vee, \Rightarrow, \Leftrightarrow$

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Number of Possible Worlds/Models :

In propositional logic, suppose that you have 100 symbols. How many possible worlds do you have?

– 2^{100}

– One for each row of the truth table.

In first-order logic, how can we even count the number of possible worlds? Number of

possible worlds is counted in terms of:

- The number of objects
- Mapping of Constant Symbols to Objects (No. of predicates and its arity)

Number of possible models are unbounded. Checking entailment via Model Checking is not feasible as the number of combinations can become very large.

Number of Possible worlds in FOPL:

Suppose that we have:

- Five constants.
 - No functions.
 - One predicate, that takes one argument.
- How many possible worlds can we have?
 - For each possible argument of the predicate, we must specify if the predicate returns true or false.
 - We have five possible arguments.
 - In total, 2^5 possible worlds.

Number of Possible worlds in FOPL:

Suppose that we have:

- Let the Five constants be V, W, X, Y, Z
- No functions.
- Let the predicate be P, that takes one argument(either of V, W, X, Y, Z) .

There will be 2^5 possible worlds/models (each row of the truth table is a model).

Model	P(V)	P(W)	P(X)	P(Y)	P(Z)
1	false	false	false	false	false
2	true	false	false	false	false
...	...	...	...	...	...
...	...	...	...	...	...
$2^5=32$	true	true	true	true	true

Number of Possible worlds in FOPL:

Suppose that we have:

- Five constants.
 - No functions.
 - One predicate, that takes two arguments.
- How many possible worlds can we have?
 - For each possible combination of arguments of the predicate, we must specify if the predicate returns true or false.
 - We have 25 possible combinations of arguments.
 - In total, 2^{25} possible worlds.

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Number of Possible worlds in FOPL:

Suppose that we have:

- Let the Five constants be V, W, X, Y, Z
- No functions.
- Let the predicate be P, that takes two arguments(either of V, W, X, Y, Z).

• We have 2^{25} possible worlds/models

Model	P(V,V)	P(V,W)	...	...	P(Z,Z)
1	false	false	false	false	false
2	true	false	false	false	false
...	...	...	...	...	...
...	...	...	...	...	...
2^{25}	true	true	true	true	true

Number of Possible worlds in FOPL:

Suppose that we have:

- Five constants.
 - No functions.
 - Three predicates, that take two arguments.
 - One predicate that takes one argument.
-
- How many possible worlds can we have?
 - For each possible combination of arguments of each predicate, we must specify if the predicate returns true or false.
 - For two arguments, we have 25 combinations of arguments. We have three such predicates, so we must specify 75 values.
 - For one argument, we have 5 combinations of arguments. We have one such predicate, so we must specify 5 values.
 - In total, $2^{5+75} = 2^{80}$ possible worlds.

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Example:

For the wumpus world, to say that "pits cause breezes in adjacent squares" using **propositional logic**, we need 16 rules like:

$$B_{12} \rightarrow (P_{11} \vee P_{22} \vee P_{13})$$

In **first-order logic**, how can we say the same thing?

Objects : **Pits, Squares** Relations : **Breeze, adjacent**

$$\forall x_1, y_1 \text{ Breeze}(x_1, y_1) \leftrightarrow (\exists x_2, y_2 \text{ Pit}(x_2, y_2) \text{ AND Adjacent}(x_1, y_1, x_2, y_2))$$



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724



Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 4

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Knowledge Representation using FOPL

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Knowledge Representation using FOPL :

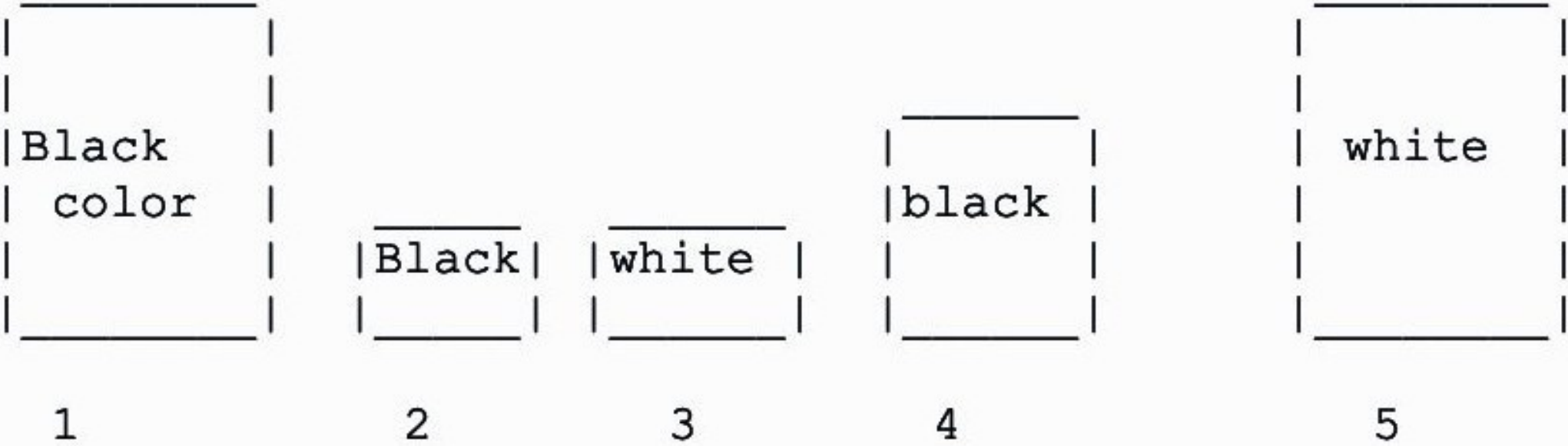
We will look at examples from the following Domain:

- A puzzle - Box Solver
- Kinship (Family Relationships)
- Mathematical Sets

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Example 1 : Let us look at an example called Box Solver. There are 5 boxes



Ann, Bill, Charlie, Don, Eric own one box each but we don't know which box.

Try to find each boxes' owner if you know that:

Ann and Bill have boxes with the same color.

Don and Eric have boxes with the same color.

Charlie and Don have boxes with the same size.

Eric's box is smaller than Bill's.

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

In the puzzle - Box Solver we have:

- Objects : Box
- A Unary Predicate : box
- A Ternary Predicate : box_details
- A Function : Owners

Ann, Bill, Charlie, Don, Eric own one box each but we don't know which box.

Try to find each boxes' owner if you know that:

Ann and Bill have boxes with the same color.

Don and Eric have boxes with the same color.

Charlie and Don have boxes with the same size.

Eric's box is smaller than Bill's.

In the puzzle - FOPL representation:

$\exists A, B, C, D, E, c1, c2, c3, s1, s2, s3$

$(A \neq B \neq C \neq D \neq E) \wedge$

$\text{box_details}(A, c1, s1) \wedge \text{box_details}(B, c1, s2)$

$\wedge \text{box_details}(D, c2, s3) \wedge$

$\text{box_details}(E, c2, s4) \wedge$

$\text{box_details}(C, c3, s3) \wedge \text{box_details}(D, c2, s3)$

$\wedge \text{box_details}(E, c2, s4) \wedge$

$\text{box_details}(B, c1, s2) \wedge s4 < s2$

% Ann and Bill's box have same color

% Don and Eric's box have same color

% Charlie and Don's box have same size

% Eric's box is smaller than Bill's

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

In order to solve the problem, you can:

- 1. Write a Prolog database with the characteristics of the boxes, associating a number to each box, for instance:
box_details(2, black, 1) represents that the second box is of color black and size 1;**
- 2. Write a predicate to compute the solution of the problem: owner(A, B, C, D, E) must be true if A is the box owned by Ann, B the one owned by Bill, and so on. . .**

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

% First define numbers for each box.

box(1).

box(2).

box(3).

box(4).

box(5).

% Box number, color, size.

box_details(1,black,3).

box_details(2,black,1).

box_details(3,white,1).

box_details(4,black,2).

box_details(5,white,3).

%Unary Predicate

%Ternary Predicate

owners(A,B,C,D,E):-

box(A), box(B), box(C), box(D), box(E), A\=B,A\

=C,A\=D,A\=E, B\=C,B\=D,B\=E,

C\=D,C\=E, D\=E,

box_details(A,ColorA,_), box_details(B,ColorA,_),

box_details(D,ColorD,_), box_details(E,ColorD,_),

box_details(C,_,SizeC), box_details(D,_,SizeC),

box_details(E,_,SizeE), box_details(B,_,SizeB),

SizeE < SizeB.

%Function implementation as a rule

%Distinct values

% Ann and Bill have same color

% Don and Eric have same color

% Charlie and Don have same size

% Eric's box is smaller than Bill's

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Example 2 : Kinship Domain

It consists of

- Object – People
- Unary Predicates - Male and Female
- Binary Predicates - Parent, Brother, Sister
- Functions – Father, Mother
- Relations – Brotherhood, sisterhood.

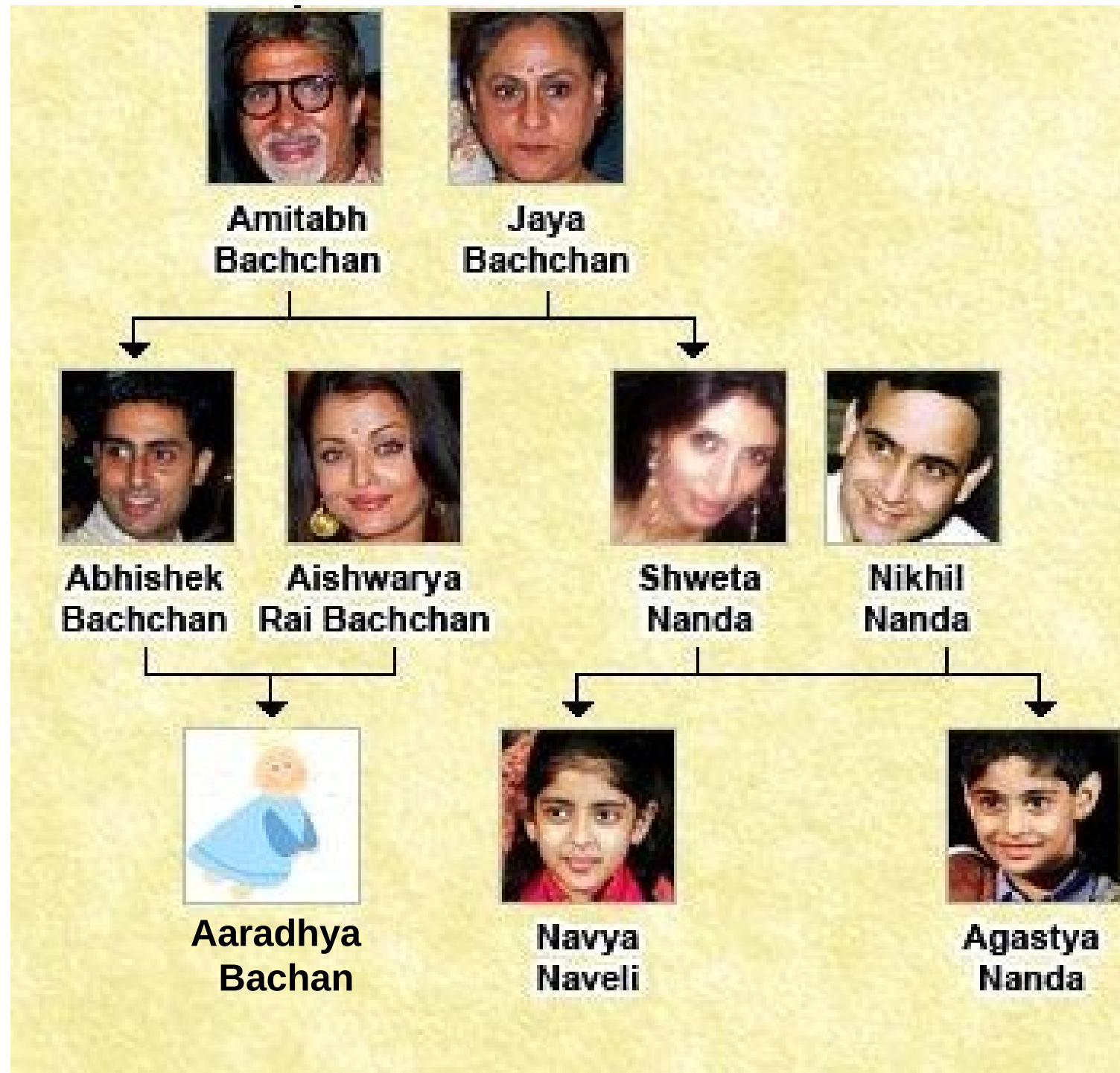
Examples

The kinship domain:

- Brothers are siblings
 $\forall x, y \text{ Brother}(x, y) \Rightarrow \text{Sibling}(x, y)$
- Male and female are disjoint categories
 $\forall x, \text{Male}(x) \Leftrightarrow \neg \text{Female}(x)$
- Parent and child are inverse relations
 $\forall p, c \text{ Parent}(p, c) \Leftrightarrow \text{Child}(c, p)$

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic



Example 2 :

Kinship

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

/* Kinship Program */

/* facts */ male(amitabh). male(abhishek). male(agastya).
male(nikhil). female(jaya). female(aishwarya).
female(aaradhya). female(shweta). female(navya).
parent(amitabh, abhishek). parent(jaya, abhishek).
parent(amitabh, shweta). parent(abhishek, aaradhya).
parent(aishwarya, aaradhya). parent(shweta, navya).
parent(shweta, agastya). parent(nikhil, navya).

/* rules */

mother(M,X):- parent(M,X),female(M).

father(F,X):- parent(F,X), male(F).

grandparent(G, X):- parent(G,P), parent(P,X).

sister(S,X):- parent(Z,X), parent(Z,S), female(S), S\=X.

brother(B,X):- parent(P, B), parent(P, X),male(B), not(B=X).

aunt(X,Y) :- female(X), sister(X,Z), parent(Z,Y).

Example 4 : Mathematical set representation

- Constant – Empty set ($s = \{\}$)
- Predicate – Member and subset ($s_1 \subseteq s_2$)
- Functions – Intersection ($\cap$) and union ($\cup$)
- Example: Two sets are equal if and only if each is a subset of the other.

$$\forall s_1, s_2 (s_1 = s_2) \Leftrightarrow (\text{subset}(s_1, s_2) \wedge \text{subset}(s_2, s_1))$$

Other eg:

$$\forall x, s_1, s_2 \ x \in (s_1 \cap s_2) \Leftrightarrow (x \in s_1 \wedge x \in s_2)$$

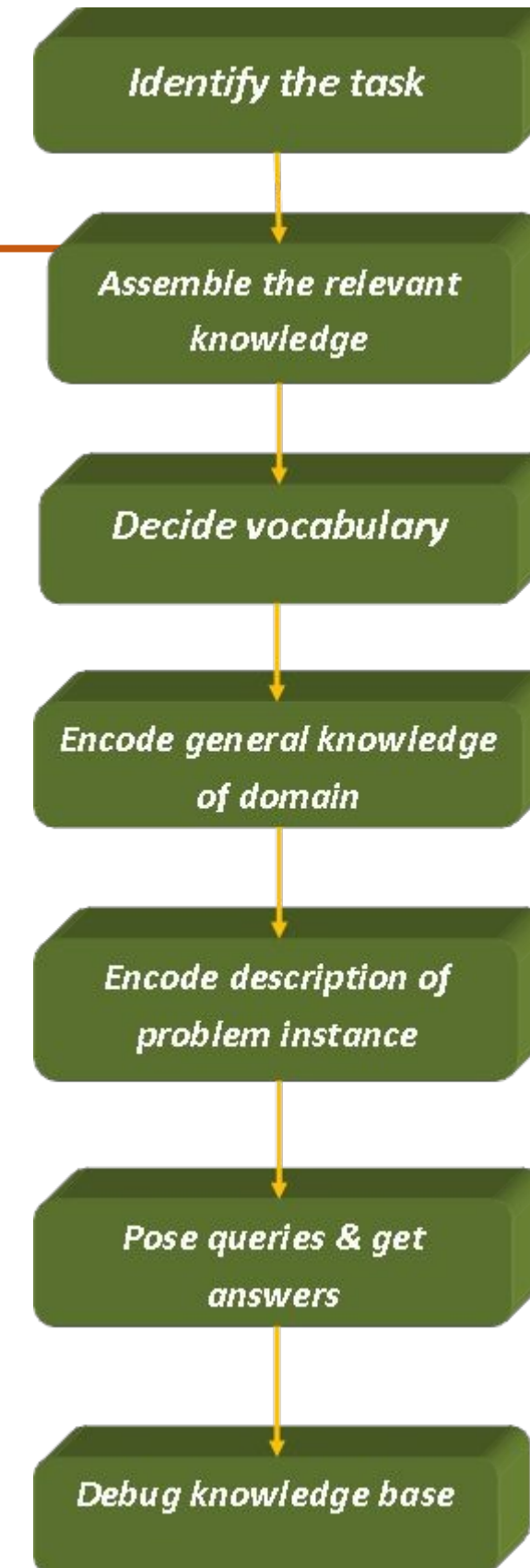
$$\forall x, s_1, s_2 \ x \in (s_1 \cup s_2) \Leftrightarrow (x \in s_1 \vee x \in s_2)$$

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Knowledge Engineering in FOPL:

The process of constructing a knowledge-base in first-order logic is called as knowledge-engineering.



Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Practice Problems

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Which of the following are correct?

- a. $(A \wedge B) \models (A \Leftrightarrow B)$.
- b. $A \Leftrightarrow B \models A \vee B$.
- c. $(A \vee B) \wedge \neg(A \Rightarrow B)$ is satisfiable.
- d. $(A \Leftrightarrow B) \wedge (\neg A \vee B)$ is satisfiable.
- e. $\alpha \models \beta$ if and only if the sentence $(\alpha \Rightarrow \beta)$ is valid
- f. If $\alpha \models \gamma$ or $\beta \models \gamma$ (or both) then $(\alpha \wedge \beta) \models \gamma$

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

**Convert Natural Language
Sentences to FOPL**

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

From Natural Language to First order logic (or vv.).

Consider the following three sentences:

- “Each animal is an organism”
- “All animals are organisms”
- “If it is an animal then it is an organism”

All the above statements can be formalised as:

$$\forall x (Animal(x) \rightarrow Organism(x))$$

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Convert the given Natural Language Sentences to Predicate Logic :

- **“Not all birds can fly.”**
 $\neg(\forall x \text{ Bird}(x) \Rightarrow \text{Fly}(x))$, which is the same as
 $\exists x \text{ Bird}(x) \wedge \neg \text{Fly}(x)$
- **“All birds cannot fly.”**
 $\forall x \text{ Bird}(x) \Rightarrow \neg \text{Fly}(x)$, which is the same as
 $\neg(\exists x \text{ Bird}(x) \wedge \text{Fly}(x))$
- **“If anyone can solve the problem, then Hilary can.”** ($\exists x$
 $\text{Solves}(x, \text{problem}) \Rightarrow \text{Solves}(\text{Hilary}, \text{problem})$)
- **“John hates all people who do not hate themselves.”**
 $\forall x \text{ Person}(x) \wedge \neg \text{Hates}(x, x) \Rightarrow \text{Hates}(\text{John}, x)$

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Convert the given Natural Language Sentences to Predicate Logic :

- Marcus was a man $\text{Man}(\text{Marcus})$
- Marcus was a Pompeian $\text{Pompeian}(\text{Marcus})$
- All Pompeians were Romans
 $\forall x [\text{Pompeian}(x) \rightarrow \text{Roman}(x)]$
- Caesar was a ruler $\text{Ruler}(\text{Caesar})$
- All Romans were either loyal to Caesar or hated him
 $\forall x [\text{Roman}(x) \rightarrow (\text{LoyalTo}(x, \text{Caesar}) \vee \text{Hate}(x, \text{Caesar}))]$
- Everyone is loyal to someone
 $\forall x \exists y \text{LoyalTo}(x, y)$

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Convert the given Natural Language Sentences to Predicate Logic :

- Whoever can read is literate

$$\forall x [R(x) \rightarrow L(x)]$$

- Dolphins are not literate

$$\forall x [D(x) \rightarrow \neg L(x)]$$

- Some Dolphins are intelligent

$$\exists x [D(x) \wedge I(x)]$$

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Problem 1

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Using the following notation, express the given statements in FOPL :

$Q(x)$: x is a rational number

$R(x)$: x is a real number $LESS(x, y)$: x is less than y

Statements:

- i) There exists a number that is rational.
- ii) Every rational number is a real number
- iii) For every number x , there exists a number y , which is greater than x .
- iv) Some real numbers are rational numbers.

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Using the following notation, express the given statements in FOPL : $Q(x)$: x is a rational number

$R(x)$: x is a real number $LESS(x, y)$: x is less than y Statements:

- i) There exists a number that is rational.
- ii) Every rational number is a real number
- iii) For every number x , there exists a number y , which is greater than x .
- iv) Some real numbers are rational numbers. **Solution:**

(i) $(\exists x) Q(x)$

(ii) $(\forall x) (Q(x) \rightarrow R(x))$

(iii) $(\forall x) (\exists y) LESS(x, y)$

(iv) $(\exists x) (Q(x) \wedge R(x))$

Each of the expressions (i), (ii), and (iii) is called a formula or a well-formed formula or wff.

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Problem 2

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Problem Description:

- The domain of interest is the natural numbers, $\mathbb{N}$.
- There are objects, 0, 1, 2, 3,
- There are functions, addition and multiplication, as well as the square function, on this domain.
- There are predicates on this domain, “even,” “odd,” and “prime.”
- There are relations between elements of this domain, “equal,” “less than”, and “divides.”
- For our logical language, we will choose symbols 1, 2, 3, add, mul, square, even, odd, prime, div, and so on, to denote these things.

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Consider some ordinary statements about the natural numbers, express the same in FOPL:

1. Every natural number is even or odd, but not both.
2. A natural number is even if and only if it is divisible by two.
3. If some natural number, x , is even, then so is x^2 .
4. A natural number x is even if and only if $x+1$ is odd.
5. Any prime number that is greater than 2 is odd.
6. For any three natural numbers x , y , and z , if x divides y and y divides z , then x divides z .
7. There exists an odd composite number.

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Solution

1. $\forall x((\text{even}(x) \vee \text{odd}(x)) \wedge \neg(\text{even}(x) \wedge \text{odd}(x)))$
2. $\forall x(\text{even}(x) \leftrightarrow \text{div}(x,2))$
3. $\forall x(\text{even}(x) \rightarrow \text{even}(x^2))$
4. $\forall x(\text{even}(x) \leftrightarrow \text{odd}(x+1))$
5. $\forall x(\text{prime}(x) \wedge x > 2 \rightarrow \text{odd}(x))$
6. $\forall x \forall y \forall z(\text{div}(x,y) \wedge \text{div}(y,z) \rightarrow \text{div}(x,z))$
7. $\exists x(\text{odd}(x) \wedge \text{composite}(x))$

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Problem 3

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Using a language with variables ranging over people, and predicates $\text{trusts}(x,y)$, $\text{politician}(x)$, $\text{crazy}(x)$, $\text{knows}(x,y)$, $\text{related-to}(x,y)$, and $\text{rich}(x)$, write down first-order sentences asserting the following:

1. Nobody trusts a politician.
2. Anyone who trusts a politician is crazy.
3. Everyone knows someone who is related to a politician.
4. Everyone who is rich is either a politician or knows a politician.

Using a language with variables ranging over people, and predicates trusts(x,y), politician(x), crazy(x), knows(x,y), related-to(x,y), and rich(x), write down first-order sentences asserting the following:

Solution

1	Nobody trusts a politician.	$\forall x \forall y \text{ politician}(x) \rightarrow \neg \text{trusts}(y,x)$
2	Anyone who trusts a politician is crazy.	$\forall x \forall y (\text{trust}(x,y) \wedge \text{politician}(y)) \rightarrow \text{crazy}(x)$
3	Everyone knows someone who is related to a politician.	$\forall x \exists y \exists z (\text{knows}(x,y) \wedge \text{related}(y,z) \wedge \text{politician}(z))$
4	Everyone who is rich is either a politician or knows a politician.	$\forall x \exists y \text{ rich}(x) \rightarrow (\text{politician}(x) \vee (\text{knows}(x,y) \wedge \text{politician}(y)))$

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Problem 4

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Let $C(x)$ mean “ x is a used-car dealer,” and $H(x)$ mean “ x is honest.” Translate each of the following formulas into English:

i) $(\exists x)C(x)$

ii) $(\exists x) H(x)$

iii) $(\forall x)C(x) \rightarrow \sim H(x)$

(iv) $(\exists x) (C(x) \wedge H(x))$

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Let $C(x)$ mean “ x is a used-car dealer,” and $H(x)$ mean “ x is honest.” Translate each of the following formulas into English:

- i) There is (at least) one (person) who is a used-car dealer.
- ii) There is (at least) one (person) who is honest.
- iii) All used-car dealers are dishonest.
- iv) (At least) one used-car dealer is honest. OR-----

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic - Extra

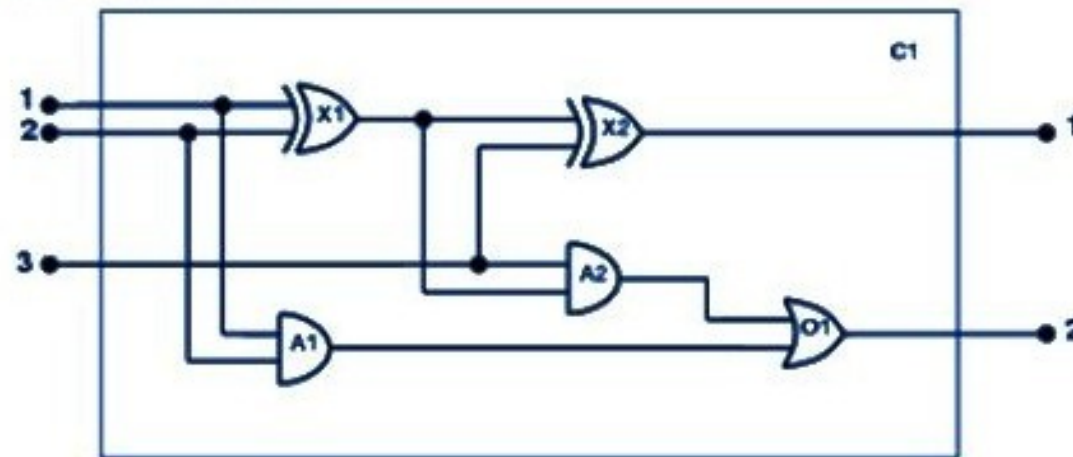
Electronic circuit domain

<https://www.javatpoint.com/ai-knowledge-engineering-in-first-order-logic>

<https://slideplayer.com/slide/260102/>

The electronic circuits domain

One-bit full adder



Possible queries:

- does the circuit function properly?
 - what gates are connected to the first input terminal?
 - what would happen if one of the gates is broken?
- and so on

The electronic circuits domain

1. Identify the task

- Does the circuit actually add properly?

2. Assemble the relevant knowledge

- Composed of wires and gates; Types of gates (AND, OR, XOR, NOT)
- Two input terminals and one output terminal

3. Decide on a vocabulary of Constant, Predicate and Function Symbols

- Alternatives:

Type(X_1) = XOR (function)

Type(X_1 , XOR) (binary predicate)

XOR(X_1) (unary predicate)

It can be represented by either binary predicate or individual type.

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

We use the following Vocabulary to describe an electronic circuit :

Objects : Gates,
Signal Values -1,0 ,
Terminals,
wires, circuit

Constant Symbols : C_1 (maps to object Circuit)
 X_1 (maps to object XOR gate 1),
 X_2 (maps to object XOR Gate 2),
 A_1 (maps to object AND gate 1),
 A_2 (maps to object AND gate 2),
 O_1 (maps to object OR gate 1),
1(maps to Terminal 1),
2(maps to Terminal 2), 3(maps
to Terminal 3)

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

We use the following Vocabulary to describe an electronic circuit :

Predicate (s)	: Gate(X)	%returns true if X is a gate
	: Circuit(X)	%returns true if X is a circuit
	: Terminal(X)	%returns true if X is a Terminal
	: Arity(X, i, j)	%returns true if X(gate or circuit) has i input terminals and j output terminals
	: Connected(t1, t2)	%returns true if terminal t1 is connected to terminal t2
Functions :	Type(X)	%returns the type of gate that X (a constant symbol) maps to.
	: In(n, Gate)	%returns nth Input Terminal of a Gate
	: Out(n, Gate)	%returns nth Output Terminal of a Gate
	: Signal(t)	%returns value of signal at Terminal t which could be either 0 or 1.

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

4. Encode general knowledge of the domain

1. If two terminals are connected, then they have the same signal.

$$\forall t_1, t_2 \text{ Connected}(t_1, t_2) \Rightarrow \text{Signal}(t_1) = \text{Signal}(t_2)$$

2. The signal at every terminal is either 1 or 0
(but not both)

$$\forall t \text{ Signal}(t) = 1 \vee \text{Signal}(t) = 0$$

$$1 \neq 0$$

3. Connected is a commutative predicate.

$$\forall t_1, t_2 \text{ Connected}(t_1, t_2) \Rightarrow \text{Connected}(t_2, t_1)$$

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

4. An OR gate's output is 1 if and only if any of its input is 1.

$$\forall g \text{ Type}(g) = \text{OR} \Rightarrow \\ \text{Signal}(\text{Out}(1,g)) = 1 \Leftrightarrow \exists n \text{ Signal}(\text{In}(n,g)) = 1$$

5. An AND gate's output is 0 if and only if any of its input is 0.

$$\forall g \text{ Type}(g) = \text{AND} \Rightarrow \\ \text{Signal}(\text{Out}(1,g)) = 0 \Leftrightarrow \exists n \text{ Signal}(\text{In}(n,g)) = 0$$

6. An XOR gate's output is 1 if and only if any of its inputs are different:

$$\forall g \text{ Type}(g) = \text{XOR} \Rightarrow \text{Signal}(\text{Out}(1,g)) = 1 \Leftrightarrow \\ \text{Signal}(\text{In}(1,g)) \neq \text{Signal}(\text{In}(2,g))$$

5. Encode the specific problem instance

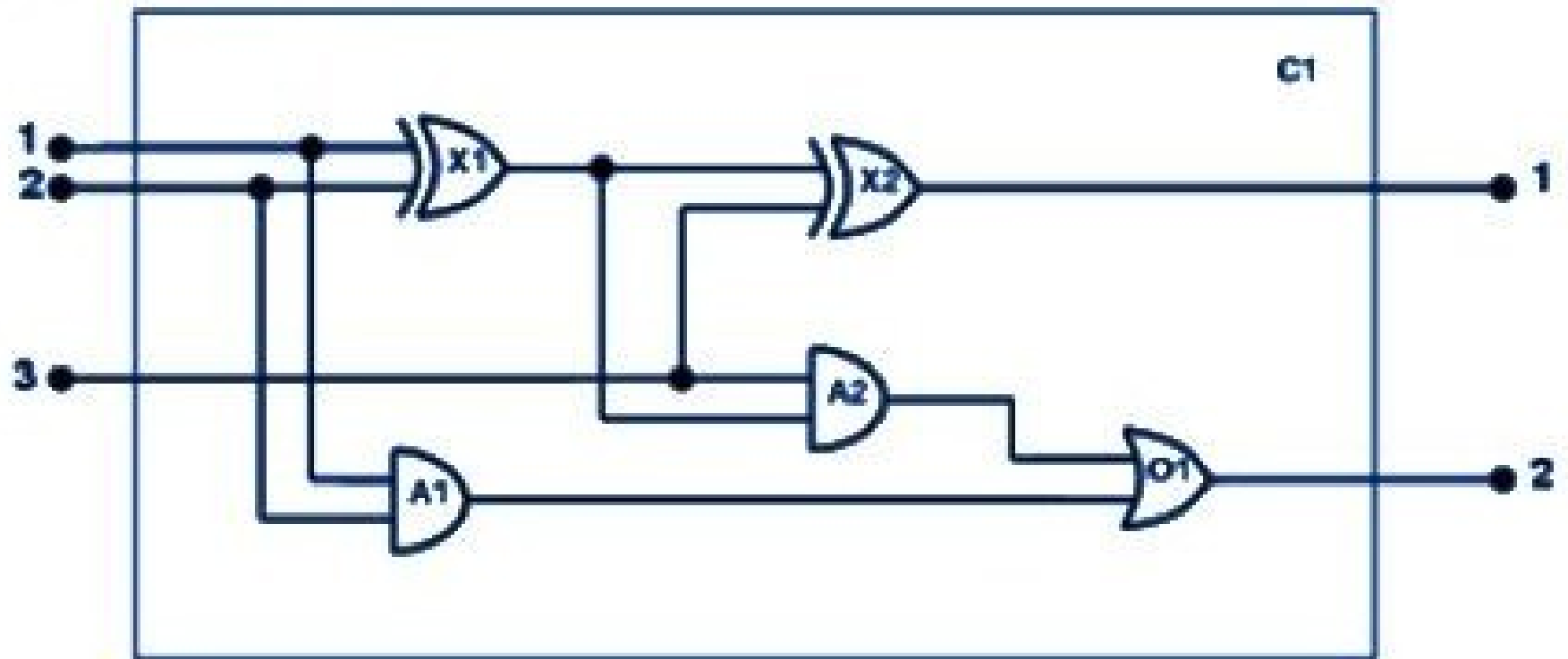
- First we categorize the gates:

Type(X_1) = XOR
Type(A_1) = AND
Type(O_1) = OR

Type(X_2) = XOR
Type(A_2) = AND

- We specify the Arity of the Circuit and each gate :

Arity($C1$, 3, 2)
Arity($X1$, 2, 1)
Arity($X2$, 2, 1)
Arity($A1$, 2, 1)
Arity($A2$, 2, 1)
Arity($O1$, 2, 1)

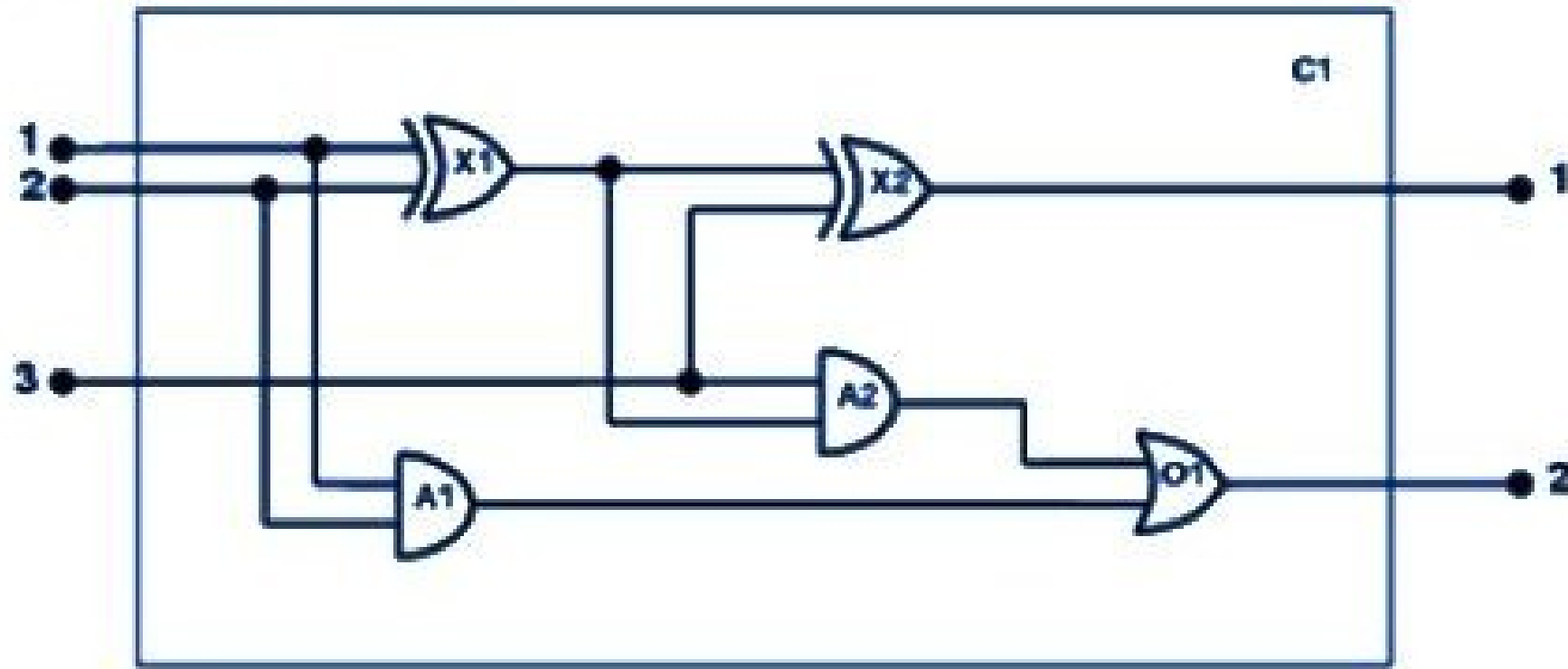


Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

5. Encode the specific problem instance

- Then show the connections between them:



Connected(In(1,C1), In(1, X1))
Connected(In(2,C1), In(2, X1))

Connected(In(1,C1), In(1, A1))
Connected(In(2,C1), In(2, A1))

Connected(Out(1,X1), In(1, X2))
Connected(In(3,C1), In(2, X2))

(
Connected(In(3,C1), In(1, A2))
Connected(Out(1,X1), In(2, A2))

(
Connected(Out(1,A2), In(1, O1))
Connected(Out(1,A1), In(2, O1))

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

7. Query and get answers

Query represented in FOPL:

$$\exists i1, i2, i3, o1, o2 \text{ signal}(\text{in}(1, c1)) = i1 \wedge \text{signal}(\text{in}(2, c1)) = i2 \wedge \text{signal}(\text{in}(3, c1)) = i3 \wedge \text{signal}(\text{out}(1, c1)) = o1 \wedge \text{signal}(\text{out}(2, c1)) = o2$$

In Prolog we can write a rule `find(i1, i2, i3, o1, o2)` to do the same:

`-? find(i1, i2, i3, o1, o2)`

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

7. Query and get answers

We can examine output of each gate:

$\exists$ i1, i2, o

signal(in(1,x1)) = i1 $\wedge$

signal(in(2,x1)) = i2 $\wedge$

signal(out(1,x1)) = o



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724



Automata Formal Languages & Logic

Preet Kanwal

Department of Computer Science & Engineering

Automata Formal Languages & Logic

Unit 4

Preet Kanwal

Department of Computer Science & Engineering

Knowledge Representation using FOPL

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Knowledge Representation using FOPL :

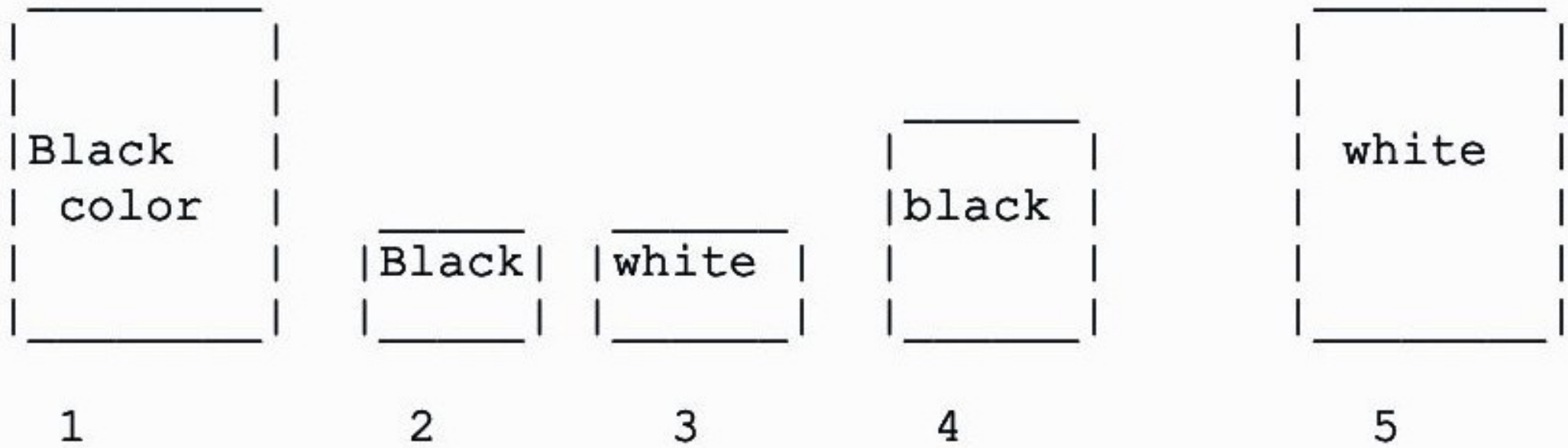
We will look at examples from the following Domain:

- A puzzle - Box Solver
- Kinship (Family Relationships)
- Mathematical Sets

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Example 1 : Let us look at an example called Box Solver. There are 5 boxes



Ann, Bill, Charlie, Don, Eric own one box each but we don't know which box.

Try to find each boxes' owner if you know that:

Ann and Bill have boxes with the same color.

Don and Eric have boxes with the same color.

Charlie and Don have boxes with the same size.

Eric's box is smaller than Bill's.

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

In the puzzle - Box Solver we have:

- Objects : Box
- A Unary Predicate : box
- A Ternary Predicate : box_details
- A Function : Owners

Ann, Bill, Charlie, Don, Eric own one box each but we don't know which box.

Try to find each boxes' owner if you know that:

Ann and Bill have boxes with the same color.

Don and Eric have boxes with the same color.

Charlie and Don have boxes with the same size.

Eric's box is smaller than Bill's.

In the puzzle - FOPL representation:

$\exists A, B, C, D, E, c1, c2, c3, s1, s2, s3$

$(A \neq B \neq C \neq D \neq E) \wedge$

$\text{box_details}(A, c1, s1) \wedge \text{box_details}(B, c1, s2)$

$\wedge \text{box_details}(D, c2, s3) \wedge$

$\text{box_details}(E, c2, s4) \wedge$

$\text{box_details}(C, c3, s3) \wedge \text{box_details}(D, c2, s3)$

$\wedge \text{box_details}(E, c2, s4) \wedge$

$\text{box_details}(B, c1, s2) \wedge s4 < s2$

% Ann and Bill's box have same color

% Don and Eric's box have same color

% Charlie and Don's box have same size

% Eric's box is smaller than Bill's

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

In order to solve the problem, you can:

- 1. Write a Prolog database with the characteristics of the boxes, associating a number to each box, for instance:
box_details(2, black, 1) represents that the second box is of color black and size 1;**
- 2. Write a predicate to compute the solution of the problem: owner(A, B, C, D, E) must be true if A is the box owned by Ann, B the one owned by Bill, and so on. . .**

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

% First define numbers for each box.

box(1).

box(2).

box(3).

box(4).

box(5).

% Box number, color, size.

box_details(1,black,3).

box_details(2,black,1).

box_details(3,white,1).

box_details(4,black,2).

box_details(5,white,3).

%Unary Predicate

%Ternary Predicate

owners(A,B,C,D,E):-

box(A), box(B), box(C), box(D), box(E), A\=B,A\

=C,A\=D,A\=E, B\=C,B\=D,B\=E,

C\=D,C\=E, D\=E,

box_details(A,ColorA,_), box_details(B,ColorA,_),

box_details(D,ColorD,_), box_details(E,ColorD,_),

box_details(C,_,SizeC), box_details(D,_,SizeC),

box_details(E,_,SizeE), box_details(B,_,SizeB),

SizeE < SizeB.

%Function implementation as a rule

%Distinct values

% Ann and Bill have same color

% Don and Eric have same color

% Charlie and Don have same size

% Eric's box is smaller than Bill's

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Example 2 : Kinship Domain

It consists of

- Object – People
- Unary Predicates - Male and Female
- Binary Predicates - Parent, Brother, Sister
- Functions – Father, Mother
- Relations – Brotherhood, sisterhood.

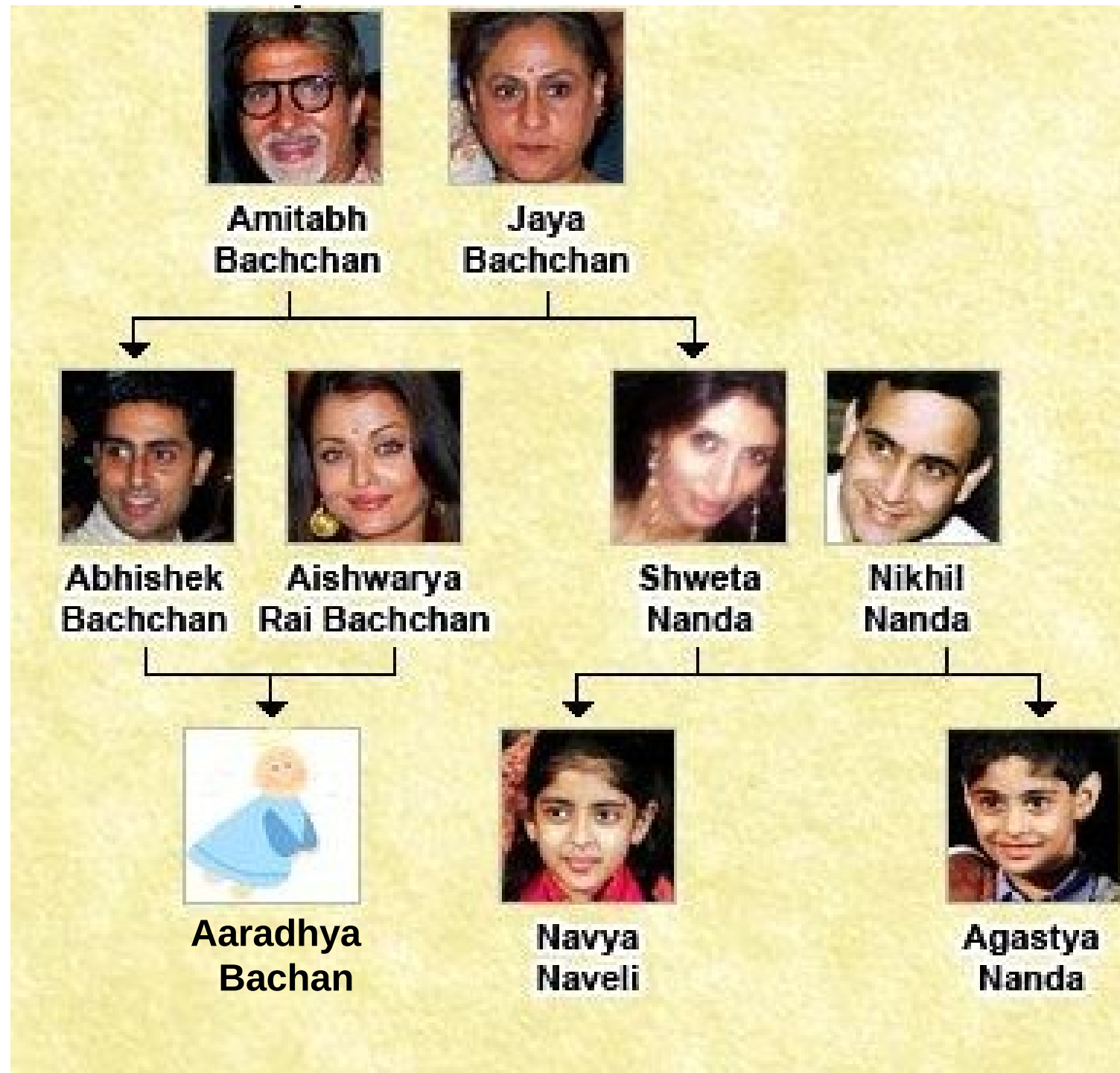
Examples

The kinship domain:

- Brothers are siblings
 $\forall x, y \text{ Brother}(x, y) \Rightarrow \text{Sibling}(x, y)$
- Male and female are disjoint categories
 $\forall x, \text{Male}(x) \Leftrightarrow \neg \text{Female}(x)$
- Parent and child are inverse relations
 $\forall p, c \text{ Parent}(p, c) \Leftrightarrow \text{Child}(c, p)$

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic



Example 2 :

Kinship

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

/* Kinship Program */

/* facts */ male(amitabh). male(abhishek). male(agastya).
male(nikhil). female(jaya). female(aishwarya).
female(aaradhya). female(shweta). female(navya).
parent(amitabh, abhishek). parent(jaya, abhishek).
parent(amitabh, shweta). parent(abhishek, aaradhya).
parent(aishwarya, aaradhya). parent(shweta, navya).
parent(shweta, agastya). parent(nikhil, navya).

/* rules */

mother(M,X):- parent(M,X),female(M).

father(F,X):- parent(F,X), male(F).

grandparent(G, X):- parent(G,P), parent(P,X).

sister(S,X):- parent(Z,X), parent(Z,S), female(S), S\=X.

brother(B,X):- parent(P, B), parent(P, X),male(B), not(B=X).

aunt(X,Y) :- female(X), sister(X,Z), parent(Z,Y).

Example 4 : Mathematical set representation

- Constant – Empty set ($s = \{\}$)
- Predicate – Member and subset ($s_1 \subseteq s_2$)
- Functions – Intersection ($\cap$) and union ($\cup$)
- Example: Two sets are equal if and only if each is a subset of the other.

$$\forall s_1, s_2 (s_1 = s_2) \Leftrightarrow (\text{subset}(s_1, s_2) \wedge \text{subset}(s_2, s_1))$$

Other eg:

$$\forall x, s_1, s_2 \ x \in (s_1 \cap s_2) \Leftrightarrow (x \in s_1 \wedge x \in s_2)$$

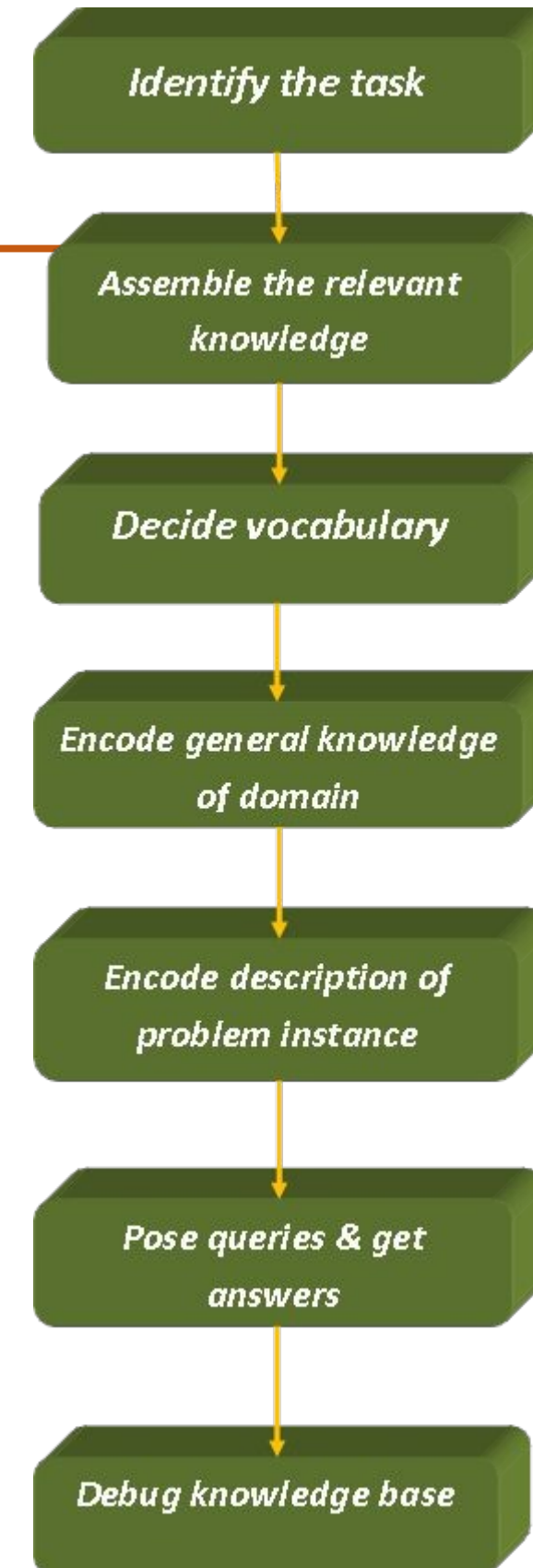
$$\forall x, s_1, s_2 \ x \in (s_1 \cup s_2) \Leftrightarrow (x \in s_1 \vee x \in s_2)$$

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Knowledge Engineering in FOPL:

The process of constructing a knowledge-base in first-order logic is called as knowledge-engineering.



Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Practice Problems

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Which of the following are correct?

- a. $(A \wedge B) \models (A \Leftrightarrow B)$.
- b. $A \Leftrightarrow B \models A \vee B$.
- c. $(A \vee B) \wedge \neg(A \Rightarrow B)$ is satisfiable.
- d. $(A \Leftrightarrow B) \wedge (\neg A \vee B)$ is satisfiable.
- e. $\alpha \models \beta$ if and only if the sentence $(\alpha \Rightarrow \beta)$ is valid
- f. If $\alpha \models \gamma$ or $\beta \models \gamma$ (or both) then $(\alpha \wedge \beta) \models \gamma$

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

**Convert Natural Language
Sentences to FOPL**

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

From Natural Language to First order logic (or vv.).

Consider the following three sentences:

- “Each animal is an organism”
- “All animals are organisms”
- “If it is an animal then it is an organism”

All the above statements can be formalised as:

$$\forall x (Animal(x) \rightarrow Organism(x))$$

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Convert the given Natural Language Sentences to Predicate Logic :

- **“Not all birds can fly.”**
 $\neg(\forall x \text{ Bird}(x) \Rightarrow \text{Fly}(x))$, which is the same as
 $\exists x \text{ Bird}(x) \wedge \neg \text{Fly}(x)$
- **“All birds cannot fly.”**
 $\forall x \text{ Bird}(x) \Rightarrow \neg \text{Fly}(x)$, which is the same as
 $\neg(\exists x \text{ Bird}(x) \wedge \text{Fly}(x))$
- **“If anyone can solve the problem, then Hilary can.”** ($\exists x$
 $\text{Solves}(x, \text{problem}) \Rightarrow \text{Solves}(\text{Hilary}, \text{problem})$)
- **“John hates all people who do not hate themselves.”**
 $\forall x \text{ Person}(x) \wedge \neg \text{Hates}(x, x) \Rightarrow \text{Hates}(\text{John}, x)$

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Convert the given Natural Language Sentences to Predicate Logic :

- Marcus was a man $\text{Man}(\text{Marcus})$
- Marcus was a Pompeian $\text{Pompeian}(\text{Marcus})$
- All Pompeians were Romans
 $\forall x [\text{Pompeian}(x) \rightarrow \text{Roman}(x)]$
- Caesar was a ruler $\text{Ruler}(\text{Caesar})$
- All Romans were either loyal to Caesar or hated him
 $\forall x [\text{Roman}(x) \rightarrow (\text{LoyalTo}(x, \text{Caesar}) \vee \text{Hate}(x, \text{Caesar}))]$
- Everyone is loyal to someone
 $\forall x \exists y \text{LoyalTo}(x, y)$

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Convert the given Natural Language Sentences to Predicate Logic :

- Whoever can read is literate

$$\forall x [R(x) \rightarrow L(x)]$$

- Dolphins are not literate

$$\forall x [D(x) \rightarrow \neg L(x)]$$

- Some Dolphins are intelligent

$$\exists x [D(x) \wedge I(x)]$$

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Problem 1

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Using the following notation, express the given statements in FOPL :

$Q(x)$: x is a rational number

$R(x)$: x is a real number $LESS(x, y)$: x is less than y

Statements:

- i) There exists a number that is rational.
- ii) Every rational number is a real number
- iii) For every number x , there exists a number y , which is greater than x .
- iv) Some real numbers are rational numbers.

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Using the following notation, express the given statements in FOPL : $Q(x)$: x is a rational number

$R(x)$: x is a real number $LESS(x, y)$: x is less than y Statements:

- i) There exists a number that is rational.
- ii) Every rational number is a real number
- iii) For every number x, there exists a number y, which is greater than x.
- iv) Some real numbers are rational numbers. **Solution:**

(i) $(\exists x) Q(x)$

(ii) $(\forall x) (Q(x) \rightarrow R(x))$

(iii) $(\forall x) (\exists y) LESS(x, y)$

(iv) $(\exists x) (Q(x) \wedge R(x))$

Each of the expressions (i), (ii), and (iii) is called a formula or a well-formed formula or wff.

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Problem 2

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Problem Description:

- The domain of interest is the natural numbers, $\mathbb{N}$.
- There are objects, 0, 1, 2, 3,
- There are functions, addition and multiplication, as well as the square function, on this domain.
- There are predicates on this domain, “even,” “odd,” and “prime.”
- There are relations between elements of this domain, “equal,” “less than”, and “divides.”
- For our logical language, we will choose symbols 1, 2, 3, add, mul, square, even, odd, prime, div, and so on, to denote these things.

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Consider some ordinary statements about the natural numbers, express the same in FOPL:

1. Every natural number is even or odd, but not both.
2. A natural number is even if and only if it is divisible by two.
3. If some natural number, x , is even, then so is x^2 .
4. A natural number x is even if and only if $x+1$ is odd.
5. Any prime number that is greater than 2 is odd.
6. For any three natural numbers x , y , and z , if x divides y and y divides z , then x divides z .
7. There exists an odd composite number.

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Solution

1. $\forall x((\text{even}(x) \vee \text{odd}(x)) \wedge \neg(\text{even}(x) \wedge \text{odd}(x)))$
2. $\forall x(\text{even}(x) \leftrightarrow \text{div}(x,2))$
3. $\forall x(\text{even}(x) \rightarrow \text{even}(x^2))$
4. $\forall x(\text{even}(x) \leftrightarrow \text{odd}(x+1))$
5. $\forall x(\text{prime}(x) \wedge x > 2 \rightarrow \text{odd}(x))$
6. $\forall x \forall y \forall z(\text{div}(x,y) \wedge \text{div}(y,z) \rightarrow \text{div}(x,z))$
7. $\exists x(\text{odd}(x) \wedge \text{composite}(x))$

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Problem 3

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Using a language with variables ranging over people, and predicates $\text{trusts}(x,y)$, $\text{politician}(x)$, $\text{crazy}(x)$, $\text{knows}(x,y)$, $\text{related-to}(x,y)$, and $\text{rich}(x)$, write down first-order sentences asserting the following:

1. Nobody trusts a politician.
2. Anyone who trusts a politician is crazy.
3. Everyone knows someone who is related to a politician.
4. Everyone who is rich is either a politician or knows a politician.

Using a language with variables ranging over people, and predicates trusts(x,y), politician(x), crazy(x), knows(x,y), related-to(x,y), and rich(x), write down first-order sentences asserting the following:

Solution

1	Nobody trusts a politician.	$\forall x \forall y \text{ politician}(x) \rightarrow \neg \text{trusts}(y,x)$
2	Anyone who trusts a politician is crazy.	$\forall x \forall y (\text{trust}(x,y) \wedge \text{politician}(y)) \rightarrow \text{crazy}(x)$
3	Everyone knows someone who is related to a politician.	$\forall x \exists y \exists z (\text{knows}(x,y) \wedge \text{related}(y,z) \wedge \text{politician}(z))$
4	Everyone who is rich is either a politician or knows a politician.	$\forall x \exists y \text{ rich}(x) \rightarrow (\text{politician}(x) \vee (\text{knows}(x,y) \wedge \text{politician}(y)))$

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Problem 4

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Let $C(x)$ mean “ x is a used-car dealer,” and $H(x)$ mean “ x is honest.” Translate each of the following formulas into English:

i) $(\exists x)C(x)$

ii) $(\exists x) H(x)$

iii) $(\forall x)C(x) \rightarrow \sim H(x)$

(iv) $(\exists x) (C(x) \wedge H(x))$

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

Let $C(x)$ mean “ x is a used-car dealer,” and $H(x)$ mean “ x is honest.” Translate each of the following formulas into English:

- i) There is (at least) one (person) who is a used-car dealer.
- ii) There is (at least) one (person) who is honest.
- iii) All used-car dealers are dishonest.
- iv) (At least) one used-car dealer is honest. OR-----

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic - Extra

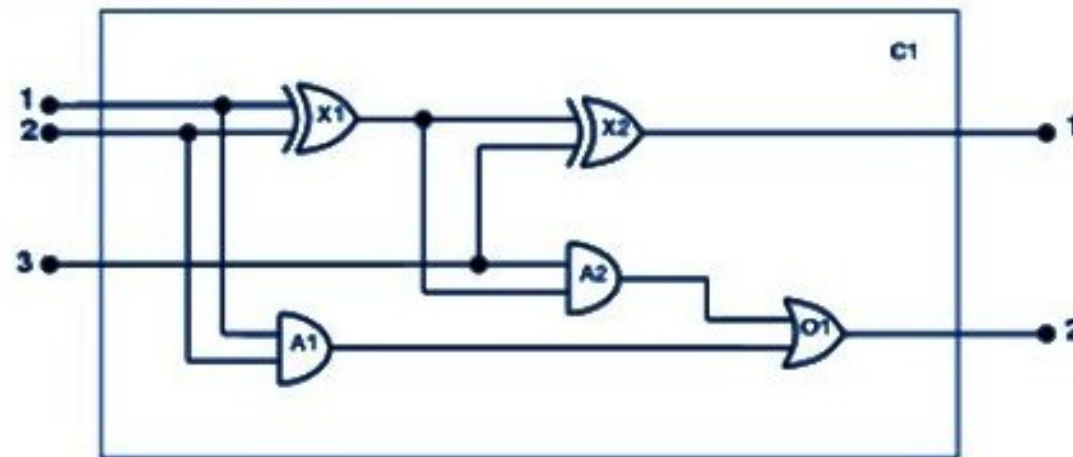
Electronic circuit domain

<https://www.javatpoint.com/ai-knowledge-engineering-in-first-order-logic>

<https://slideplayer.com/slide/260102/>

The electronic circuits domain

One-bit full adder



Possible queries:

- does the circuit function properly?
 - what gates are connected to the first input terminal?
 - what would happen if one of the gates is broken?
- and so on

The electronic circuits domain

1. Identify the task

- Does the circuit actually add properly?

2. Assemble the relevant knowledge

- Composed of wires and gates; Types of gates (AND, OR, XOR, NOT)
- Two input terminals and one output terminal

3. Decide on a vocabulary of Constant, Predicate and Function Symbols

- Alternatives:

Type(X_1) = XOR (function)

Type(X_1 , XOR) (binary predicate)

XOR(X_1) (unary predicate)

It can be represented by either binary predicate or individual type.

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

We use the following Vocabulary to describe an electronic circuit :

Objects : Gates,
Signal Values -1,0 ,
Terminals,
wires, circuit

Constant Symbols : C_1 (maps to object Circuit)
 X_1 (maps to object XOR gate 1),
 X_2 (maps to object XOR Gate 2),
 A_1 (maps to object AND gate 1),
 A_2 (maps to object AND gate 2),
 O_1 (maps to object OR gate 1),
1(maps to Terminal 1),
2(maps to Terminal 2), 3(maps
to Terminal 3)

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

We use the following Vocabulary to describe an electronic circuit :

Predicate (s)	: Gate(X)	%returns true if X is a gate
	: Circuit(X)	%returns true if X is a circuit
	: Terminal(X)	%returns true if X is a Terminal
	: Arity(X, i, j)	%returns true if X(gate or circuit) has i input terminals and j output terminals
	: Connected(t1, t2)	%returns true if terminal t1 is connected to terminal t2
Functions :	Type(X)	%returns the type of gate that X (a constant symbol) maps to.
	: In(n, Gate)	%returns nth Input Terminal of a Gate
	: Out(n, Gate)	%returns nth Output Terminal of a Gate
	: Signal(t)	%returns value of signal at Terminal t which could be either 0 or 1.

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

4. Encode general knowledge of the domain

1. If two terminals are connected, then they have the same signal.

$$\forall t_1, t_2 \text{ Connected}(t_1, t_2) \Rightarrow \text{Signal}(t_1) = \text{Signal}(t_2)$$

2. The signal at every terminal is either 1 or 0
(but not both)

$$\forall t \text{ Signal}(t) = 1 \vee \text{Signal}(t) = 0$$

$$1 \neq 0$$

3. Connected is a commutative predicate.

$$\forall t_1, t_2 \text{ Connected}(t_1, t_2) \Rightarrow \text{Connected}(t_2, t_1)$$

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

4. An OR gate's output is 1 if and only if any of its input is 1.

$$\forall g \text{ Type}(g) = \text{OR} \Rightarrow \\ \text{Signal}(\text{Out}(1,g)) = 1 \Leftrightarrow \exists n \text{ Signal}(\text{In}(n,g)) = 1$$

5. An AND gate's output is 0 if and only if any of its input is 0.

$$\forall g \text{ Type}(g) = \text{AND} \Rightarrow \\ \text{Signal}(\text{Out}(1,g)) = 0 \Leftrightarrow \exists n \text{ Signal}(\text{In}(n,g)) = 0$$

6. An XOR gate's output is 1 if and only if any of its inputs are different:

$$\forall g \text{ Type}(g) = \text{XOR} \Rightarrow \text{Signal}(\text{Out}(1,g)) = 1 \Leftrightarrow \\ \text{Signal}(\text{In}(1,g)) \neq \text{Signal}(\text{In}(2,g))$$

5. Encode the specific problem instance

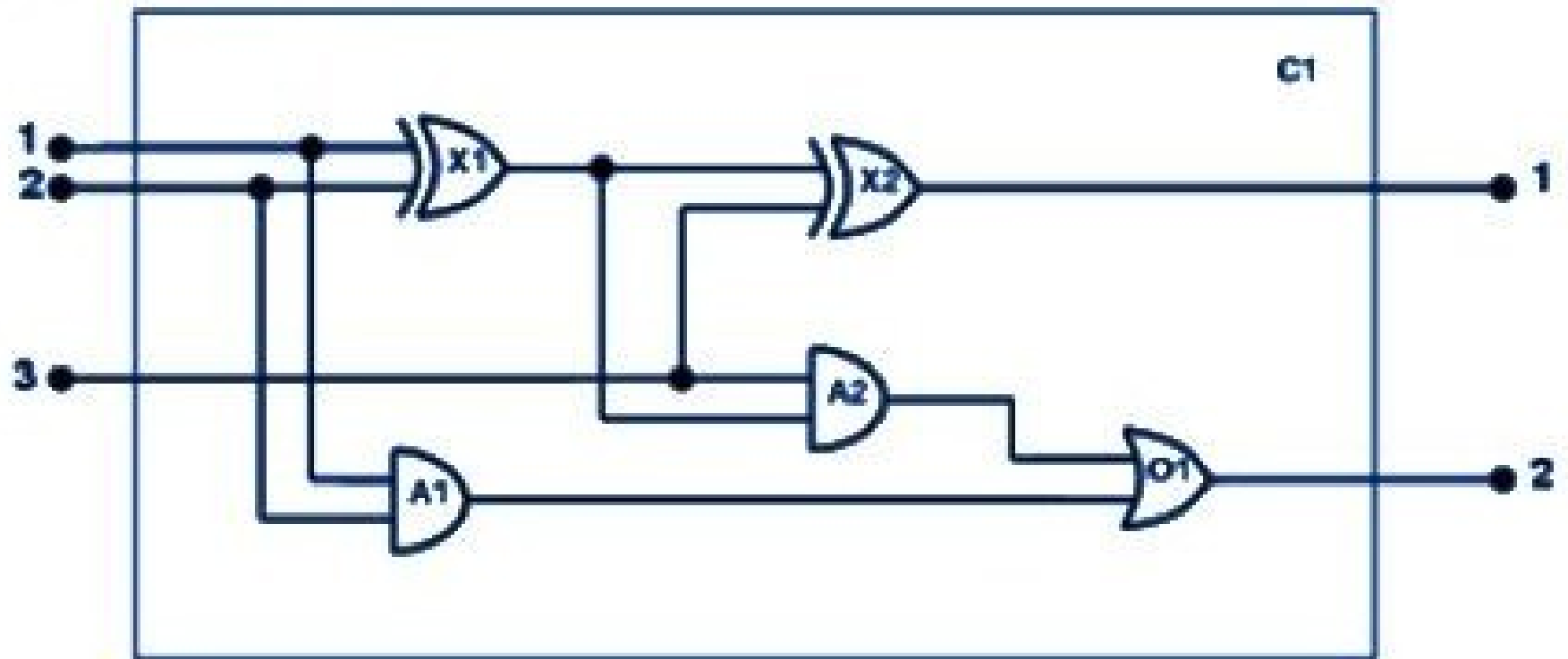
- First we categorize the gates:

Type(X_1) = XOR
Type(A_1) = AND
Type(O_1) = OR

Type(X_2) = XOR
Type(A_2) = AND

- We specify the Arity of the Circuit and each gate :

Arity($C1$, 3, 2)
Arity($X1$, 2, 1)
Arity($X2$, 2, 1)
Arity($A1$, 2, 1)
Arity($A2$, 2, 1)
Arity($O1$, 2, 1)

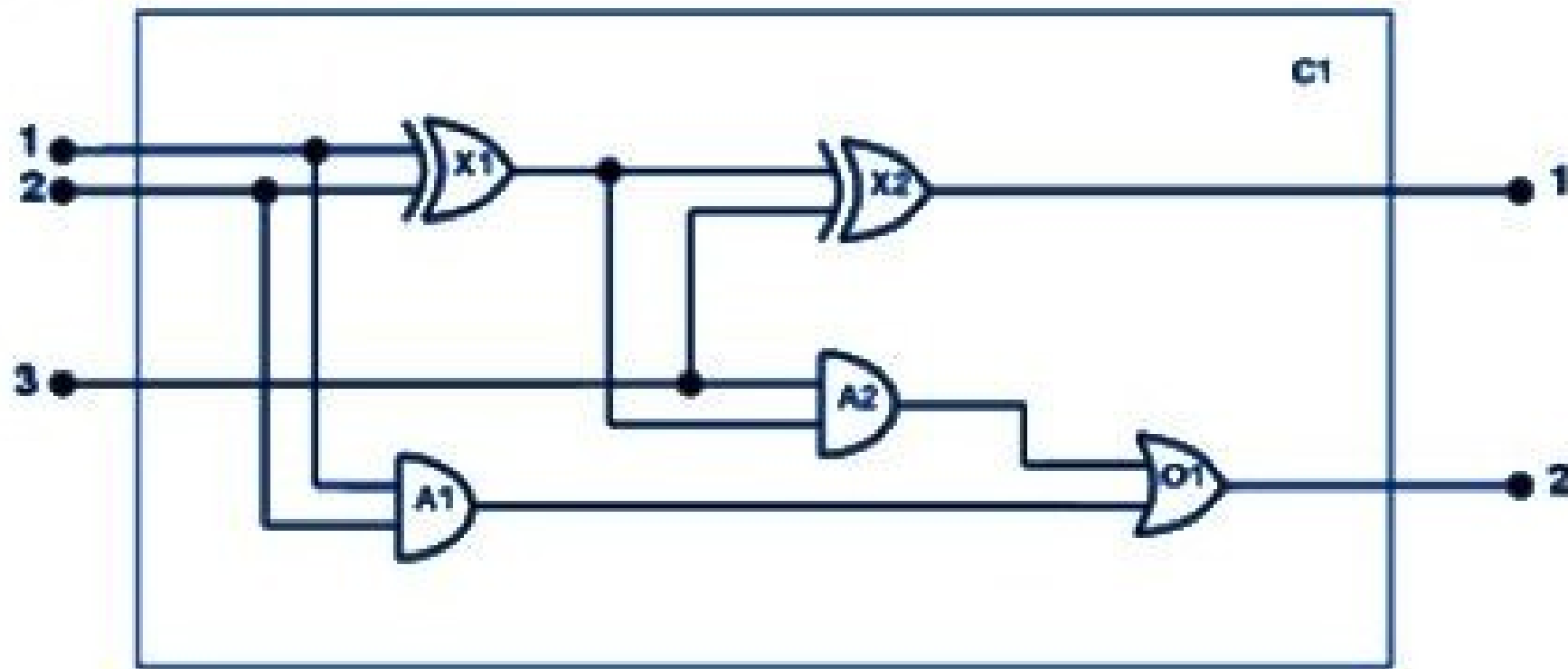


Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

5. Encode the specific problem instance

- Then show the connections between them:



Connected(In(1,C1), In(1, X1))
Connected(In(2,C1), In(2, X1))

Connected(In(1,C1), In(1, A1))
Connected(In(2,C1), In(2, A1))

Connected(Out(1,X1), In(1, X2))
Connected(In(3,C1), In(2, X2))

(
Connected(In(3,C1), In(1, A2))
Connected(Out(1,X1), In(2, A2))

(
Connected(Out(1,A2), In(1, O1))
Connected(Out(1,A1), In(2, O1))

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

7. Query and get answers

Query represented in FOPL:

$$\exists i1, i2, i3, o1, o2 \text{ signal}(\text{in}(1, c1)) = i1 \wedge \text{signal}(\text{in}(2, c1)) = i2 \wedge \text{signal}(\text{in}(3, c1)) = i3 \wedge \text{signal}(\text{out}(1, c1)) = o1 \wedge \text{signal}(\text{out}(2, c1)) = o2$$

In Prolog we can write a rule `find(i1, i2, i3, o1, o2)` to do the same:

`-? find(i1, i2, i3, o1, o2)`

Automata Formal Languages and Logic

Unit 4 - First-Order Predicate Logic

7. Query and get answers

We can examine output of each gate:

$\exists$ i1, i2, o

signal(in(1,x1)) = i1 $\wedge$

signal(in(2,x1)) = i2 $\wedge$

signal(out(1,x1)) = o



THANK YOU

Preet Kanwal

Department of Computer Science & Engineering

preetkanwal@pes.edu

+91 80 6666 3333 Extn 724